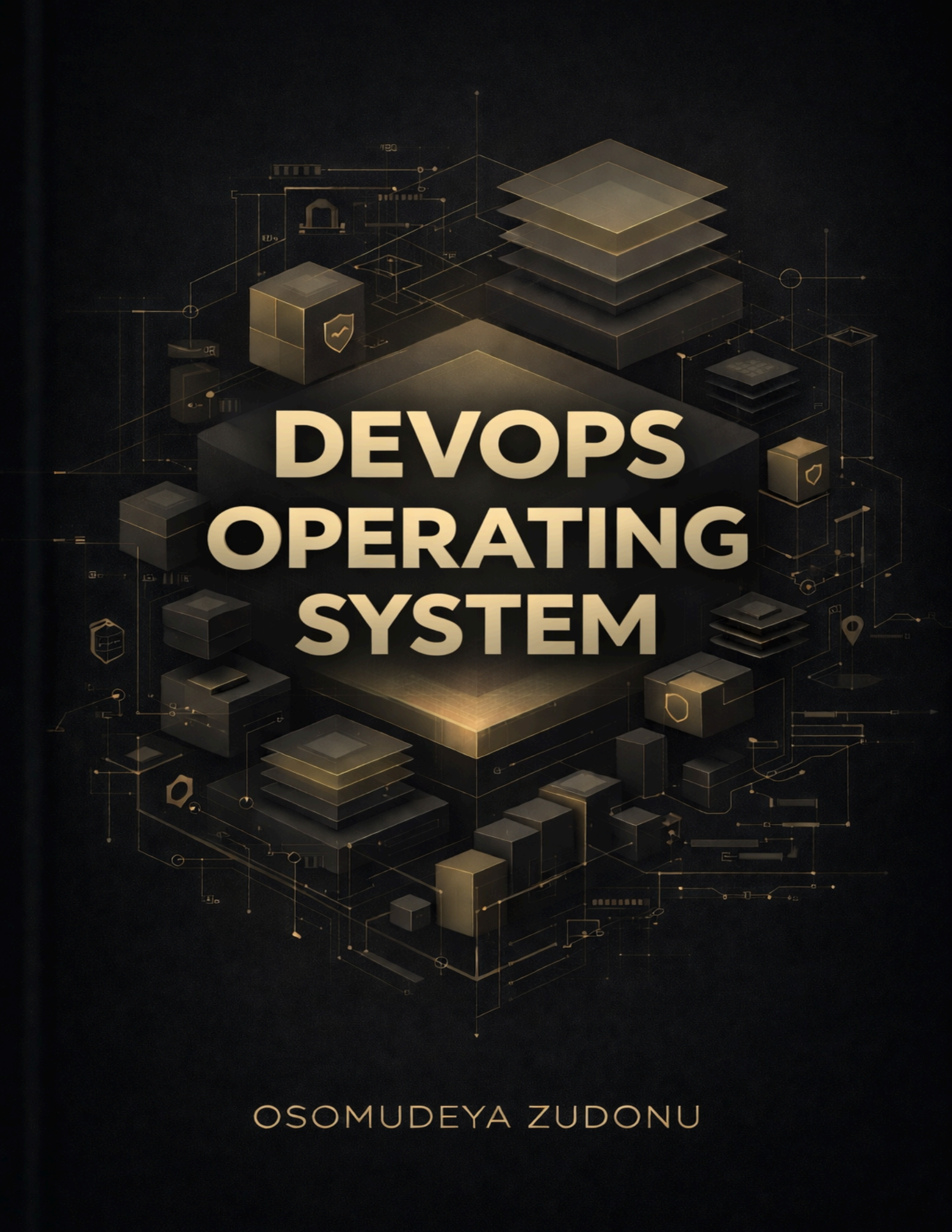




DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

DevOps Operating System (2026 Edition)

The Complete Roadmap from Absolute Beginner to Job-Ready DevOps Engineer

Linux • Docker • Kubernetes • Terraform • CI/CD • GitOps • Observability • DevSecOps
• AI-Assisted Workflows

A practical guide for anyone who wants DevOps skills but doesn't know where to start.

All Code Files & Labs Available On GitHub: [[repository-link](#)]

About This Book

You don't need years of experience or a CS degree. You need one clear path from zero to job ready.

This book is that path.

What you'll build:

- Linux, Docker/Kubernetes, Terraform/Ansible, CI/CD pipelines
- Monitoring stack (Prometheus, Grafana, ELK)
- Full capstone project proving you're job-ready

What you'll learn:

- Core DevOps technologies used daily
- Real production troubleshooting (not toy examples)
- DevSecOps, GitOps, AI-assisted workflows
- Portfolio that gets interviews

Timeline: 3-6 months (self-paced) |

Prerequisites: None

Perfect for: Beginners, career switchers, self-taught learners stuck in tutorial hell

No confusion. No scattered roadmaps. No tutorial hell.

One book. One path. Complete transformation.

Let's build your DevOps career.

Connect: talk2osomudeya@gmail.com | shipwithzee.com

LinkedIn: <https://www.linkedin.com/in/osomudeya-zudonu-17290b124/>

GitHub: <https://github.com/Osomudeya/humour-game-playbook>

DevOps Operating System (2026 Edition)

© 2025 Zudonu Osomudeya

All rights reserved. No part of this book may be reproduced without prior written permission, except for brief quotations in reviews. For permissions: talk2osomudeya@gmail.com

Disclaimer: This book is provided for educational purposes. Technology evolves rapidly tools and platforms may change after publication. The author is not providing professional career counseling or guaranteed job placement.

All code examples and configurations are provided "as is" without warranty. The author is not liable for career decisions, technical implementations, or outcomes from following this guidance. Test all configurations in non production environments. Back up your data.

Third party software, trademarks, and platforms mentioned are property of their respective owners. Reference does not imply endorsement.

Career Note: Job market conditions vary. This book provides skills training and portfolio guidance but does not guarantee employment. Success requires effort, practice, and active job searching.

Platform Note: Tools covered work across Mac, Windows, and Linux. Some labs may require specific systems, alternatives provided where applicable.

First Edition, January 2026

Published independently

TABLE OF CONTENTS

About This Book	2
DevOps Operating System (2026 Edition)	3
How To Use This Playbook	34
FOUNDATIONAL SKILLS	62

Who This Playbook Is For

The Problem You're Facing Right Now

You've learned DevOps tools through tutorials and courses, but you still don't feel job ready.

You follow steps, but building or troubleshooting from scratch is hard. That's tutorial loop.

This playbook is for you if:

- You're a **career switcher** aiming for DevOps, Cloud, or Platform roles
- You're a **junior/intern** who can run commands but lacks real understanding
- You're **overwhelmed** by massive DevOps roadmaps
- You're **unsure** what to learn first or what to skip
- You **want real, job** like projects, not toy demos

What you need is not more tutorials. You need a **clear learning order, real-world tasks, and projects you can explain in interviews.**

That's exactly what this playbook provides.

Preview: What "Real Work" Actually Looks Like

Here's an example of the kind of ticket you'll complete in Week 8:

TICKET: DEVOPS-456

Title: Containerize payments API and optimize image size

Priority: Medium

Assignee: You (Junior DevOps Engineer)

Business Context:

The payments service is currently deployed as a traditional app on EC2.

We're migrating to Kubernetes for better scaling and reliability.

Constraints:

- Final image must be <150MB (currently 850MB)
- Must use multi-stage builds
- Cannot change application code
- Must pass security scan (no HIGH/CRITICAL vulnerabilities)

Your Tasks:

1. Write production-ready Dockerfile
2. Optimize image size using multi-stage build
3. Scan with Trivy, fix vulnerabilities
4. Test locally, verify functionality
5. Document build process in README
6. Submit PR with before/after comparison

Deliverables:

- ✓ Optimized Dockerfile
- ✓ Trivy scan results (clean)
- ✓ Size comparison report (before: 850MB → after: <150MB)
- ✓ README with build instructions
- ✓ Git PR with changes and justification

Acceptance Criteria:

- Image builds successfully
- Image size <150MB
- No HIGH/CRITICAL vulnerabilities
- All tests pass
- Documentation clear enough for another engineer to replicate

This is what you'll actually be doing.

Not "follow this tutorial." Not "deploy hello-world." but, real tasks, real constraints, real deliverables.

By the end of this playbook, you'll have completed 15-20 tickets like this.

That's how you become job-ready.

What Makes You Job Ready (According to Actual Hiring Managers)

I've talked to lots of hiring managers, senior DevOps engineers, and platform leads. Here's what they actually look for when hiring junior DevOps engineers in 2026:

1. You can troubleshoot in production

Can you SSH into a VM, read logs, identify a failing service, and restore it? Can you explain your reasoning?

2. You understand the "why" behind tools

Can you explain *when* to use Docker vs a VM? *Why* your team chose Kubernetes over ECS? *What tradeoffs* you made when designing your CI/CD pipeline?

3. You can work in a GitOps workflow

Can you open a clean PR? Write a meaningful commit message? Review someone else's code? Rollback a deployment using Git history?

4. You think about security by default

Do you scan container images? Harden SSH? Use IAM roles instead of access keys? Check secrets into Git? (The answer should be "never".)

5. You can build observable systems

Can you add metrics, logs, and alerts to your services? Can you create a Grafana dashboard? Can you investigate a latency spike using Prometheus?

6. You can demonstrate real work

Do you have a GitHub profile with real projects? Can you show a runbook you wrote? A dashboard you built? A postmortem you documented?

This playbook teaches you all six of these competencies.

What You'll Actually Build in This Playbook

By the end of this playbook, you will have built:

A Production-Grade Home Lab That Includes:

- A containerized 3-service microservices platform
- Infrastructure provisioned with Terraform
- Kubernetes cluster with autoscaling and ingress
- Full CI/CD pipeline with security scanning
- Observability stack (Prometheus + Grafana)
- GitOps deployment with ArgoCD
- Runbooks, dashboards, and incident postmortems

A Portfolio That Includes:

- 6–8 real world projects with clean GitHub repos
- Architecture diagrams and README documentation
- Security scan results and remediation notes
- Demo videos showing your systems working
- Real JIRA style tickets you "completed"

Interview Ready Skills Including:

- Troubleshooting workflows you can explain aloud
- Hands on assessment practice (real take-home tasks)
- Behavioral answers for DevOps interview questions
- A resume template optimized for ATS systems
- A LinkedIn profile that gets recruiter attention

The Promise of This Playbook

If you follow this playbook:

You will **stop feeling overwhelmed** by the DevOps landscape

You will **stop doing toy projects** and start building real systems

You will **stop guessing** which tools to learn and in what order

You will **stop relying on AI** to do your thinking for you

You will **start thinking like a DevOps engineer**, not a tutorial follower

You will **build confidence** in your ability to solve real production problems

You will **have proof** of your skills that hiring managers respect

You will be job-ready in 6 months.

Not "tutorial complete." Not "certified." **Job ready.**

Ready?

This playbook will not be easy. It will require:

- 10–15 hours per week of focused work
- A willingness to break things and fix them
- Tolerance for frustration (you will get stuck)
- Commitment to building real systems, not skipping ahead

But if you're ready to do the work, this playbook will take you from "**I don't know where to start**" to "**I'm ready for my first DevOps role.**"

Let's begin.

SECTION 0

How To Use This Playbook

SECTION 0.1: The DevOps System Explained

Every tutorial starts with "Let's learn Docker!" without explaining where Docker fits in the bigger picture.

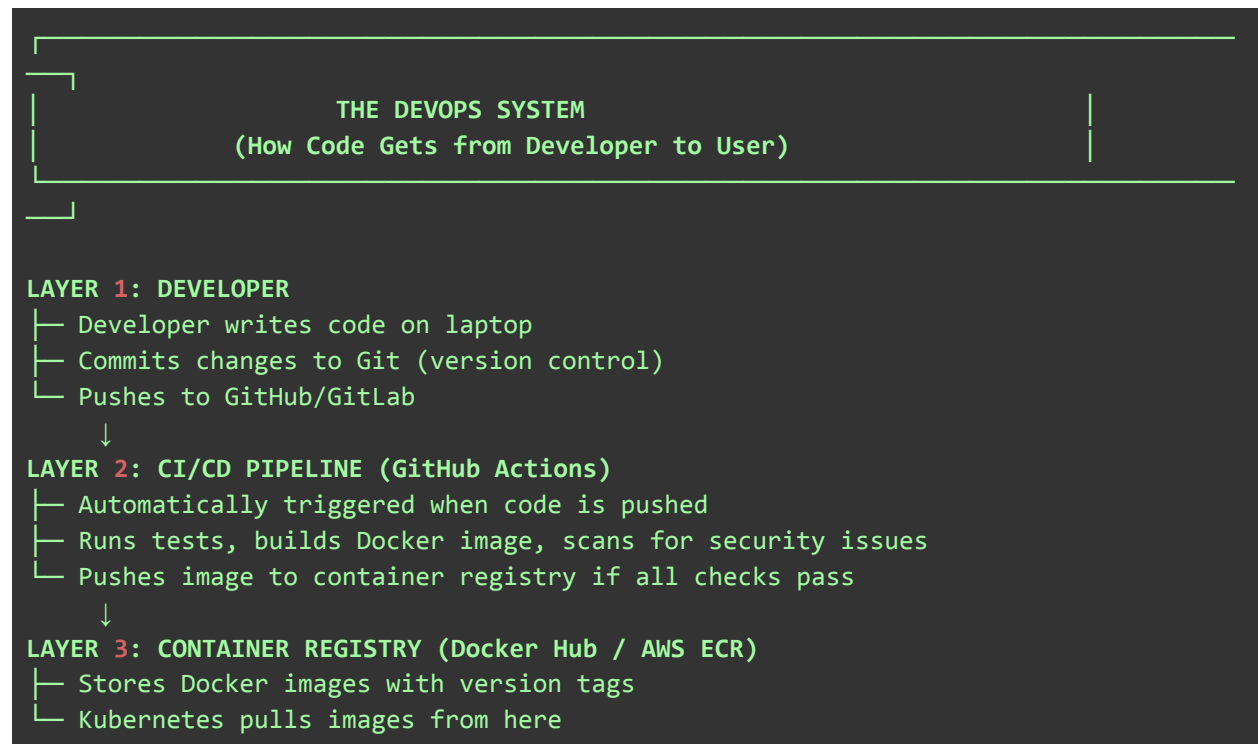
So you learn Docker in isolation. Then Kubernetes in isolation. Then Terraform in isolation.

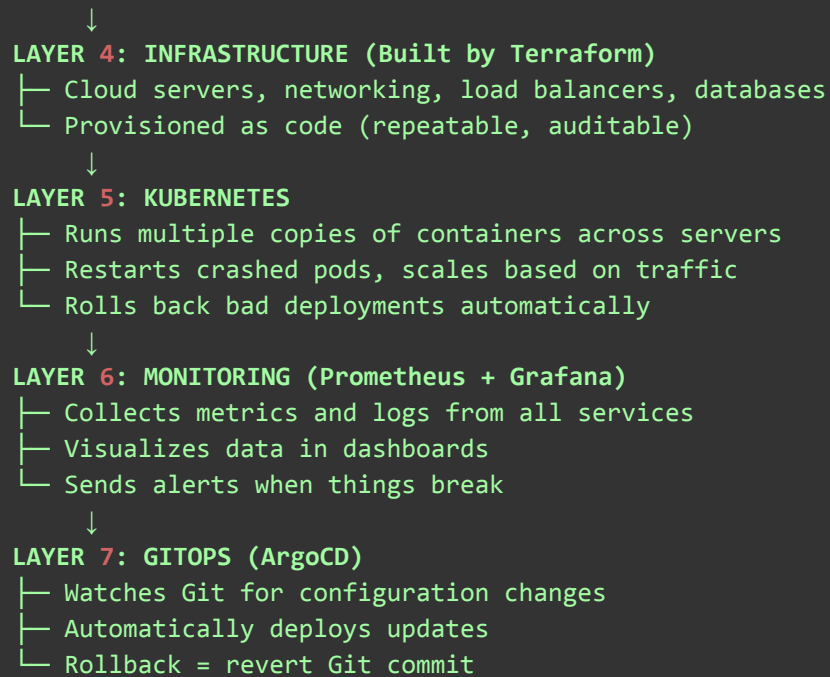
They never connect. The system never clicks.

This section fixes that.

The Complete DevOps System

Here's how code goes from a developer's laptop to production:





The Core 6 Tools You Actually Need

Master these six tools and you're job-ready:

1. **Linux** - The OS everything runs on
2. **Docker** - Packages apps into portable containers
3. **Kubernetes** - Runs containers at scale across servers
4. **Terraform** - Provisions cloud infrastructure as code
5. **GitHub Actions** - Automates testing and deployment
6. **Prometheus + Grafana** - Monitors systems and visualizes metrics

Everything else is either:

- Nice to have (ArgoCD, Helm, Ansible)
- An alternative (Jenkins instead of GitHub Actions, Azure instead of AWS)
- Learned later (service meshes, chaos engineering)

This book teaches the Core 6.

How the System Works

Step 1: Developer pushes code to Git

Step 2: CI/CD pipeline tests it and builds a Docker image

Step 3: Image is stored in a registry

Step 4: Kubernetes pulls the image and runs it on servers that Terraform built

Step 5: Prometheus monitors everything; Grafana shows dashboards

Step 6: When something breaks, engineers investigate and fix it

Step 7: GitOps automatically deploys the fix

The key insight: These tools aren't separate skills. They're parts of ONE system working together.

- Terraform builds the infrastructure
- CI/CD builds the Docker images
- Kubernetes runs the images on that infrastructure
- Monitoring watches Kubernetes
- GitOps automates deployments via Git
- **Git is the source of truth for everything**

By the end, you'll have a working DevOps platform where you can:

- ✓ Push code to Git
- ✓ Watch CI/CD automatically test and build it
- ✓ See Kubernetes deploy it across multiple servers
- ✓ Monitor it in real-time with Grafana dashboards
- ✓ Debug production issues using Prometheus metrics
- ✓ Roll back bad deployments with a Git revert

That's what "job ready" means.

Not memorizing commands. Not collecting certifications.

Building and operating real systems.

System Lock-In Check (Do Not Skip)

You've just seen the complete DevOps system, the one this entire book is built around.

Before moving on, confirm that it actually clicked.

Close the book.

On a blank page, recreate the flow you just saw:

- From developer commit
- Through CI/CD
- Into the container registry
- Onto infrastructure
- Through Kubernetes
- Into monitoring
- And back through GitOps

You don't need perfect wording, you need the correct flow.

If you struggle to recreate it, reread this section once and try again.

Do not continue until you can explain this system out loud without looking.

Every tool you learn next exists to serve one layer of this system.

SECTION 0.2: Why Traditional DevOps Roadmaps Fail

The Roadmap Problem

Traditional DevOps roadmaps overwhelm beginners by listing too many tools without priorities or learning order. This causes people to learn the wrong tools, start advanced topics too early, and build toy projects that don't translate to real jobs.

DevOps skills have strict dependencies and must be learned in sequence:

foundations → **containers** → **infrastructure** → **automation** → **observability** → **GitOps**. This playbook follows that order, teaches each tool with real world context, uses task based learning instead of tutorials, and ensures every project is portfolio-ready.

SECTION 0.3: Philosophy of This Playbook

This playbook is built on one core belief: **you learn DevOps by doing real work, not by watching tutorials**. Since beginners don't have access to real production systems, the playbook **simulates real DevOps jobs** using realistic tickets, constraints, acceptance criteria, and deliverables, exactly like what you'd get on day one at work.

You won't follow step-by-step tutorials. Instead, you'll **build, break, debug, fix, automate, and document** everything yourself. This forces real learning: struggling with problems, forming hypotheses, testing solutions, and understanding *why* things work, not just making them work.

AI is used as a **mentor, not a shortcut**. You try first, think deeply, then ask AI for explanations, not finished solutions. This ensures you can explain, modify, and debug your work confidently, including in interviews.

Finally, all learning happens in **enterprise grade labs**, not toy projects. By the end, you'll have built a full production style platform with infrastructure, Kubernetes, CI/CD, security, observability, and GitOps, giving you the same skills you'd use in a real DevOps role.

The 4-Level Learning Loop (How You Master Each Skill)

Every skill is learned the same way so it actually sticks.

Level 1: Concept (When & Why)

You learn where the tool fits, why it's used, and what it does *and doesn't* solve.

Level 2: Hands-On (How)

You build it from scratch, break it, debug it, and fix it in a lab or cloud setup.

Level 3: Real Tasks (Do the Job)

You complete realistic DevOps tickets with constraints and deliverables: PRs, docs, dashboards.

Level 4: Interview Proof

You explain your decisions, trade-offs, and troubleshooting clearly and confidently.

Result: You don't just use tools. You understand, apply, debug, and defend them.

How the Playbook Progresses

- **Weeks 1–4:** Foundations (Linux, Bash, Git, Networking)
- **Weeks 5–10:** Containers & Orchestration (Docker, Kubernetes)
- **Weeks 11–18:** Infrastructure & Security (Terraform, DevSecOps)
- **Weeks 19–24:** Automation & Observability (CI/CD, Monitoring, GitOps)

SECTION 0.4: Success System

Most people fail because they learn randomly. This playbook gives you a system.

- One focus per week
- Clear deliverables and “done” criteria
- Progress measured by real tasks.
- You don’t move on until all 4 learning levels are complete

You’re interview-ready when you can design systems, debug failures, and prove it with your portfolio.

Ready to Move Forward?

The AI Dependency Test (Weekly)

Pick one task you completed.

Delete it. Rebuild it:

- Without AI
- Without Google
- Without notes (docs allowed)

Results

- $<2\times$ original time $\rightarrow$ learned
- $2-3\times$ time $\rightarrow$ review
- $3\times$ or can't rebuild $\rightarrow$ you copied, redo the week

This is strict by design. You're training for real work, not tutorials.

The Socratic Debugging Method (Your Core Problem Solving Framework)

When you get stuck, use this framework:

Step 1: Observe

What is the actual behavior? (Copy exact error messages, logs)

Step 2: Hypothesize

What do I think is causing this? (List 2–3 possibilities)

Step 3: Test

How can I test my hypothesis? (Run a command, check a config, add a log)

Step 4: Validate

Did my test confirm or reject my hypothesis?

Step 5: Document

What did I learn? What's the fix?

Example:

Problem: Container won't start, exits immediately

Step 1 (Observe):

```
$ docker ps -a
CONTAINER ID   STATUS
a3f2bc         Exited (127) 2 seconds ago
```

Step 2 (Hypothesize):

- Hypothesis 1: Command in CMD or ENTRYPOINT doesn't exist
- Hypothesis 2: Missing dependency
- Hypothesis 3: Port already in use

Step 3 (Test Hypothesis 1):

```
$ docker logs a3f2bc
/bin/sh: /app/start.sh: not found
```

Step 4 (Validate): Hypothesis 1 confirmed. Script path is wrong or not copied into image.

Step 5 (Document): Fix: Add `COPY start.sh /app/` to Dockerfile.

Learning: Always check that files referenced in CMD exist in the image.

This is how you should debug.

The "Don't Touch Yet" List (Critical for Success)

These are advanced topics you should **ignore** until you've completed the core playbook:

Don't Touch Yet:

- Service mesh (Istio, Linkerd)
- Advanced Kubernetes operators

- Multi-cloud strategies
- Chaos engineering
- eBPF and kernel tracing
- Advanced networking (CNI deep-dives)
- Custom Kubernetes controllers
- Platform engineering frameworks (Backstage, etc.)

Why?

These are advanced topics that require solid fundamentals. If you try to learn them now, you'll get overwhelmed and confused.

You'll learn these later, after you've mastered the fundamentals and gotten your first job

SECTION 1

CLARITY FIRST

SECTION 1.1: What DevOps Actually Is in 2026

What DevOps Really Means

DevOps has many definitions because it looks different in every company. The constant is the goal: **help teams ship software faster and more reliably through automation and shared responsibility.**

DevOps started to fix the gap between developers (who want speed) and operations (who want stability). The solution was automation, infrastructure as code, CI/CD, monitoring, and shared ownership of production. That idea still holds today.

DevOps in 2026

DevOps is both a culture and a job role.

As a role, DevOps engineers build and maintain the systems developers use to ship code, CI/CD pipelines, cloud infrastructure, Kubernetes, monitoring, security scanning, and deployment automation. Their job is not just deploying apps, but **making deployments safe, repeatable, and fast.**

Related Roles (Brief)

- Cloud Engineers focus on cloud infrastructure and architecture.
- SREs focus on reliability, uptime, and performance at scale.
- Platform Engineers build internal platforms so developers can self-serve.
- In small companies, one DevOps engineer may do all of this. In larger companies, these roles are split.

Typical DevOps Setup

Most teams have a central DevOps or platform team that provides CI/CD, infrastructure, and tooling. Product teams use these tools to ship features. As companies grow, platform teams become more common.

Modern DevOps Stack

Git, CI/CD (GitHub Actions), Docker, Kubernetes, Terraform, GitOps (ArgoCD), monitoring (Prometheus/Grafana), security scanning, and one main cloud (usually AWS).

Common Myths Debunked

- Learn one cloud deeply, not all.
- Real projects matter more than certifications.
- DevOps is automation and systems, not console clicking.
- You don't need a CS degree, proof of skill matters.

Bottom Line

DevOps is about **building systems that make software delivery fast, reliable, and repeatable**. This playbook teaches the practical DevOps role you'll actually be hired for.

SECTION 1.2: Role Self-Assessment (Choose Your Path)

Why Choosing a Path Matters

“DevOps” is not one job. It includes several different roles with different focuses.

Beginners fail by trying to learn everything at once. The right move is to **pick one path, go deep, then expand later**.

The 4 Core Paths (2026)

Cloud Engineer

Designs and manages cloud infrastructure: networks, compute, databases, security, and costs.

DevOps Engineer

Automates delivery: CI/CD pipelines, Kubernetes, deployments, rollbacks, and developer tooling.

Platform Engineer

Builds internal platforms and self-service tools so teams can deploy without friction.

SRE

Keeps systems reliable at scale using metrics, alerts, error budgets, and automation. Usually mid-to-senior.

Small companies often combine these roles. Large companies separate them.

Quick Fit Guide

- Like infrastructure and networking → Cloud Engineer
- Like automation and pipelines → DevOps Engineer
- Like building tools for engineers → Platform Engineer
- Like reliability and incident response → SRE

Background Advantage Table

Your Background	Best Starting Path
SysAdmin / Ops	DevOps or SRE
Software Developer	Platform or DevOps
Networking / Security	Cloud Engineer
QA / Testing	DevOps
Non-tech	DevOps (start from fundamentals)

Key Takeaway

DevOps is a **family of roles**, not a single destination.

Pick one path, build real skills and projects, then branch out when you're ready.

Next: Self-assessment quiz → skill gaps → chosen path.

Self-Assessment Quiz

Answer these questions honestly:

Quick Quiz (Answer instinctively)

1. **What excites you most?**
 - A. Designing systems → Cloud Engineer
 - B. Automating pipelines → DevOps Engineer
 - C. Building tools for engineers → Platform Engineer
 - D. Solving outages → SRE

2. **How do you feel about on-call?**
 - A. Avoid it → Cloud / Platform
 - B. Occasional → DevOps
 - C. Regular / high-pressure → SRE

3. **What kind of work do you enjoy?**
 - A. Long-term foundations → Cloud / Platform
 - B. Fast delivery → DevOps
 - C. Internal products → Platform
 - D. Debugging failures → SRE

Scoring Your Assessment Count your answers:

Mostly A's: Cloud Engineer Path You'll thrive in infrastructure design and cloud architecture roles.

Mostly B's: DevOps Engineer Path You'll excel at automation, CI/CD, and enabling fast deployments.

Mostly C's: Platform Engineer Path You should focus on building internal platforms and developer tools.

Mostly D's: SRE Path You're suited for reliability engineering and incident response. Mixed results? Start with DevOps Engineer path, it's the most versatile and gives you exposure to all areas.

Skills Check (Be Honest)

Rate yourself 1–5:

1. Linux
2. Git
3. Networking
4. Docker
5. Kubernetes
6. Terraform
7. CI/CD
8. Cloud
9. Monitoring
10. Troubleshooting

Interpretation:

Mostly 1–2: Perfect! This playbook starts from fundamentals.

Mostly 3–4: You have some experience. You can move faster through early sections but don't skip fundamentals.

Mostly 5: You may be overqualified for this playbook. Consider focusing on advanced topics (SRE, platform engineering).

Background → Best Path

- SysAdmin → DevOps / SRE
- Developer → Platform / DevOps
- Networking → Cloud Engineer
- QA → DevOps
- Non-tech → DevOps (most accessible)

Path Selection Output

Based on your assessment, **choose ONE primary path:**

My chosen path: _____

Why this path fits me:

- _____
- _____
- _____

Skills I already have:

- _____
- _____

Skills I need to develop:

- _____
- _____
- _____

6-month goal: _____

SECTION 1.3: The 6-Month DevOps Strategy

The Realistic Timeline

Can you become job-ready in 6 months? Yes, if you:

- Study 10–15 hours per week consistently
- Follow a structured path
- Build real projects (not tutorials)
- Practice actively, not passively

You won't be senior in 6 months, but you *can* qualify for junior/entry-level DevOps roles with a strong portfolio.

The 6-Month High-Level Roadmap

```
Weeks 1-4   → Foundations (Linux, Bash, Git, Networking)
Weeks 5-10  → Containers (Docker + Kubernetes)
Weeks 11-16 → Infrastructure & Security (Terraform + DevSecOps)
Weeks 17-20 → CI/CD Automation
Weeks 21-24 → Observability (Monitoring & Alerting)
Weeks 25-26 → GitOps + Capstone + Job Prep
```

Each phase builds on the previous one. Nothing here is optional.

Phase Overview

Phase 1: Foundations (Weeks 1–4)

- **Focus:** Linux, Bash, networking, Git
- **Deliverables:** Ops runbook, automation scripts, hardened SSH
- **Checkpoint:** Troubleshoot servers and collaborate via Git
- **Time:** ~12 hrs/week

Phase 2: Containers & Kubernetes (Weeks 5–10)

- **Focus:** Docker, multi-stage builds, Kubernetes deployments

- **Deliverables:** Containerized apps running on K8s
- **Checkpoint:** Deploy, scale, and troubleshoot pods
- **Time:** ~12–15 hrs/week

Phase 3: Infrastructure & Security (Weeks 11–16)

- **Focus:** Terraform, cloud infrastructure, security scanning
- **Deliverables:** IaC-managed VPC, database, security pipelines
- **Checkpoint:** Provision infra safely and pass security checks
- **Time:** ~10–12 hrs/week

Phase 4: CI/CD Automation (Weeks 17–20)

- **Focus:** GitHub Actions, deployment strategies, rollbacks
- **Deliverables:** End-to-end automated pipeline
- **Checkpoint:** Build, test, deploy, and recover automatically
- **Time:** ~12 hrs/week

Phase 5: Observability (Weeks 21–24)

- **Focus:** Metrics, dashboards, alerts, SLOs
- **Deliverables:** Monitoring stack with incident workflows
- **Checkpoint:** Detect and diagnose issues before users do
- **Time:** ~12 hrs/week

Phase 6: GitOps + Capstone (Weeks 25–26)

- **Focus:** ArgoCD, environment promotion, full system integration
- **Deliverables:** Production-grade platform + portfolio
- **Checkpoint:** Explain and defend every design decision
- **Time:** ~15–20 hrs/week

End Result (After 6 Months)

You will have:

- A real, end-to-end DevOps platform
- Documented architecture and runbooks
- Proof you can deploy, secure, monitor, and recover systems
- Confidence to pass technical interviews

This is not fast learning.

It's deliberate, realistic, and employable learning.

Milestones & Checkpoints

Set clear milestones to track progress:

Milestone 1 (Week 4): Linux Proficiency

Checkpoint:

- ☐ Can troubleshoot failing services
- ☐ Can write Bash automation scripts
- ☐ Understand networking fundamentals
- ☐ Proficient with Git workflows

Deliverable: Linux operations portfolio repo

Milestone 2 (Week 10): Container Orchestration

Checkpoint:

- ☐ Can containerize applications
- ☐ Can deploy to Kubernetes
- ☐ Understand pod troubleshooting
- ☐ Can implement zero-downtime deployments

Deliverable: Kubernetes deployment portfolio repo

Milestone 3 (Week 18): Infrastructure & Automation

Checkpoint:

- ☐ Can provision infrastructure with Terraform
- ☐ Can build CI/CD pipelines
- ☐ Security scanning integrated
- ☐ Understand deployment strategies

Deliverable: Full CI/CD portfolio repo

Milestone 4 (Week 24): Observability & GitOps**Checkpoint:**

- ☐ Can monitor and alert on services
- ☐ GitOps workflow implemented
- ☐ End-to-end automation working
- ☐ Production-ready practices applied

Deliverable: Full observability stack + GitOps repo

Milestone 5 (Week 26): Job Ready**Checkpoint:**

- ☐ 6–8 portfolio projects
- ☐ Resume polished
- ☐ LinkedIn optimized
- ☐ Confident in mock interviews
- ☐ Applying to jobs

Deliverable: Job applications submitted

The "Must Learn" List (Non-Negotiable)

These are the absolute essentials. You cannot skip any of these:

Linux fundamentals (file systems, processes, systemd, logs)

Bash scripting (automation, functions, error handling)

Networking (DNS, TCP/IP, ports, firewalls, load balancers)

Git (branching, PRs, collaboration, Git workflows)

Docker (images, containers, multi-stage builds, security)

Kubernetes (deployments, services, ingress, health checks, HPA)

Terraform (providers, modules, state, security scanning)

CI/CD (GitHub Actions or GitLab CI)

Security scanning (Trivy, Checkov, SonarCloud)

Monitoring (Prometheus, Grafana, alerting)

Master these 11 areas and you're jobready.

SECTION 1.4: How to Learn DevOps with AI (Without Dependency)

AI in Learning

AI tools can speed up DevOps learning **3–5×** if used correctly.
Used incorrectly, they create dependency and fake confidence.

The goal is simple: **use AI as a mentor, not a shortcut.**

The Core Rule: Think → Try → Ask

Never ask AI first.

THINK → TRY → ASK

1. THINK

- What am I trying to do?
- What do I already know?
- What do I think is wrong?

2. TRY

- Attempt it yourself
- Read official docs
- Run commands, inspect logs
- Write something, even if wrong

3. ASK (only after 1 & 2)

- Explain what you tried
- Show your work
- Ask a specific question
- Ask for explanation, not a full solution

Skipping **THINK** or **TRY** guarantees shallow learning.

When AI Helps Most

Use AI for:

- Explaining concepts in simple terms
- Comparing tools and tradeoffs
- Debugging errors after you try
- Reviewing your approach
- Practicing interview explanations

Do **not** use AI to:

- Build full systems for you
- Write production-ready code from scratch
- Replace reading documentation
- Skip problem-solving effort

Safe AI Prompts (Good Learning)

Use prompts like these **after trying**:

- “My pod is CrashLoopBackOff. Here’s the error: [paste]. What should I check next?”
- “This Docker image is 1.2GB. Dockerfile: [paste]. How can I reduce size?”
- “This Bash script isn’t idempotent. Code: [paste]. How can I fix that?”
- “Terraform wants to recreate resources. Here’s the plan diff: [paste]. Why?”
- “My CI build passes locally but fails in GitHub Actions. Error: [paste]. What differs?”

These prompts:

- Show effort
- Focus on understanding
- Build real debugging skill

Dangerous Prompts (Avoid These)

These sabotage learning:

- “Write me a production Dockerfile”
- “Build me a complete CI/CD pipeline”
- “Generate Terraform for my infrastructure”
- “Fix this” (no context)
- “Explain everything about Kubernetes”

If AI does the thinking, **you don’t learn.**

Weekly AI Dependency Test

Run this once a week.

Test 1: Rebuild Without AI

Delete something you built and rebuild it without AI or notes.

If you can’t, you copied instead of learning.

Test 2: Explain Out Loud

Explain what you built, why it works, and what would break.

If you can’t explain it, you don’t understand it.

Test 3: Modify the Requirement

Change one thing (language, scale, security).

If you can’t adapt, you memorized instead of learned.

When to Use AI vs When to Struggle

Struggle when:

- Learning fundamentals
- Debugging basic errors
- Understanding why something works

Use AI when:

- You're blocked after real effort
- You need clarity, not answers
- You want feedback on your reasoning
- You're preparing for interviews

Struggle builds intuition.

AI accelerates learning **after** intuition starts forming.

Final Rule

If AI makes the task feel easy but you can't explain or modify the result, **you didn't learn it.**

Use AI to think better, not to think less.

The AI Learning Contract (Sign This)

Make a commitment to yourself:

I, [your name], commit to learning DevOps properly.

I will:

- Always try before asking AI
- Show my work when asking for help
- Rebuild projects from scratch weekly
- Explain my work aloud
- Read documentation before prompting AI
- Take the AI dependency test seriously

I will NOT:

- Copy-paste without understanding
- Ask AI to do my work for me
- Skip fundamentals
- Rely on AI for troubleshooting
- Use AI as a crutch

Signed: _____

Date: _____

Print this. Sign it. Put it above your desk.

SECTION 2

FOUNDATIONAL SKILLS

PHASE 1: Linux, Bash & Networking

SECTION 2.0: Choose Your Linux Practice Environment

You cannot learn Linux without a Linux environment.

Before we go into Linux concepts, troubleshooting, or real world scenarios, you need a **real Linux system** where you can:

- Run commands
- Break things safely
- Practice without fear
- Build muscle memory

This section helps you choose and set up your practice environment.

The Four Options

You have **four ways** to get a Linux environment:

Option	Best For	Cost	Internet Required	Difficulty
Virtual Box	Absolute beginners, offline learning	Free	Setup only	Easy
AWS EC2	Real-world experience, cloud learners	Free tier (12 mo)	Yes	Medium

Azure VM	Azure learners, enterprise path	Free tier (12 mo)	Yes	Medium
WSL2	Windows users	Free	Setup only	Very Easy

Which should you choose?

Choose VirtualBox if:

- You're an absolute beginner
- You want to learn completely offline
- You don't have a credit card or don't want to use cloud yet
- You want maximum safety (can't accidentally rack up bills)
- Your internet is unreliable

Choose AWS EC2 if:

- You're an absolute beginner
- You want real world cloud experience from day 1
- You're comfortable with credit cards (won't be charged in free tier)
- You have reliable internet
- You want to learn AWS (most common in jobs)
- You want realistic SSH experience

Choose Azure VM if:

- You're an absolute beginner
- You're targeting Microsoft heavy companies
- You have Azure credits or learning subscription
- Your company uses Azure
- Same benefits as AWS but Azure ecosystem

Choose WSL2 if:

- You're an absolute beginner
- You're on Windows and want the fastest setup
- You want Linux for daily commands/scripting

- You want to complement (not replace) a full VM
- **NOT recommended as your only environment** (limitations exist)

My Recommendation For this Course

AWS/Azure

- Real SSH experience
- Cloud fundamentals
- Realistic DevOps workflows

If you want VirtualBox You can check out [Full Guide Here](#)

Set Up Your AWS Linux Lab Environment

This playbook uses AWS EC2 as the standard learning environment so you gain real Linux and cloud experience at the same time. AWS is widely used in DevOps roles, offers a free tier, and gives you real SSH access in a realistic environment. If something breaks, you can easily rebuild and continue.

What you'll have by the end:

- A secured AWS account with billing alerts
- A free-tier EC2 Linux server (Ubuntu)
- SSH access from your laptop
- A reusable setup for all Linux tasks

Requirements: computer, email, credit/debit card (no charges if you follow the guide), stable internet.

Time needed: ~60 minutes, one-time setup.

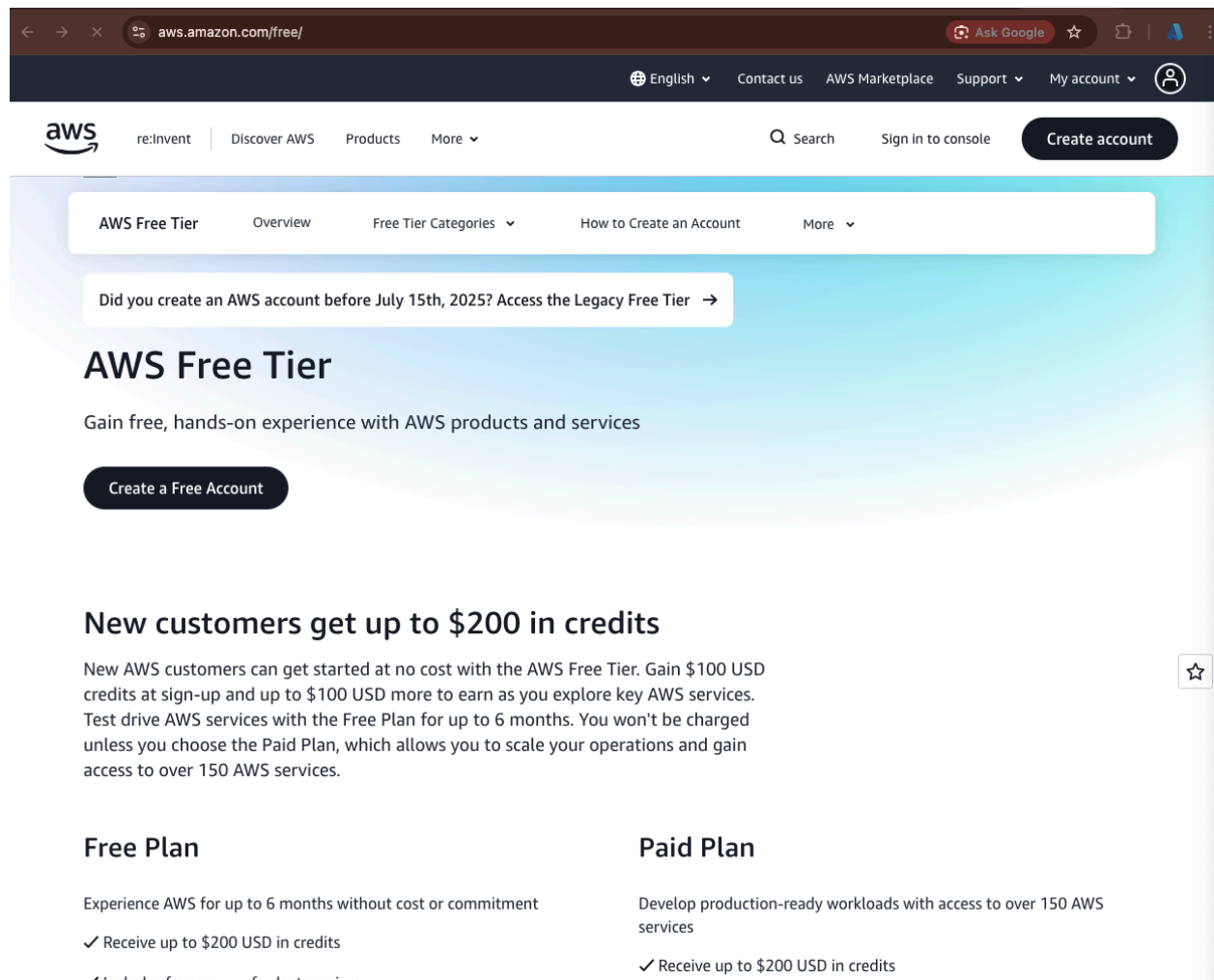
Step by Step AWS Setup

Step 1: Create AWS Account

1.1 Go to AWS:

<https://aws.amazon.com/free/>

1.2 Click "Create a free AWS Account" (top right)



The screenshot shows the AWS Free Tier landing page. At the top, there's a navigation bar with the AWS logo, 're:Invent', 'Discover AWS', 'Products', and a 'More' dropdown. On the right, there are links for 'Contact us', 'AWS Marketplace', 'Support', 'My account', and a 'Create account' button. Below the navigation bar, there's a section titled 'AWS Free Tier' with a sub-header 'Gain free, hands-on experience with AWS products and services'. A prominent 'Create a Free Account' button is visible. Below this, there's a section titled 'New customers get up to \$200 in credits' with a detailed description of the offer. At the bottom, there are two columns: 'Free Plan' and 'Paid Plan', each with a description and a list of benefits.

Did you create an AWS account before July 15th, 2025? Access the Legacy Free Tier →

AWS Free Tier

Gain free, hands-on experience with AWS products and services

[Create a Free Account](#)

New customers get up to \$200 in credits

New AWS customers can get started at no cost with the AWS Free Tier. Gain \$100 USD credits at sign-up and up to \$100 USD more to earn as you explore key AWS services. Test drive AWS services with the Free Plan for up to 6 months. You won't be charged unless you choose the Paid Plan, which allows you to scale your operations and gain access to over 150 AWS services.

Free Plan	Paid Plan
Experience AWS for up to 6 months without cost or commitment	Develop production-ready workloads with access to over 150 AWS services
✓ Receive up to \$200 USD in credits	✓ Receive up to \$200 USD in credits
✓ Includes free usage of select services	

1.3 Account Details:

- **Email address:** Your email (you'll receive verification codes here)
- **Password:** Create a strong password
 - At least 8 characters
 - Mix of uppercase, lowercase, numbers, symbols
 - **Save in password manager** (1Password, Bitwarden, LastPass)
- **AWS account name:** Choose a name (e.g., "John DevOps Learning")

Click "Continue"

1.4 Contact Information:

- **Account type:** Select **Personal**
- **Full name:** Your name
- **Phone number:** Your mobile (for verification)
- **Country/Region:** Your country
- **Address:** Your address
- **Postal code:** Your postal code

Check the AWS Customer Agreement box

Click "Continue"

1.5 Payment Information:

READ THIS CAREFULLY:

- AWS requires a credit/debit card for verification
- **You will NOT be charged** if you:
 - Stay within free tier limits (we'll ensure this)
 - Set up billing alerts (we'll do this next)
 - Follow this playbook's guidelines
- AWS will temporarily charge **\$1 USD** for verification (refunded immediately)

Enter your card details and click "Verify and Continue"

1.6 Identity Verification:

- **Phone verification method:** Choose "Text message (SMS)" or "Voice call"
- Enter your phone number
- Click "Send SMS" or "Call me now"
- **Enter the 4-digit verification code** you receive
- Click "Continue"

1.7 Select Support Plan:

- Choose **Basic support - Free**
- Click "Complete sign up"

1.8 Wait for account activation:

You'll see: "Congratulations! Your account is being activated."

- This usually takes **5-10 minutes**
- You'll receive an email: "Welcome to Amazon Web Services"
- **Check your email before proceeding**

Checkpoint 1: You now have an AWS account!

Step 2: Secure Your AWS Account (CRITICAL - DO NOT SKIP)

 **THIS IS THE MOST IMPORTANT SECTION** 

Why this matters:

Without proper security:

- Someone could access your account
- Rack up thousands of dollars in charges
- Launch cryptocurrency miners using your account
- **This happens regularly to unsecured accounts**

We'll prevent this with 3 security measures:

1. Multi-Factor Authentication (MFA)
2. Billing alerts
3. CloudWatch cost alarms

2.1 Sign in to AWS Console:

Go to: <https://console.aws.amazon.com/>

- Click "Sign In"
- Select "Root user"
- Enter your email
- Enter your password
- Click "Sign in"

You should see the AWS Management Console (dashboard).

The screenshot shows the AWS Management Console dashboard for the 'us-east-1' region. The top navigation bar includes the AWS logo, a search bar, and account information (United States (N. Virginia), Account ID: 3340-9176-9766, shipwithzee). The dashboard is divided into several sections:

- Console Home**: Includes a 'Reset to default layout' button and an 'Add widgets' button.
- Recently visited**: A list of services including Billing and Cost Management, EC2, AWS Health Dashboard, DynamoDB, S3, VPC, Elastic Kubernetes Service, Elastic Container Registry, IAM, AWS Organizations, Key Management Service, AWS Resource Explorer, Elastic Container Service, and Resource Groups & Tag Editor. A 'View all services' link is at the bottom.
- Applications (0)**: A section for managing applications, showing 'Region: US East (N. Virginia)' and a 'Create application' button. It includes a search bar and a table with columns for Name, Description, Region, and Order. A message states 'No applications. Get started by creating an application.' with a 'Create application' button and a 'Go to myApplications' link.
- Welcome to AWS**: A section with links for 'Getting started with AWS', 'Training and certification', and 'AWS Builder Center'.
- AWS Health**: A section showing 'Open issues' (0), 'Scheduled changes' (0), and 'Other notifications' (0), all for the 'Past 7 days'. A 'Go to AWS Health' link is at the bottom.
- Cost and usage**: A section showing 'Credits remaining' (\$147.25 USD), 'Days remaining' (134 days, Apr 12, 2026), 'Current month' cost (\$7.08, up 24%), and 'Forecasted month end' cost (\$7.14, up 25%). It includes a bar chart showing costs over time (Jun 25, Aug 25, Oct 25) and a 'Go to Billing and Cost Management' link.
- Solutions (13)**: A section showing 'Vetted Solutions from AWS for popular business and technical use cases.' with tabs for 'Cloud Architect (4)', 'Platform Engineer (4)', and 'AI Ops'.
- Explore AWS**: A section with links for 'Amazon Redshift' and 'Amazon S3'.

Account ID: 3340-9176-9766

shipwithzee

[Option+S]



Global ▾

action > Overview

W Info

Cost Anomaly Detection works

When detecting cost anomalies, you need to perform the following steps.

Cost Anomaly Detection summary

Costs detected (MTD):

Total cost impact (MTD):

Total savings (MTD):

\$0.18

\$7.00

Cost anomalies

Cost monitors

Alert subscriptions

Cost anomalies (2) Info

When a cost anomaly is detected throughout an anomaly's duration, the total cost impact can increase or decrease up to three times daily.

Cost anomalies detected by property or value

Cost anomaly ID	Last detected	Duration	Cost impact	Root cause	Default	View 1 potential root cause(s)
02	2025-11-03	2 days	\$0.12	N/A	Default	View 1 potential root cause(s)
02	2025-11-02	1 day	\$0.06	N/A	Default	View 1 potential root cause(s)

Free plan status

Credits remaining

Days remaining

\$147.25 USD

134 days

Your free access to AWS services will end on Apr 12, 2026 or when you have depleted all credits. To ensure uninterrupted AWS access, see [upgrading your plan](#) for details.

Account ID

Account color

 3340-9176-9766

☐ Unset

Account

Organization

Service Quotas

Billing and Cost Management

Security credentials

Console Mobile App

Turn on multi-session support

Sign out

b) Scroll to "Multi-factor authentication (MFA)"

c) Click "Assign MFA device"

d) Configure MFA:

- **Device name:** "MyPhone" (or your device name)
- **MFA device:** Select "Authenticator app"
- Click "Next"

My security credentials Root user Info

The root user has access to all AWS resources in this account, and we recommend following [best practices](#). To learn more about the types of AWS credentials and how they're used, see [AWS Security Credentials](#) in AWS General Reference.

Account details Actions

Account name
shipwithzee

Account ID
 334091769766

Email address
shipwithzee@protonmail.com

Canonical user ID
 bb631b0f5e735b85186e56dd063f5fb36ca74f0b4ebc11d5379eb16222c895a2

Multi-factor authentication (MFA) (1) Remove Resync Assign MFA device

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

Type	Identifier	Certifications	Created on
☐ Virtual	arn:aws:iam::334091769766:mfa/shipwithzee	Not Applicable	Sun Oct 12 2025

Access keys (0) Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

e) Set up authenticator app:

On your phone:

- Download **Google Authenticator** (iOS/Android)
 - Or use: Authy, Microsoft Authenticator, 1Password

Back in AWS:

- You'll see a QR code

- Open authenticator app → Tap "+" → Scan QR code
- The app will show a 6-digit code (changes every 30 seconds)

f) Enter codes:

- **MFA code 1:** Enter the current 6-digit code
- **Wait 30 seconds** for code to refresh
- **MFA code 2:** Enter the new code
- Click "Add MFA"

Success message: "You have successfully assigned virtual MFA"

g) Save recovery codes (CRITICAL):

- Click "Security credentials" in breadcrumb
- Scroll to MFA → Click on your device
- Click "Actions" → "Retrieve MFA recovery codes"
- **Download and save these codes** in a safe place (password manager or printed)
- These are your backup if you lose your phone

Checkpoint 2: Your account is now protected with MFA!

Test it: Sign out and sign back in. You'll now need both password AND MFA code.

2.3 Set Up Billing Alerts (Prevent Surprise Charges):

This section prevents accidental AWS bills.

a) Go to Billing Dashboard:

- Top navigation bar → Click your account name → "**Billing and Cost Management**"
- Or go to: <https://console.aws.amazon.com/billing/>

b) Enable billing alerts:

- Left sidebar → Click "**Billing preferences**"

- Under "Alert preferences", check these boxes:
 - **"Receive Free Tier Usage Alerts"**
 - **"Receive Billing Alerts"**
- **Email:** Your email should be pre-filled
- Click **"Save preferences"**

c) Create CloudWatch Billing Alarm:

This sends you an email if your bill exceeds a threshold (e.g., \$5).

PREREQUISITE: You **MUST** complete **step 2.3 (b) "Enable billing alerts"** **FIRST** before billing metrics will appear in CloudWatch!

If you already enabled billing alerts in step (b), wait 15 minutes and then proceed.

Verify Billing Alerts are Enabled:

1. Go to: <https://console.aws.amazon.com/billing/>
2. Left sidebar → Click "Billing preferences"
3. Verify these boxes are CHECKED:
 - "Receive Free Tier Usage Alerts"
 - "Receive Billing Alerts"
4. If they weren't checked, check them now and click "Save preferences"
5. Wait 15 minutes for AWS to create billing metrics

Now Create the Alarm:

Go to CloudWatch:

- Services menu → Search "CloudWatch" → Click it
- Or go to: <https://console.aws.amazon.com/cloudwatch/>
- Top right corner → Click region dropdown
- Select "US East (N. Virginia) us-east-1"
- Billing metrics **ONLY** exist in us-east-1

Create alarm:

- Left sidebar → Click "Alarms" → "All alarms"
- Click "Create alarm" (orange button)

Step 1: Specify metric

- Click "Select metric"
- In the search box at the top, type: **Billing**
- If you see "Billing" category appear: Click on it
- If you DON'T see "Billing": STOP - Troubleshoot first:
 1. Verify you're in us-east-1 (N. Virginia) top right corner
 2. Go back to Billing Preferences and verify alerts are enabled
 3. Wait 15 minutes after enabling billing alerts
 4. Try clearing browser cache or using incognito/private window
 5. Sign out and sign back in
 6. If still not appearing after 30 minutes, it may be a new account delay - try again in a few hours

Once you see Billing category:

- Click "Billing"
- Click "Total Estimated Charge"
- You'll see a row with Currency: USD
- Check the checkbox next to that row
- Click "Select metric" (bottom right)

Step 2: Configure conditions

- Statistic: Average (default)
- Period: 6 hours (default)
- Threshold type: Static
- Whenever EstimatedCharges is...: Greater
- than...: Enter **5**
- Click "Next"

Step 3: Configure actions

- Alarm state trigger: In alarm

- Select an SNS topic: Create new topic
- Topic name: **Billing-Alerts**
- Email endpoints: Your email address
- Click "Create topic"
- Click "Next"

Step 4: Name and description

- Alarm name: **Billing-Alert-5USD**
- Alarm description: **Alert when AWS bill exceeds \$5**
- Click "Next"

Step 5: Preview and create

- Review settings
- Click "Create alarm"

d) Confirm email subscription:

1. Check your email (including spam folder)
2. Subject: "AWS Notification - Subscription Confirmation"
3. Click "Confirm subscription" link
4. You'll see: "Subscription confirmed!"

Alternative: If Billing Metrics Still Don't Appear

You can use AWS Budgets instead (simpler and works immediately):

1. Go to: <https://console.aws.amazon.com/billing/home#/budgets>
2. Click "Create budget"
3. Select "Use a template (simplified)"
4. Choose "Zero spend budget"
5. Budget name: **My-Zero-Spend-Budget**
6. Email: Your email
7. Click "Create budget"

This will alert you if you have ANY charges at all (even \$0.01), which is even safer than the \$5 alarm.

My Recommendation:

Use AWS Budgets (alternative method above) instead of CloudWatch billing alarms.

Why?

- Works immediately (no 15-minute wait)
- Simpler to set up
- Zero spend budget catches charges faster
- More reliable for new accounts

You can always add CloudWatch billing alarms later once the metrics appear.

Checkpoint 3: You now have billing protection!

You'll receive email if charges exceed \$5.

Step 3: Understand AWS Free Tier

What you get for **FREE** (12 months from account creation):

EC2 (Virtual Servers):

- **Instance type:** t2.micro (1 vCPU, 1GB RAM)
- **Hours per month:** 750 (run 1 instance 24/7)
- **Operating System:** Linux/Ubuntu
- **Duration:** 12 months

EBS Storage (Disk):

- **Amount:** 30 GB General Purpose (SSD)
- **Duration:** 12 months

Data Transfer:

- **Outbound:** 1 GB/month
- **Inbound:** Always free
- **Duration:** 12 months (then ongoing)

Snapshots/AMIs:

- 1 GB of snapshots
- Duration: 12 months

How to stay within free tier (RULES):

DO:

- Use **t2.micro only** (not t2.small, t2.medium, etc.)
- Use **≤ 30GB storage** total
- **Stop instance when not using it** (evenings, weekends)
- Keep **1 instance running** at a time
- Use **Linux/Ubuntu** (Windows costs extra)

- Delete unused snapshots

Cost breakdown if you exceed free tier:

- t2.micro: ~\$0.0116/hour (~\$8.50/month if running 24/7)
- Storage: ~\$0.10/GB/month
- Data transfer: ~\$0.09/GB

scenario:

- 1 t2.micro running 24/7 = \$0 (within free tier)
- 2 t2.micro running 24/7 = ~\$8.50/month (one exceeds free tier)

We'll stay well within limits in this playbook.

Checkpoint 4: You understand free tier limits!

Step 4: Launch Your EC2 Instance (Your Linux Server)

Now we'll create your actual Linux server.

4.1 Go to EC2 Console:

- Services menu → Search "EC2" → Click "EC2"
- Or go to: <https://console.aws.amazon.com/ec2/>

4.2 Select Your Region:

IMPORTANT: Choose a region and stick with it.

- Top right corner → Click region dropdown
- **Recommended regions:**
 - **US East (N. Virginia)** - us-east-1 (most services available)
 - **US West (Oregon)** - us-west-2 (good alternative)
 - Or: Choose the region **closest to you** geographically

Write down your region. Use it for all future AWS work.

4.3 Launch Instance:

- Left sidebar → Click **"Instances"**
- Click orange button: **"Launch instances"**

4.4 Configure Instance (Step-by-Step):

Name and Tags

Name: devops-lab-vm

Application and OS Images (AMI)

- **Quick Start:** Already selected
- Select: **Ubuntu**
- **AMI:** Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
- **Architecture:** 64-bit (x86)
- **Free tier eligible** ✓ (you should see this label)

Quick Start

Amazon Linux, macOS, **Ubuntu**, Windows, Red Hat, SUSE Linux, [Browse more AMIs](#)

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-0ecb62995f68bb549 (64-bit (x86)) / ami-01b9f1e7dc427266e (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Description
Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).
Canonical, Ubuntu, 24.04, amd64 noble image

Architecture	AMI ID	Publish Date	Username	Verified provider
64-bit	ami-0ecb62995f68bb549	2025-10-22	ubuntu	Verified provider

Instance Type

- **Instance type:** t3.micro

- 1 vCPU
- 1 GiB Memory
- **Free tier eligible** ✓ ← VERIFY THIS

CRITICAL: Must be t3.micro for free tier! Or t2.micro

Key Pair (login)

This is how you'll access your server via SSH.

- Click "Create new key pair"

Create key pair popup:

- **Key pair name:** `devops-lab-key`
- **Key pair type:** RSA
- **Private key file format:**
 - macOS/Linux: **.pem**
 - Windows: **.pem** (we'll use Git Bash or PowerShell)
- Click "Create key pair"

File downloads: `devops-lab-key.pem`

SAVE THIS FILE SECURELY:

- You cannot download it again
- Anyone with this file can access your server
- Move it to `~/ .ssh/` folder later

Network Settings

What are Security Groups? Think of security groups as a **virtual firewall** for your EC2 instance. They control what network traffic is allowed to reach your server and what traffic can leave it.

Key Concepts:

- **Inbound rules:** Control what traffic can reach your server FROM the internet

- **Outbound rules:** Control what traffic can leave your server TO the internet (we'll use defaults)
- **Port:** A numbered gateway on your server (like apartment numbers in a building)
- **Protocol:** The "language" used for communication (TCP, UDP, ICMP)
- **Source:** Who is allowed to connect (your IP, anywhere, specific networks)

Why we need these specific rules:

1. **SSH (Port 22):** This is how YOU will connect to your server from your laptop
2. **HTTP (Port 80):** This allows people to visit websites you host (insecure)
3. **HTTPS (Port 443):** This allows people to visit websites you host (secure)

Security Best Practice:

- SSH should ONLY be allowed from YOUR IP address
- Web traffic (HTTP/HTTPS) can be allowed from anywhere since websites are meant to be public

Configuration Steps:

Click "**Edit**" button on the right

VPC:

- Leave as default

Subnet:

- No preference (default)

Auto-assign public IP:

- **Enable ✓**

Firewall (security groups):

- Select: "**Create security group**"

Security group name: devops-lab-sg

Description: Security group for DevOps learning lab

Inbound security group rules:

Rule 1 - SSH (for your access):

- **Type:** SSH
- **Protocol:** TCP
- **Port range:** 22
- **Source type:** My IP ← This restricts SSH to only your IP address
- **Description:** "SSH from my laptop"

Why "My IP" for SSH?

- SSH gives you complete control over the server
- If you allow "Anywhere", hackers will try to break in
- "My IP" means only your current internet connection can SSH
- You can always update this if your IP changes

Rule 2 - HTTP (for web servers):

- Click "Add security group rule"
- **Type:** HTTP
- **Protocol:** TCP
- **Port range:** 80
- **Source type:** Anywhere IPv4 (0.0.0.0/0)
- **Description:** "HTTP traffic"

Why "Anywhere" for HTTP?

- Web servers are meant to be accessed by everyone
- Port 80 is the standard port for websites
- This is NOT a security risk - web servers are designed to be public

Rule 3 - HTTPS (for secure web):

- Click "**Add security group rule**"
- **Type:** HTTPS
- **Protocol:** TCP
- **Port range:** 443
- **Source type:** Anywhere IPv4 (0.0.0.0/0)
- **Description:** "HTTPS traffic"

Why "Anywhere" for HTTPS?

- Port 443 is for secure (encrypted) web traffic
- Modern websites should use HTTPS
- Like HTTP, this is meant to be publicly accessible

Security Summary:

- **SSH (22):** Only YOU can connect ✓
- **HTTP (80):** Anyone can visit your websites ✓
- **HTTPS (443):** Anyone can visit your secure websites ✓
- **Everything else:** BLOCKED by default ✓

Configure Storage

- **Volume 1 (Root):**
 - Size: **8 GiB** (we don't need much for learning)
 - Volume type: **gp3** (General Purpose SSD - latest)
 - **Delete on termination:** Checked
 - **Encrypted:** Leave unchecked (not needed for lab)

Advanced Details

- Leave all as default (we don't need to change anything)

4.5 Review and Launch

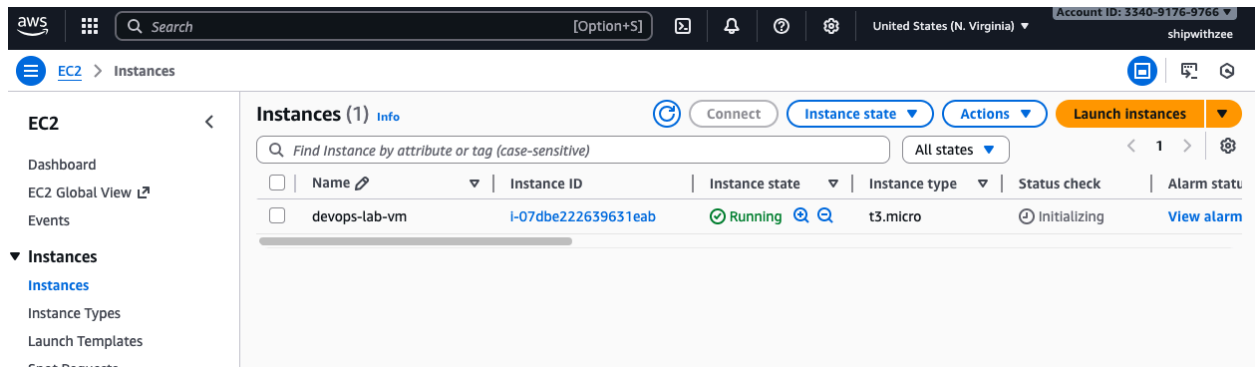
- Review all settings
- Click "Launch instance"
- You'll see: "Successfully initiated launch of instance..."

- Click "View all instances"

4.6 Wait for Instance to Start

- **Instance State:** Should change from "Pending" → "Running"
- **Status Checks:** Should show "2/2 checks passed"
- This takes 2-3 minutes

Checkpoint 5: Your Linux server is running in AWS!

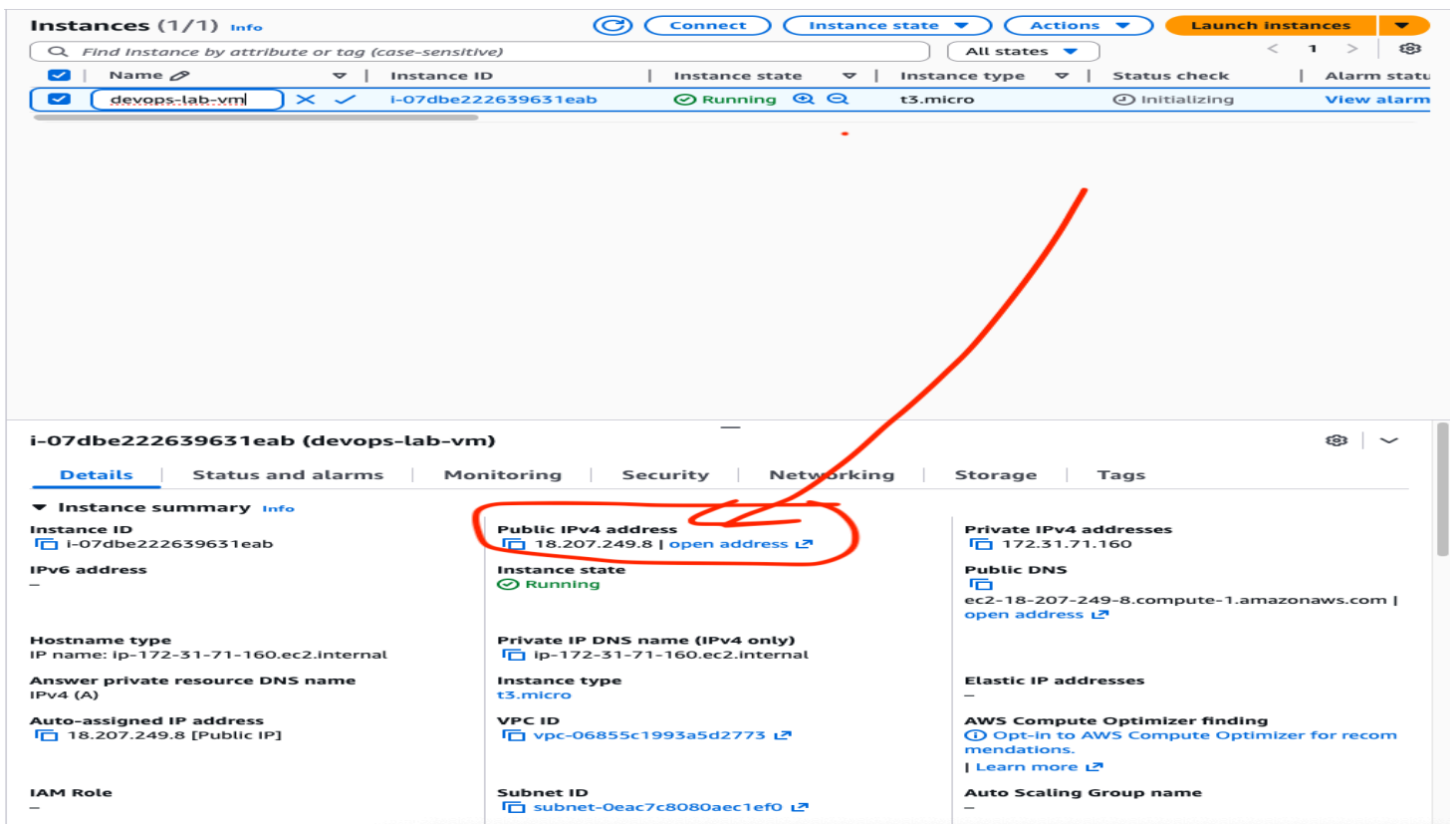


Step 5: Connect to Your Instance (SSH Setup)

Now let's connect to your server from your computer.

5.1 Get Connection Details:

- In EC2 Instances list, **click on your instance** (checkbox)
- In the details panel below, find:
 - **Public IPv4 address:** (e.g., 54.123.45.67)



- **Copy this IP address** (you'll need it)

5.2 Set Up SSH Key (Depends on Your OS):

For macOS / Linux Users:

Open Terminal

```
# Move the key to SSH directory
mv ~/Downloads/devops-lab-key.pem ~/.ssh/

# Set correct permissions (SSH requires this for security)
chmod 400 ~/.ssh/devops-lab-key.pem

# Verify permissions
ls -l ~/.ssh/devops-lab-key.pem
# Should show: -r----- (only you can read)
```

For Windows Users (PowerShell):

Open PowerShell

```
# Create .ssh directory if it doesn't exist
mkdir $env:USERPROFILE\.ssh -Force

# Move key to .ssh directory
Move-Item -Path "$env:USERPROFILE\Downloads\devops-lab-key.pem" `
    -Destination "$env:USERPROFILE\.ssh\"

# The file is now at: C:\Users\YourName\.ssh\devops-lab-key.pem
```

For Windows Users (Git Bash - Alternative):

If you have Git for Windows installed:

```
# Create .ssh directory
mkdir -p ~/.ssh

# Move key
mv ~/Downloads/devops-lab-key.pem ~/.ssh/

# Set permissions
chmod 400 ~/.ssh/devops-lab-key.pem
```

5.3 Connect via SSH:

Replace **54.123.45.67** with YOUR instance's public IP

macOS / Linux:

```
ssh -i ~/.ssh/devops-lab-key.pem ubuntu@54.123.45.67
```

Windows (PowerShell):

```
ssh -i "$env:USERPROFILE\.ssh\devops-lab-key.pem" ubuntu@54.123.45.67
```

Windows (Git Bash):

```
ssh -i ~/.ssh/devops-lab-key.pem ubuntu@54.123.45.67
```

5.4 First Connection:

You'll see this security warning:

```
The authenticity of host '54.123.45.67 (54.123.45.67)' can't be
established.
ED25519 key fingerprint is
```

```
SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type **yes** and press Enter

Success looks like:

```
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1018-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

ubuntu@ip-172-31-xx-xx:~$
```

YOU ARE NOW CONNECTED TO YOUR AWS LINUX SERVER!

Checkpoint 6: You can SSH into your server!

Step 6: Configure Easy SSH Access

Typing the full SSH command every time is tedious. Let's create an alias.

6.1 Create SSH Config File (On Your Local Machine - NOT on server):

First, disconnect from server:

```
exit
```

You're back on your local machine.

macOS / Linux:

```
# Create or edit SSH config
vim ~/.ssh/config
```

Press **i** to enter insert mode, then add:

```
Host devops-lab
  HostName 54.123.45.67 (replace with your own IP)
  User ubuntu
  IdentityFile ~/.ssh/devops-lab-key.pem
  ServerAliveInterval 60
```

Replace **54.123.45.67** with your actual public IP

Press **Esc**, then type **:wq** and press Enter to save

Windows (PowerShell or Git Bash):

```
notepad $env:USERPROFILE\.ssh\config
```

Add this:

```
Host devops-lab
  HostName 54.123.45.67 (replace with your own IP)
  User ubuntu
  IdentityFile ~/.ssh/devops-lab-key.pem
  ServerAliveInterval 60
```

Save and close

6.2 Test Simplified Connection:

```
ssh devops-lab
```

That's it! Just two words instead of the long command.

Checkpoint 7: You can now connect with **ssh devops-lab**

Step 7: Initial Server Setup

You're connected to your server. Now let's set it up properly.

7.1 Update Package Lists:

```
sudo apt update
```

What this does: Downloads list of latest available software packages

Output: You'll see progress of package lists being downloaded

7.2 Upgrade Installed Packages:

```
sudo apt upgrade -y
```

What this does: Installs newer versions of all installed software

The `-y` flag: Automatically answers "yes" to prompts

This takes 2-5 minutes - be patient

7.3 Install Essential DevOps Tools:

```
sudo apt install -y \  
    vim \  
    htop \  
    curl \  
    wget \  
    git \  
    net-tools \  
    tree \  
    unzip \  
    build-essential \  
    software-properties-common \  
    ca-certificates
```

What each tool does:

- `vim` - Text editor for editing files
- `htop` - Interactive process viewer (better than top)
- `curl` / `wget` - Download files from URLs
- `git` - Version control system
- `net-tools` - Network utilities (ifconfig, netstat)
- `tree` - Display directory structure visually
- `unzip` - Extract zip files
- `build-essential` - Compilers and build tools
- `software-properties-common` - Manage software repositories
- `ca-certificates` - SSL/TLS certificates for HTTPS

7.4 Set Timezone (Optional but Recommended):

```
# Check current timezone
timedatectl

# List available timezones
timedatectl list-timezones

# Example: Set to US Eastern
sudo timedatectl set-timezone America/New_York

# Or find your timezone
timedatectl list-timezones | grep -i new_york
timedatectl list-timezones | grep -i london
timedatectl list-timezones | grep -i tokyo
```

7.5 Verify Installation:

```
# Check system info
uname -a          # Kernel version
lsb_release -a    # Ubuntu version

# Test tools
vim --version     # Should show Vim version
git --version     # Should show Git version
htop --version    # Should show htop version

# Check resource usage
free -h          # Memory
df -h            # Disk
```

Checkpoint 8: Your server is fully updated with essential tools!

Step 8: Install nginx (Test Web Server)

Let's install a web server to test everything works:

8.1 Install nginx:

```
sudo apt install -y nginx
```

8.2 Check if nginx is running:

```
sudo systemctl status nginx
```

Expected output:

```
● nginx.service - A high performance web server and a reverse proxy
server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
   Active: active (running) since...
```

Press **q** to quit

8.3 Test from inside the server:

```
curl http://localhost
```

You should see HTML output (nginx default page)

8.4 Test from your web browser:

Open browser and go to:

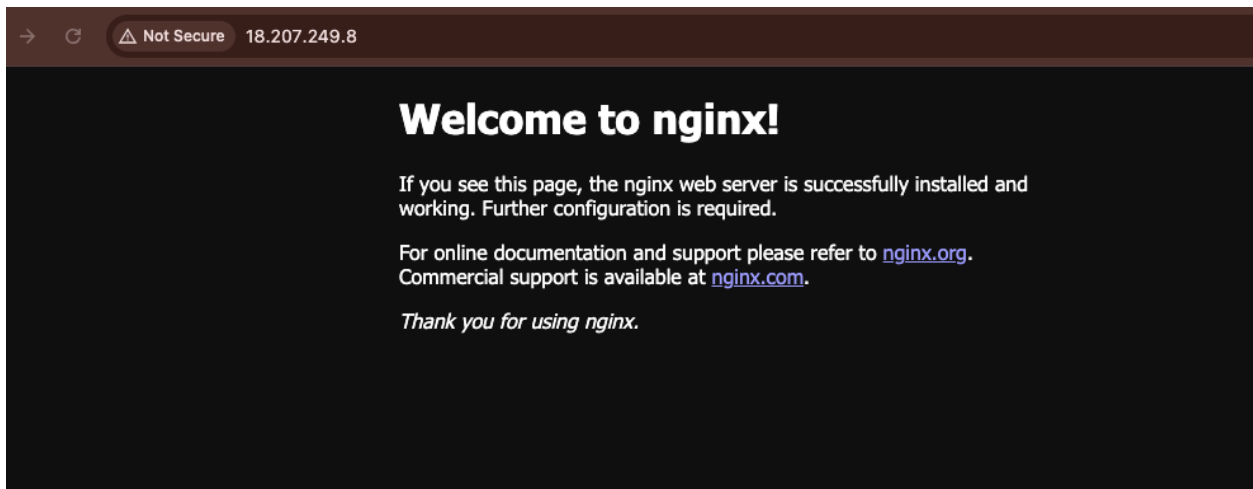
```
http://YOUR_PUBLIC_IP
```

Replace with your actual public IP (e.g., <http://54.123.45.67>)

You should see:

```
Welcome to nginx!
```

If you see this page, everything is working perfectly!



If it doesn't work:

1. Check that security group allows HTTP (port 80)
2. Check nginx is running: `sudo systemctl status nginx`
3. Check firewall: `sudo ufw status` (should be inactive)

Checkpoint 9: nginx is running and accessible from the internet!

Step 9: Create AMI Snapshot (Backup)

Before you start experimenting and potentially breaking things, create a backup.

9.1 In AWS Console:

- Go to EC2 → Instances
- **Select your instance** (checkbox)
- **Actions** dropdown → **Image and templates** → **Create image**

9.2 Configure Image:

- **Image name:** `devops-lab-clean-install`
- **Image description:** `Fresh Ubuntu 24.04 with essential tools and nginx`
- **No reboot:** Leave unchecked
- **Instance volumes:** Leave as default

Click "Create image"

9.3 Wait for creation:

- Left sidebar → Click "AMIs" (under Images)
- Status: Pending → Available (takes ~5 minutes)

When status is "Available", your backup is ready!

 ami-0c76fdefad4aa4087	machine	Linux/UNIX	EBS
AMI name  devops-lab-clean-install	Owner account ID  334091769766	Architecture x86_64	Usage operation RunInstances
Root device name  /dev/sda1	Status  Available	Source  334091769766/devops-lab-clean-install	Virtualization type hvm
Boot mode uefi-preferred	State reason -	Creation date  2025-11-30T23:41:05.000Z	Kernel ID -

If you ever break your server completely, you can launch a new instance from this AMI.

Checkpoint 10: You have a backup of your server!

Step 10: Understanding Stop vs Terminate

CRITICAL: Know the difference!

STOP (Recommended - Saves Free Tier Hours)

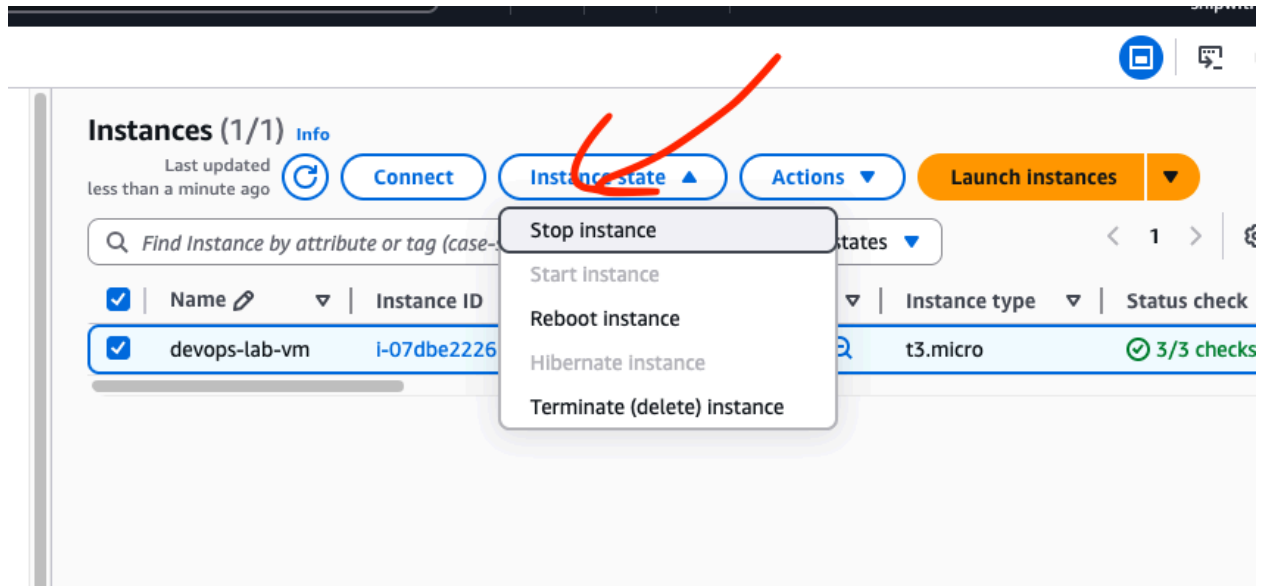
What happens:

- Instance stops running (no compute charges)
- Storage remains (still within free tier $\leq 30\text{GB}$)
- **Public IP changes** when you start it again
- You can start it again anytime

How to stop:

From AWS Console:

- Select instance → **Instance state** → **Stop instance**



From terminal (inside server):

```
sudo shutdown now
```

When to stop:

- When you're done practicing for the day
- Overnight
- Weekends
- Any time you're not using it

Cost: \$0 (still within free tier)

TERMINATE (Delete - PERMANENT)

What happens:

- Instance is DELETED

- Storage is DELETED
- **Cannot be recovered**
- Public IP is released

How to terminate:

- Select instance → **Instance state** → **Terminate instance**

When to terminate:

- When you want to permanently delete the server
- When switching to a new AMI
- **BE CAREFUL** - this is irreversible!

START (After stopping)

How to start:

- Select instance → **Instance state** → **Start instance**
- Wait 1-2 minutes
- **Get the new public IP** (it changed!)
- Update SSH config with new IP

Best practice for this playbook:

Daily workflow:

```
# Morning: Start instance (AWS Console)
# Work: SSH in and practice
# Evening: Stop instance (save free tier hours)
```

Weekend:

```
# Keep instance stopped (saves ~48 hours of free tier)
```

Checkpoint 11: You understand instance lifecycle management!

Step 11: Practice Basic Linux Commands

Now that everything is set up, let's practice some commands:

```
# System Information
hostname                # Server name
whoami                  # Current user
pwd                     # Current directory
uname -a                # Kernel info
lsb_release -a          # Ubuntu version

# Resource Usage
free -h                 # Memory usage
df -h                   # Disk usage
top                      # Process monitor (press 'q' to quit)
htop                    # Better process monitor (press F10 to quit)

# File System
ls -la /                 # List root directory
ls -lh ~                 # List home directory
tree -L 2 /etc           # Show /etc structure (2 levels)
cat /etc/os-release      # OS information

# Services
sudo systemctl list-units --type=service --state=running
sudo systemctl status nginx

# Network
ip addr show             # IP addresses
ip route show            # Routing table
ss -tlnp                 # Open ports (listening)

# Logs
sudo journalctl -u nginx -n 20      # Last 20 nginx log lines
sudo journalctl --since "1 hour ago" # Last hour of logs
```

Practice these commands. Get comfortable with the terminal.

Checkpoint 12: You can navigate and inspect your Linux system!

Troubleshooting Common Issues

Issue 1: Can't Connect "Connection refused"

Possible causes:

1. Instance is not running
2. Security group doesn't allow SSH from your IP
3. Your IP address changed (**check**)
4. Wrong key file path

Solutions:

```
# Check instance state (AWS Console)
# Should be "Running"

# Check security group (AWS Console)
# EC2 → Security Groups → devops-lab-sg
# Inbound rules should have SSH port 22 from your IP

# Check your laptops current IP
curl ifconfig.me

# Update security group if your IP changed:
# Edit inbound rules → My IP (updates automatically)

# Verify key file location
ls -l ~/.ssh/devops-lab-key.pem
```

Issue 2: "Permission denied (publickey)"

Cause: Wrong key permissions or wrong key file

Solution:

```
# Check key permissions
ls -l ~/.ssh/devops-lab-key.pem
# Should show: -r----- (400)

# Fix permissions
chmod 400 ~/.ssh/devops-lab-key.pem

# Verify you're using the right key
ssh -i ~/.ssh/devops-lab-key.pem ubuntu@YOUR_IP -v
# The -v shows verbose output for debugging
```

Issue 3: Public IP Changed After Stop/Start

This is normal and expected.

Solution:

1. Get new public IP from AWS Console
2. Update SSH config:

```
# Edit SSH config
vim ~/.ssh/config

# Change the HostName line:
HostName 54.NEW.IP.HERE
```

3. Save and reconnect

Issue 4: "Too many authentication failures"

Cause: SSH is trying multiple keys

Solution:

```
# Connect with specific key only
ssh -o IdentitiesOnly=yes -i ~/.ssh/devops-lab-key.pem ubuntu@YOUR_IP

# Or add to SSH config:
Host devops-lab
    IdentitiesOnly yes
    HostName YOUR_IP
    User ubuntu
    IdentityFile ~/.ssh/devops-lab-key.pem
```

Issue 5: Can't Access nginx from Browser

Solutions:

```
# Check nginx is running
sudo systemctl status nginx

# Check security group allows HTTP (AWS Console)
# Should have inbound rule: HTTP (80) from 0.0.0.0/0

# Check nginx is listening
sudo ss -tlnp | grep 80

# Test from server
curl http://localhost

# Restart nginx
sudo systemctl restart nginx
```

Final Verification Checklist

Before moving to Linux tasks, verify ALL of these:

- ☐ AWS account created and activated
- ☐ MFA enabled (2-factor authentication)
- ☐ Billing alerts configured (\$5 threshold)
- ☐ Email subscription confirmed
- ☐ EC2 instance running (t2.micro, Ubuntu 24.04)
- ☐ Can connect: ssh devops-lab
- ☐ System updated (apt update && upgrade)
- ☐ Essential tools installed (vim, htop, curl, git, etc.)
- ☐ nginx installed and running
- ☐ Can access nginx from browser (http://YOUR_IP)
- ☐ AMI snapshot created
- ☐ Understand how to stop/start instance
- ☐ Understand free tier limits

If ALL boxes are checked: YOU'RE READY FOR SECTION 2.1!

NEXT STEPS

Take a 15 minute break. Seriously. You earned it!

Then proceed to:

→ **SECTION 2.1 - When You Use Linux as a DevOps Engineer**

This is where the real DevOps learning begins. You'll use the server you just created to complete real-world tasks.

You're ready. Let's go!

SECTION 2.1: When You Use Linux as a DevOps Engineer

The Reality

You will use Linux every single day.

Not occasionally. Not "when needed." **Daily.**

Here's what a typical DevOps day looks like:

9:00 AM: SSH into staging server to check deployment from last night

10:30 AM: Troubleshoot failed CI/CD pipeline (container build issue)

11:45 AM: Review and merge PR that updates nginx configuration

1:00 PM: Deploy new microservice, verify logs show healthy startup

2:30 PM: Investigate alert: high memory usage on API server

4:00 PM: Write Bash script to automate database backup verification

5:15 PM: Update security group rules, test SSH access still works

Every single one of these tasks happens on Linux.

The Six Core Scenarios

1. Troubleshooting Production (Daily)

When: Service is down, slow, or throwing errors.

What you do:

- SSH into the affected server
- Check if the service is running (`systemctl status`)
- Read recent logs (`journalctl`)
- Test network connectivity (DNS, ports, firewalls)
- Identify root cause
- Apply fix
- Verify resolution
- Document in postmortem

Example: API returning 500 errors. You SSH in, check logs, find database connection pool exhausted, restart service, increase pool size in config.

Frequency: Multiple times per week.

2. Deploying Services (Weekly)

When: New service needs to go live, or existing service needs updating.

What you do:

- SSH to deployment server
- Pull application code/binary
- Configure environment variables
- Create systemd service file
- Enable and start service
- Verify health endpoint responds
- Check logs for errors
- Update load balancer to route traffic

Example: Deploy new checkout-service v2.1.3 to staging, verify it connects to Redis, promote to production.

Frequency: 2-5 times per week.

3. Security Hardening (Monthly)

When: Security audit flags issues, compliance review, or best practice enforcement.

What you do:

- Audit SSH configuration
- Disable password authentication
- Restrict users who can login
- Configure firewall rules (iptables/ufw)
- Set up log monitoring
- Apply OS security patches
- Verify no unauthorized access

Example: Bastion host allows root login and password auth. You disable both, restrict to key only auth from specific IPs, document changes.

Frequency: Every few weeks, or triggered by audits.

4. Performance Investigation (Weekly)

When: Server is slow, CPU/memory/disk is maxed out, or users report latency.

What you do:

- Check resource usage (`top`, `htop`, `free`, `df`)
- Identify problematic processes
- Trace system calls if needed (`strace`)
- Check open files and connections (`lsof`, `ss`)
- Review recent changes (deployments, config)
- Apply fix (restart service, increase resources, rollback)
- Monitor to confirm resolution

Example: API server at 95% CPU. You find an infinite retry loop in new deployment, rollback to previous version, CPU drops to normal.

Frequency: 1-3 times per week.

5. Automation (Daily)

When: Repetitive task needs to be automated, or manual process is error-prone.

What you do:

- Write Bash script
- Make it idempotent (safe to run multiple times)
- Add error handling
- Test thoroughly
- Set up cron job or systemd timer
- Monitor for failures
- Document what it does

Example: Automate daily health check of all services, send email if any are down, log results to file.

Frequency: Daily. Automation is constant.

6. Incident Response (On-Call)

When: Midnight. PagerDuty alert. Production is broken.

What you do:

- Acknowledge alert
- SSH into affected system
- Quick triage (service status, logs, resource usage)
- Apply immediate fix (restart service, clear disk space, etc.)
- Verify issue resolved
- Resolve alert
- Write postmortem in the morning

Example: Database replica lag > 60 seconds. You SSH in, find disk full from old logs, delete old logs, verify replication catches up.

Frequency: Depends on on-call rotation. Could be weekly or monthly.

What This Means for You

If you cannot:

- Navigate Linux file system
- Read systemd logs
- Manage services
- Edit configuration files
- Debug network issues
- Write basic Bash scripts

You cannot do DevOps work.

Linux is not optional. It's not "nice to have."

It's the foundation everything else is built on.

Skills You'll Use Daily

File System:

- Navigate: `cd`, `ls`, `pwd`
- Read files: `cat`, `less`, `tail`
- Find files: `find`, `grep`
- Edit files: `vim`, `nano`

Process Management:

- View processes: `ps`, `top`, `htop`
- Kill processes: `kill`, `pkill`
- Background jobs: `&`, `nohup`

Services (systemd):

- Status: `systemctl status SERVICE`
- Start/stop/restart: `systemctl start/stop/restart SERVICE`
- Enable on boot: `systemctl enable SERVICE`
- View logs: `journalctl -u SERVICE`

Networking:

- Check connections: `ss`, `netstat`
- Test connectivity: `ping`, `curl`, `nc`
- DNS lookup: `dig`, `nslookup`
- View routes: `ip route`

System Resources:

- CPU/Memory: `top`, `htop`, `free`
- Disk: `df`, `du`
- Open files: `lsof`

Logs:

- systemd logs: `journalctl`
- Application logs: `/var/log/`
- Real-time: `tail -f, journalctl -f`

You'll master all of these in **Section 2.4**.

SECTION 2.2: Why Linux Matters

By The Numbers

Linux runs:

- 96.3% of the world's top 1 million web servers
- 100% of the world's top 500 supercomputers
- 90% of cloud infrastructure
- 100% of Kubernetes clusters
- Every Docker container
- Every major DevOps tool

If it's in the cloud, it's probably Linux.

Why Companies Choose Linux

1. Cost

Windows Server 2022 Standard:

- \$1,069 per 16-core license
- \$500-\$1,000 per server (typical)

Ubuntu Server:

- \$0

For 100 servers:

- Windows: ~\$100,000 in licensing

- Linux: \$0

Companies save millions by using **Linux**.

2. DevOps Toolchain Runs on Linux

Everything you'll learn in this playbook runs on Linux:

```
Docker      → Linux containers
Kubernetes  → Linux orchestration
Terraform   → Manages Linux infrastructure
Jenkins/GH Actions → Run on Linux
Prometheus  → Runs on Linux
nginx/Apache → Linux web servers
PostgreSQL/MySQL → Run on Linux
Redis/MongoDB → Run on Linux
```

You cannot do modern DevOps without Linux.

3. Containers ARE Linux

Docker containers:

- Use Linux kernel
- Linux processes
- Linux file system
- Linux networking

Even Docker Desktop on Windows/Mac:

- Runs a Linux VM underneath
- Your containers are still Linux

Kubernetes = Linux containers at scale.

No Linux knowledge = No container knowledge.

4. Automation & Scriptability

GUI (Windows) approach:

1. Click Start
2. Click Server Manager
3. Navigate to Services
4. Right-click service
5. Click Restart
6. Click Yes
7. Wait for confirmation

Repeat for 100 servers? 8 hours of clicking.

CLI (Linux) approach:

```
for server in $(cat servers.txt); do
    ssh $server "sudo systemctl restart myapp"
done
```

100 servers in 30 seconds.

This is why DevOps engineers love Linux.

5. Stability & Security

Typical Windows Server uptime:

- 30-60 days (forced reboots for updates)

Typical Linux server uptime:

- 200-400+ days (live kernel patching available)

Linux security advantages:

- No registry to corrupt

- Permissions model (users, groups, chmod)
- Smaller attack surface
- Faster security patches
- No need for antivirus (typically)

The Linux Philosophy

Do one thing well:

Each tool has one purpose:

- `grep` searches text
- `sed` transforms text
- `awk` processes columns
- `sort` sorts lines

Combine them with pipes:

```
cat access.log | grep "ERROR" | awk '{print $1}' | sort | uniq -c | sort -rn | head -10
```

Find the top 10 IPs generating errors.

This composability makes automation incredibly powerful.

Everything Is a File

In Linux, EVERYTHING is a file:

```
/dev/sda          # Your hard drive
/proc/cpuinfo     # CPU information
/sys/class/net/   # Network interfaces
/var/log/         # Application logs
```

You interact with hardware using file operations:

```
# Read CPU info
cat /proc/cpuinfo

# Check memory
cat /proc/meminfo

# View network stats
```

```
cat /proc/net/dev
```

Uniform interface. Simple. Powerful.

Job Market Reality (2026)

DevOps job postings mentioning Linux:

- Required: 94%
- Preferred: 6%
- Not mentioned: 0%

You cannot get a DevOps job without Linux skills.

Salary data (US, 2026):

- Junior DevOps with Linux: \$75k-\$95k
- Junior DevOps without Linux: **Cannot get hired**

Linux vs Windows vs macOS (DevOps Context)

OS COMPARISON FOR DEVOPS

Criteria	Linux	Windows	MacOs
Server usage	90%+	<10%	0%
Cost	Free	Expensive	N/A
Containers	Native	Via VM	Via VM
Automation	Excellent	Limited	Good
Job Demand	Very High	Low	None

For servers: Linux dominates.

Common Myths (Debunked)

Myth: "Linux is hard to learn."

Reality: Linux is simpler once you understand the basics. No hidden registry. Config is text files. Behavior is predictable.

Myth: "I need to memorize hundreds of commands."

Reality: You use less than ~20 commands 90% of the time. Everything else you can look up.

Myth: "GUI is easier than CLI."

Reality: CLI is faster once you learn it. Typing one command beats 10 clicks.

Myth: "Linux breaks easily."

Reality: Linux is stable if you understand what you're doing. Most "Linux broke my system" stories are from running random commands from the internet without understanding them.

What You're Learning

By mastering Linux, you're learning:

- The foundation of cloud infrastructure
- The OS that runs every container
- The platform for every DevOps tool
- The environment where production runs
- The system you'll troubleshoot daily

Master Linux and Everything else gets easier.

Key Takeaway: Linux is not optional in DevOps. It's the foundation. Companies use it because it's free, stable, and automatable. Every tool you'll learn runs on Linux.

SECTION 2.3: How to Think Like a Linux Engineer

The Difference

Beginner Approach: Problem → Panic → Try random things → Give up

Engineer Approach: Problem → Gather data → Fix root cause → Document

It's not about knowing more commands. It's about systematic thinking.

The 4 Step Framework (Use This Every Time)

1. Gather Data

```
systemctl status myapp      # Is service running?
journalctl -u myapp -n 50   # What do logs say?
top                          # Is CPU/memory exhausted?
df -h                       # Is disk full?
```

Don't guess. Read what the system tells you.

2. Identify Root Cause

Logs say: "Error: ECONNREFUSED postgres:5432"

Translation: App can't reach database on port 5432

Likely causes (ranked):

1. PostgreSQL stopped (most common)
2. Firewall blocking port (less likely)
3. Wrong connection string (rare)

3. Test and Fix One Thing at a Time

```
# Test #1: Is PostgreSQL running?
systemctl status postgresql
# Output: "inactive (dead)" ← FOUND IT

# Fix: Start PostgreSQL
```

```
sudo systemctl start postgresql

# Verify: Does app work now?
curl http://localhost:8080/health
# Output: {"status": "healthy"} ← FIXED
```

Why one change at a time? If you change 10 things and it breaks, you don't know which one caused it.

4. Document

Problem: App returning 500 errors

Root Cause: PostgreSQL crashed

Fix: Restarted PostgreSQL

Prevention: Add monitoring alert for PostgreSQL uptime

Time: 5 minutes

Why document? Next time it breaks, you (or your teammate) solve it in 30 seconds.

The 3 Core Principles

Principle 1: Logs Are Truth

```
# Read logs FIRST, guess LAST
journalctl -u myapp -n 50    # Last 50 lines
journalctl -u myapp -f       # Follow live (like tail -f)

# Application logs
tail -f /var/log/myapp.log
grep ERROR /var/log/myapp.log
```

The error message tells you what's wrong. Read it carefully instead of guessing.

Principle 2: Simple to Complex Order

Wrong order: Check VPC routing, NACLs, security groups

Right order:

```
1. Is service running?      systemctl status myapp
2. Are logs showing errors? journalctl -u myapp
3. Is disk full?            df -h
4. Is port open?            netstat -tlnp | grep 8080
```

Start with obvious causes. Only go deeper if needed.

Principle 3: Services Are Connected

```
User Request
  ↓
nginx (port 80)
  ↓
myapp (port 8080)
  ↓
PostgreSQL (port 5432)
  ↓
Disk (/var/lib/postgresql)
```

If disk is full → PostgreSQL can't write → myapp crashes → nginx returns 500

Fix the root (disk space), not the symptom (restart myapp).

systemd: The Service Manager

What it does: Starts services, restarts them if they crash, collects logs

4 commands you'll use daily:

```
systemctl status nginx      # Is it running?
systemctl restart nginx     # Restart service
journalctl -u nginx -f      # Watch logs live
systemctl enable nginx      # Start on boot
```

Service states:

- active (running) → Working
- failed → Crashed (read logs to see why)
- inactive (dead) → Stopped

Always check status first, then read logs.

Pattern Recognition (It Clicks With Practice)

Pattern 1: Service Won't Start

journalctl -u myapp → "Address already in use: 8080"

Diagnosis: Another process using port 8080

Fix: `sudo lsof -i :8080` (find process), kill it or change app port

Pattern 2: Slow Performance

top → myapp using 97% CPU

journalctl -u myapp → "Retrying connection... (attempt 5000)"

Diagnosis: Infinite retry loop (bug in code)

Fix: Restart service (immediate), deploy fix (permanent)

Pattern 3: Connection Refused

`curl http://localhost:8080` → "Connection refused"

Checklist:

1. Is service running? `systemctl status myapp`
2. Is it listening on 8080? `netstat -tlnp | grep 8080`
3. Is firewall blocking? `sudo ufw status`

Key Takeaways

- **Systematic thinking beats random guessing**
- **Read logs first** (they tell you what's broken)
- **Test one change at a time** (always know what broke)
- **Work simple → complex** (start with obvious causes)
- **Services depend on each other** (fix the root cause)

Next: Apply this framework in real tasks (Section 2.4)

SECTION 2.4: Linux Tasks (Hands-On)

At the end of this section, you will have:

- A real web server (nginx)
- A real API service (payments-api)
- Both running as managed Linux services
- Logs you can inspect
- A system you can break and fix like a real DevOps engineer

What You're About to Do

This is where theory becomes practice.

System you are building Here

```
- User → nginx (web)
- User → payments-api (API)
- systemd keeps both alive
- logs tell you what went wrong
```

You'll complete **5 real world tasks** that simulate actual DevOps work:

1. **TASK L1:** Troubleshoot and restore a broken web service
2. **TASK L2:** Harden SSH access for security compliance
3. **TASK L3:** Automate system health checks with systemd timers
4. **TASK L4:** Investigate and resolve high CPU usage
5. **TASK L5:** Analyze logs and write a root cause analysis

Each task is structured like a real JIRA ticket:

- Business context (why this matters)
- Constraints (what you can/can't do)
- Deliverables (what to produce)
- Acceptance criteria (how you know you're done)
- Step-by-step instructions
- What you learned

Prerequisites Check

Before continuing, confirm:

- ☐ You can SSH into your EC2 instance
- ☐ Ubuntu 24.04 is installed
- ☐ `vim`, `curl`, `wget`, and `htop` exist
- ☐ `sudo whoami` works

If any item fails, **stop and fix it** before continuing.

Before starting, verify:

```
# You should be connected to your AWS EC2 instance
ssh devops-lab-vm

# Check Ubuntu version
lsb_release -a
# Should show: Ubuntu 24.04 LTS

# Check essential tools are installed
which vim curl wget htop
# All should return paths

# Check you have sudo access
sudo whoami
# Should return: root or ubuntu
```

If anything fails, review Section 2.0 (AWS setup).

Environment Setup (15 minutes)

How to Think About This Section (Important)

Before you start running commands, take a step back.

In this section, you are touching several core Linux/DevOps concepts at the same time. You are **not** expected to master any of them yet.

Here's how to mentally group what you're doing:

Concept groups you're touching (don't master yet):

- **Services:** `nginx`, `payments-api`
These are applications that run continuously in the background.
- **Service manager:** `systemd`
This is what starts, stops, and watches those services.
- **Access:** ports, `localhost`
This is how users and tools reach running services.
- **Observability:** logs, metrics
This is how you know what's happening when things work or break.

Your goal here is not deep understanding, your goal is to wire these pieces together into a working system.

You will revisit each concept later in isolation. Before starting tasks, install the practice services.

Step 1: Install nginx

Why?

nginx represents a **real production web service**. You are not learning nginx configuration here, you are learning how Linux runs, monitors, and troubleshoots services.

```
# Install nginx web server
sudo apt update
sudo apt install -y nginx

# Verify it's running
sudo systemctl status nginx
# Should show: active (running)

# Test from browser
```



```
curl http://localhost
# Should return HTML
```

Visit your EC2 public IP in browser:

```
http://YOUR_PUBLIC_IP
```

You should see "Welcome to nginx!"

Step 2: Create Mock API Service

Why? This API simulates a real microservice you'd manage at work. It gives you something safe to break, restart, monitor, and debug — without touching real production systems.

```
# Create service directory
sudo mkdir -p /opt/payments-api

ls /opt/ (to view the directory we just created)

sudo chown ubuntu:ubuntu /opt/payments-api
# This command changes the ownership of /opt/payments-api to the user
ubuntu and group ubuntu.
```

Breakdown:

sudo: runs with administrator privileges

Chown: change ownership

ubuntu:ubuntu: user:group (user ubuntu, group ubuntu)

/opt/payments-api: target file or director

Create simple Python API

```
cat > /opt/payments-api/api.py << 'EOF'
#!/usr/bin/env python3
from http.server import HTTPServer, BaseHTTPRequestHandler
import json
```

```

class PaymentsAPI(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == '/health':
            self.send_response(200)
            self.send_header('Content-type', 'application/json')
            self.end_headers()
            response = {'status': 'healthy', 'service':
'payments-api', 'version': '1.0.0'}
            self.wfile.write(json.dumps(response).encode())
        elif self.path == '/metrics':
            self.send_response(200)
            self.send_header('Content-type', 'text/plain')
            self.end_headers()
            self.wfile.write(b'payments_processed_total
1234\npayments_failed_total 5\n')
        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        pass # Suppress console logs

if __name__ == '__main__':
    server = HTTPServer(('0.0.0.0', 8080), PaymentsAPI)
    print('Payments API running on port 8080')
    server.serve_forever()
EOF

```

Important

You do **not** need to understand this Python code. Treat it as a black box that exposes `/health` and `/metrics`. This is a Linux and DevOps exercise, not a programming one.

You can access the Dir. by doing

```

# Navigate to the application directory
cd /opt/payments-api

```

```

# View the directory contents
ls

# View the Python API code
cat api.py

# Make the script executable
chmod +x /opt/payments-api/api.py

# Test that it works
python3 /opt/payments-api/api.py &    # Start in background
sleep 2                                # Wait for startup
curl http://localhost:8080/health      # Test health endpoint
# Expected output: {"status": "healthy", ...}

# Stop the test
pkill -f api.py                        # Kill the process

```

What you're doing

You're creating a simple payments API service that responds to HTTP requests, similar to a real microservice. (An **API** service is a middleman that lets different applications talk to each other in a structured way. It receives requests, processes them, and sends back responses, usually as data rather than a user interface.)

Step-by-step breakdown

1. Create the directory

```
sudo mkdir -p /opt/payments-api
```

Which Creates a folder at `/opt/payments-api` for the service files.

2. Change ownership

```
sudo chown ubuntu:ubuntu /opt/payments-api #Makes the ubuntu user
the owner so you can edit files even without sudo.
```

3. Create the Python API file

- Writes a Python script (`api.py`) that:
- Listens on port 8080
- Responds to `/health` with a JSON status
- Responds to `/metrics` with metrics data
- Returns 404 for other paths

4. Make it executable

```
chmod +x /opt/payments-api/api.py
```

- Allows running the script directly (e.g., `./api.py`).

5. Test it

- Start the server in the background (`&`)
- Wait 2 seconds
- Call `curl http://localhost:8080/health` to verify it responds
- Stop the test server with `kill`

Why this matters

This simulates a real service you might manage. You'll later configure systemd to run it automatically, monitor it, and manage it like a production service.

Checklist

You should be able to confirm:

- ☐ `payments-api` runs on port 8080
- ☐ `/health` returns JSON
- ☐ `/metrics` returns text
- ☐ Killing the process stops the service (for now)

If this doesn't work, **do not move on**.

Step 3: Create systemd Service

Think of systemd like a supervisor

- You don't run the app anymore
- systemd runs it *for* you
- If it dies, systemd brings it back
- Killing the process is useless unless you stop the service

Create systemd service file

This creates a **systemd** service file that tells Linux how to run your payments API as a background service. The service will automatically start on boot, restart if it crashes, and run as the ubuntu user.

systemd is the service manager on Linux that starts, stops, and manages background services (like web servers, databases, and APIs). It runs services automatically at boot, restarts them if they crash, and provides commands like **systemctl** to control them.

```
sudo tee /etc/systemd/system/payments-api.service > /dev/null <<
'EOF'
[Unit]
Description=Payments API Service
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/payments-api
ExecStart=/usr/bin/python3 /opt/payments-api/api.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

```
# Reload systemd
```

```
sudo systemctl daemon-reload
```

systemd: The One Idea You Must Remember

Think of systemd as a supervisor, not a command.

Once a service is managed by systemd:

- You do not run it manually anymore
- systemd starts it for you
- If it crashes, systemd restarts it
- Killing the process does not stop the service

This is why port conflicts happen if you forget a service is running.

This behavior is intentional, it's how production systems stay alive.

Why this design and its tradeoffs:

- systemd is simple and reliable
- Not containerized
- No autoscaling
- Perfect for learning Linux fundamentals

This creates a **systemd** service file that defines how the Payments API should run as a managed background service on the system. It tells Linux to start the API using Python as the `ubuntu` user, automatically restart it if it crashes, and make it available after the network is up. Reloading systemd applies the new service definition so systemd recognizes and can manage the service.

Enable and start service

```
sudo systemctl enable payments-api  
sudo systemctl start payments-api
```

Checklist

Confirm all of the following:

- ☐ `systemctl status payments-api` shows *active (running)*
- ☐ Rebooting would restart the service automatically
- ☐ Killing the process does NOT stop the service
- ☐ systemd restarts it if it crashes

If this feels confusing, that's normal.

You will revisit it later.

Verify it's running

```
sudo systemctl status payments-api
# Should show: active (running)
```

Test endpoints

```
curl http://localhost:8080/health
curl http://localhost:8080/metrics
```

Output:

```
ubuntu@ip-172-31-71-160:~$ curl http://localhost:8080/health
curl http://localhost:8080/metrics
{"status": "healthy", "service": "payments-api", "version":
"1.0.0"}payments_processed_total 1234
payments_failed_total 5
```

Step 4: Create Log Directory

```
# Create directory for task deliverables
mkdir -p ~/devops-tasks/logs
mkdir -p ~/devops-tasks/scripts
mkdir -p ~/devops-tasks/configs
```

Why?

Production systems separate logs, scripts, and configs. This structure trains you to think like an operator, not a hobbyist.

Create README

```
cat > ~/devops-tasks/README.md << 'EOF'
# DevOps Learning Tasks

This directory contains deliverables from hands-on Linux tasks.

## Structure
- `logs/` - Log analyses and findings
- `scripts/` - Automation scripts
- `configs/` - Configuration files
EOF
```

Step 5: Verification

```
# Check all services are running
sudo systemctl status nginx payments-api --no-pager

# Both should show "active (running)"

# Test nginx
curl -I http://localhost
# Should return: HTTP/1.1 200 OK

# Test payments-api
```



```
curl http://localhost:8080/health
# Should return JSON {"status": "healthy", "service": "payments-api",
"version": "1.0.0"}

# Check logs work
sudo journalctl -u nginx -n 5
sudo journalctl -u payments-api -n 5

# The curl command tests the API by sending a GET request to the
/health endpoint and returns the JSON response. The journalctl
command shows the last 5 log entries from the payments-api systemd
service, which helps you see if it started correctly or if there are
any errors.
```

If all checks pass, you're ready for tasks!

Checklist

You are ready for tasks if:

- ☐ nginx returns HTTP 200
- ☐ payments-api `/health` works
- ☐ Logs appear in `journalctl`
- ☐ You understand which command checks what

We create these folders to organize different types of work the way real production teams do. The **logs folder** is where you record and analyze service output and errors, the **scripts folder** is for automation and helper commands, and the **configs folder** stores configuration files so they are not mixed with code. The README explains the structure so anyone (including future you) understands what each file and directory is used for.

How to Read the Verification Commands

The commands below may look repetitive, but each one answers a different real-world DevOps question.

When you run:

- `systemctl status payments-api`

You are asking:

“Does the operating system believe this service is running?”

- `curl http://localhost:8080/health`

You are asking:

“Can a user or another service actually reach it?”

- `journalctl -u payments-api`

You are asking:

“If this breaks, can I see why?”

In real systems, **a service can be running but unreachable, or reachable but broken internally.**

That’s why DevOps engineers never rely on a single check.

Troubleshooting: Port Already in Use Error

Problem

```
OSError: [Errno 98] Address already in use
```

When trying to start a service on port 8080, the port is already occupied.

Diagnosis Steps

1. Check what's using the port:

```
sudo lsof -i :8080
# or
sudo netstat -tlnp | grep 8080
```

2. Check for systemd services:

```
sudo systemctl list-units --type=service | grep -i payment
sudo systemctl status payments-api.service
```

Solution

If a systemd service is auto-restarting your application:

1. Stop the systemd service:

```
sudo systemctl stop payments-api.service
```

2. Verify port is free:

```
do lsof -i :8080
S
```

Start your application manually:

```
python3 /opt/payments-api/api.py &
```

Root Cause

A systemd service (`payments-api.service`) was automatically restarting the application, keeping port 8080 occupied even after killing the process.

What You've Actually Built

You now have:

- Two real services (`nginx`, `payments-api`)
- Managed by the OS (`systemd`)
- Reachable over the network (ports)
- Observable through logs and metrics

This is the **minimum foundation** behind:

- Kubernetes
- Containers
- Cloud load balancers
- Production monitoring systems

Those tools don't replace this knowledge, they sit on top of it.

TASK L1: Troubleshoot and Restore a Broken Web Service

Business Context

Ticket ID: DEVOPS-1247

Priority: High

Assignee: You

Situation: The payments-api service stopped responding after a configuration change. The monitoring system detected the failure and paged the on call engineer (you). Customers are unable to process payments.

Impact: Revenue loss, customer complaints, SLA breach if not resolved within 30 minutes.

Your Mission

Investigate why payments-api is down and restore service as quickly as possible.

Constraints

- You cannot modify the Python application code
- Changes must be documented
- Service must pass health check after fix
- Total downtime should be minimized

Deliverables

1. Root cause identification
2. Service restored and verified
3. Short incident report (plain text file)

Step-by-Step Guide

Step 1: Break the Service (Simulate the Incident)

(Read First)

You are breaking this service *on purpose*, nothing here is permanent, nothing affects real customers, and you can always fix it.

In real DevOps work, services fail **without warning**. Practicing failure in a safe environment is how engineers learn to stay calm and effective.

Reminder: what the service originally used

Before this change, the service started with:

```
ExecStart=/usr/bin/python3 /opt/payments-api/api.py
```

You are about to change **only the Python binary path** to simulate a mistake.

From `ExecStart=/usr/bin/python3 /opt/payments-api/api.py` to

```
ExecStart=/usr/bin/python33 /opt/payments-api/api.py
```

First, let's create the problem:

```
# Stop the service
sudo systemctl stop payments-api

# Modify the systemd service file to introduce an error
sudo sed -i
's|ExecStart=/usr/bin/python3|ExecStart=/usr/bin/python33|' \
/etc/systemd/system/payments-api.service

# This command deliberately introduces a configuration error by
# pointing systemd to a Python binary that does not exist. When the
# service starts, systemd cannot execute the command, so the service
# fails immediately with an execution error. This simulates a common
# real-world outage caused by small typos or misconfigurations, not
# broken application code.

# Reload systemd
sudo systemctl daemon-reload
```

```
# Try to start (it will fail)
sudo systemctl start payments-api
```

Now you have a broken service. Time to fix it!

Step 2: Investigate (Use the Framework)

The Troubleshooting Framework You Are Practicing

Every production incident follows the same pattern:

1. **Observe** – What is broken?
2. **Gather data** – Logs, status, errors
3. **Form a hypothesis** – What do you think is wrong?
4. **Test the hypothesis** – Prove or disprove it
5. **Fix** – Apply the smallest safe change
6. **Verify** – Confirm service health
7. **Document** – Record what happened and why

You are not guessing, you are following a repeatable process.

Observe:

```
# Check service status
sudo systemctl status payments-api
```

```
● payments-api.service - Payments API Service
   Loaded: loaded (/etc/systemd/system/payments-api.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-01-13 00:12:32 WAT; 5ms ago
     Main PID: 19950 ((python33))
       Tasks: 1 (limit: 1017)
    Memory: 48.0K (peak: 720.0K)
       CPU: 3ms
    CGroup: /system.slice/payments-api.service

Jan 13 00:12:32 ip-172-31-71-160 systemd[1]: payments-api.service: Scheduled restart job, restart counter is at 7.
Jan 13 00:12:32 ip-172-31-71-160 systemd[1]: Started payments-api.service - Payments API Service.
Jan 13 00:12:32 ip-172-31-71-160 systemd[1]: payments-api.service: Main process exited, code=exited, status=203/EXEC
Jan 13 00:12:32 ip-172-31-71-160 systemd[1]: payments-api.service: Failed with result 'exit-code'.
ubuntu@ip-172-31-71-160:~$
```

How to read this output

You are not expected to memorize systemd exit codes.

For now, remember:

- `status=203/EXEC` usually means systemd could not execute the command
- This often points to:
 - A wrong binary path
 - A missing executable
 - A typo in `ExecStart`

This tells you *where* to look next.

What you'll see:

```
• payments-api.service - Payments API Service
  Loaded: loaded (/etc/systemd/system/payments-api.service;
enabled; preset: enabled)
  Active: activating (auto-restart) (Result: exit-code) since Thu
2025-12-04 20:20:34 UTC>
  Process: 2504 ExecStart=/usr/bin/python3
/opt/payments-api/api.py (code=exited, status=>
  Main PID: 2504
```

Key observation: `failed (Result: exit-code)` and `status=2`

Gather Data:

```
# Read recent logs
sudo journalctl -u payments-api -n 30 --no-pager
```

Note: The logs may not show the detailed error message. The `status=203/EXEC` indicates an execution failure.

To see more details, check the service file:

```
# Check the service file to see what command is being executed
sudo cat /etc/systemd/system/payments-api.service | grep ExecStart
```


Output:

```
ExecStart=/usr/bin/python33 /opt/payments-api/api.py
```

Confirmed! The path is wrong, but, **Pause and think**
Before scrolling further, answer this question for yourself:

“If systemd cannot execute the command, where would I check first?”

If your answer was:

- the `ExecStart` line
- the binary path
- running the command manually

You are thinking like a DevOps engineer.

Test Hypothesis

If you want to see the actual error message, you can try:

```
# Try running the command manually to see the error
/usr/bin/python33 /opt/payments-api/api.py
```

Why This Command?

We ran this command because systemd was failing to start the service using it. The `ExecStart` line in the service file tells systemd exactly what command to execute, so testing the same command manually helps isolate the problem. When it fails with “No such file or directory,” it confirms the issue is the binary path, not the application itself.

This connects the dots clearly:

- systemd failure → check `ExecStart`

- `ExecStart` → run it manually
- manual failure → confirmed root cause

This will show: `/usr/bin/python33: No such file or directory`

This makes it clear that:

The `status=203/EXEC` is the key indicator

Checking the service file reveals the wrong path

The detailed error can be seen by running the command manually

Why this test matters

Running the command manually removes systemd from the equation.

If it fails here, the problem is **not systemd**, it's the command itself.

This is a powerful technique you will reuse in many incidents.

Step 3: Fix the Issue

Apply the Fix

```
sudo sed -i 's|/usr/bin/python33|/usr/bin/python3|' \
/etc/systemd/system/payments-api.service
```

What this does

This replaces the incorrect Python path (`/usr/bin/python33`) with the correct one (`/usr/bin/python3`) in the systemd service file. The previous path pointed to a non-existent executable, which caused the `status=203/EXEC` startup failure. After this change, systemd can successfully execute Python and start the service.

Verify the Configuration

```
sudo cat /etc/systemd/system/payments-api.service | grep ExecStart
```

Expected output

```
ExecStart=/usr/bin/python3 /opt/payments-api/api.py
```

This confirms the service file now points to a valid executable.

Reload and Restart the Service

```
sudo systemctl daemon-reload  
sudo systemctl start payments-api
```

Reloading systemd applies the configuration change, and restarting the service tests whether the fix resolved the issue.

This structure mirrors real incident response:

fix → verify config → apply change → restart → observe, which helps beginners internalize the correct workflow rather than just copying commands.

Rule of production fixes

Change only what you must.

You are fixing:

- One character
- One line
- One known cause

Large changes during incidents increase risk.

Step 4: Verify

```
# Check status
sudo systemctl status payments-api
# Should show: active (running)

# Test health endpoint
curl http://localhost:8080/health
# Should return: {"status": "healthy", ...}

# Check logs for successful startup
sudo journalctl -u payments-api -n 10 --no-pager
# Should show: "Payments API running on port 8080"
```

How to Read These Logs (Don't Get Stuck Here)

You may still see earlier `status=203/EXEC` errors in the logs.

This does not mean the service is still broken, always trust checks in this order:

1. `systemctl status` → current state
2. `/health` endpoint → user-visible health
3. Logs → historical context

If:

- Service status is active (running)
- Health check returns healthy
- No new failures appear after the fix

👉 The incident is resolved. Record it and Move on.

Service restored!

Step 5: Document

Create incident report

```
cat > ~/devops-tasks/logs/incident-report-L1.txt << 'EOF'
INCIDENT REPORT: Payments API Service Failure
=====

Date: 2025-12-04
Duration: ~15 minutes
Severity: High (customer-facing service down)

SUMMARY:
Payments API failed to start due to incorrect ExecStart path in
systemd service file.

ROOT CAUSE:
The systemd service file referenced /usr/bin/python33 instead of
/usr/bin/python3.
This caused the service to fail immediately on startup.

TIMELINE:
- 10:00 - Service stopped responding
- 10:02 - Investigated systemctl status (showed failed state)
- 10:05 - Read logs, identified "python33: No such file or directory"
- 10:08 - Located error in service file
- 10:10 - Corrected ExecStart path
- 10:12 - Service restarted successfully
- 10:15 - Verified health endpoint responding

FIX APPLIED:
Changed ExecStart path from /usr/bin/python33 to /usr/bin/python3 in:
/etc/systemd/system/payments-api.service

VERIFICATION:
- Service status: active (running)
- Health check: {"status": "healthy"}
- No errors in logs
```

PREVENTION:

- Implement config validation in CI/CD pipeline
- Add automated health check after deployments
- Review all service file changes before applying

LESSONS LEARNED:

- Always check logs first (`journalctl -u SERVICE`)
 - `systemd` status shows exit codes that indicate failure type
 - Small typos in system files can cause complete service failures
- EOF

```
cat ~/devops-tasks/logs/incident-report-L1.txt
# To View Incident Report Just Created
```

Acceptance Criteria

Check all boxes:

- ☐ Service status shows "active (running)"
- ☐ Health endpoint returns 200 OK # `curl -i http://localhost:8080/health`
- ☐ Logs show successful startup
- ☐ Incident report created and complete
- ☐ Can explain root cause in your own words
- ☐ I can explain *why* this failure happened without looking at the solution

What You Learned

Technical Skills:

- Using `systemctl status` to check service state
- Reading `journalctl` logs to find errors
- Editing `systemd` service files
- Using `daemon-reload` after config changes
- Testing service health endpoints

Troubleshooting Process:

- Following the 7-step framework
- Reading error messages carefully
- Forming and testing hypotheses
- Documenting findings

Real-World Context:

- This exact scenario happens in production
- Fast response minimizes business impact
- Documentation helps prevent recurrence

Why This Task Matters

This task was not about Python, it was not about systemd syntax, it was about learning how to:

- Stay calm under pressure
- Use logs instead of guessing
- Make small, safe fixes
- Restore service quickly
- Document clearly for the next engineer

This is real DevOps work.

TASK L2: Harden SSH Access for Security Compliance

Business Context

Ticket ID: DEVOPS-1302

Priority: High

Assignee: You

Situation: Security audit flagged the EC2 instance for weak SSH configuration. The instance currently allows password authentication and root login, both of which violate company security policy.

Impact: Failed compliance audit, potential security breach, blocked production deployment.

Your Mission

Harden SSH configuration to meet security requirements while maintaining your access.

Security Requirements

Must implement:

1. Disable password authentication (key only)
2. Disable root login
3. Set idle timeout (5 minutes)
4. Limit authentication attempts (3 max)
5. Disable empty passwords
6. Disable X11 forwarding (*X11 forwarding lets you run graphical applications on a remote server and display them on your local machine through an SSH connection.*)

Constraints

- **CRITICAL:** Do NOT lock yourself out
Safety guarantee (read once)

- You will not lock yourself out if you:
- Keep your current SSH session open
- Test the config before restarting SSH
- Verify a second connection works first

These three steps are your safety net. Follow them and you're safe.

Deliverables

1. Hardened `/etc/ssh/sshd_config` file
2. Backup of original configuration
3. Test results showing new config works
4. Security checklist document

Step-by-Step Guide

Step 1: Backup Current Configuration

ALWAYS backup before modifying system files:

```
# Create backup with timestamp
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup.$(date
+%Y%m%d)

# Verify backup exists
ls -lh /etc/ssh/sshd_config*
```

Step 2: Review Current Configuration

```
# View current SSH settings (non-commented lines only)
grep -v '^#' /etc/ssh/sshd_config | grep -v '^$'
```

Note the current settings (likely insecure defaults).

Step 3: Apply Security Hardening

How to think about these changes

You are changing three categories of settings:

- **Access control (high impact):**
`PermitRootLogin, PasswordAuthentication, AllowUsers`
- **Brute-force protection (safe):**
`MaxAuthTries, PermitEmptyPasswords`
- **Session hygiene (safe):**
`ClientAliveInterval, ClientAliveCountMax, X11Forwarding`

Only access control settings can lock you out, that's why you test before restarting.

```
# Edit SSH config
sudo vim /etc/ssh/sshd_config

# Editor note
# If you are not comfortable with vim, you may use:

# sudo nano /etc/ssh/sshd_config

# The security outcome is the same. The editor does not matter.
```

Find and modify these lines (or add if missing):

Press / in vim to search, then type the setting name.

To Edit Type *Shift i* on mac

```
# Disable root login
PermitRootLogin no

# Disable password authentication
PasswordAuthentication no
```

```
# Ensure key authentication is enabled
PubkeyAuthentication yes

# Set maximum authentication attempts
MaxAuthTries 3

# Set idle timeout (5 minutes)
ClientAliveInterval 300
ClientAliveCountMax 0

# Disable empty passwords
PermitEmptyPasswords no

# Disable X11 forwarding
X11Forwarding no

# Optional: Restrict to specific users (replace 'ubuntu' with your
username)
AllowUsers ubuntu
```

Save and exit: Press **Esc**, then type **:wq** and press **Enter**.

Step 4: Validate Configuration

CRITICAL: Test config before restarting SSH!

```
# Test configuration syntax
sudo sshd -t

(The sudo sshd -t command tests the SSH daemon configuration file
(/etc/ssh/sshd_config) for syntax errors without starting the SSH
service or applying changes.)
```

If you see errors:

- Review the error message

- Edit `/etc/ssh/sshd_config` again
- Fix the syntax error
- Run `sudo sshd -t` again

Expected output if valid: (no output = success)

Why this step matters

`sshd -t` is your last line of defense.

If this passes, SSH will start. If it fails, SSH will not start, no guessing required.

Decision checkpoint

Do not restart SSH unless:

- A new SSH session works
- Your original session is still open
- `sshd -t` returned no errors

If all three are true, restarting SSH is safe.

Step 5: Restart SSH Service

IMPORTANT: Keep your current SSH session open!

```
# Open a NEW terminal window (do NOT close current session)
# This is your safety net if something goes wrong

# In the NEW terminal, try to connect
ssh devops-lab
Or
ssh -o ServerAliveInterval=60 -o ServerAliveCountMax=3 -i
~/.ssh/devops-lab-key.pem ubuntu@44.201.91.255

# If connection works, you're safe
# If connection fails, you still have your original session to fix
issues
```

Once you've verified the new connection works:

```
# In your ORIGINAL session, restart SSH
sudo systemctl enable ssh
sudo systemctl enable ssh
sudo systemctl restart sshd

# Check status
```

```
sudo systemctl status sshd
# Should show: active (running)
```

Step 6: Verify Security Settings

```
# Verify the active configuration
sudo sshd -T | grep -E
'permitrootlogin|passwordauthentication|pubkeyauthentication|maxauthtr
ies|clientaliveinterval|permitemptypasswords|x11forwarding|allowuser
s'
```

Expected output:

```
permitrootlogin no
passwordauthentication no
pubkeyauthentication yes
maxauthtries 3
clientaliveinterval 300
clientalivecountmax 0
permitemptypasswords no
x11forwarding no
allowusers ubuntu
```

Step 7: Test from Fresh Connection

```
# Disconnect from server
exit

# Reconnect using your SSH key
ssh devops-lab

# Should connect successfully using key authentication
```

Try password authentication (should fail):

You can't easily test this since you're already using keys, but the config ensures passwords won't work.

Step 8: Document Changes

Create security compliance document

```
cat > ~/devops-tasks/configs/ssh-hardening-report.md << 'EOF'
# SSH Security Hardening Report

**Date:** 2025-12-04
**System:** EC2 DevOps Lab Instance
**Performed by:** [Your Name]

## Changes Applied

### 1. Authentication Security
- Disabled password authentication (`PasswordAuthentication no`)
- Disabled root login (`PermitRootLogin no`)
- Enabled key-based authentication only
- Limited authentication attempts to 3 (`MaxAuthTries 3`)
- Disabled empty passwords (`PermitEmptyPasswords no`)

### 2. Session Security
- Enabled idle timeout: 5 minutes (`ClientAliveInterval 300`)
- Disconnect after timeout (`ClientAliveCountMax 0`)

### 3. Additional Hardening
- Disabled X11 forwarding (`X11Forwarding no`)
- Restricted login to specific user (`AllowUsers ubuntu`)

## Verification

**Configuration Test:**

sudo sshd -t (no errors)

**Active Configuration:**

sudo sshd -T | grep permitrootlogin permitrootlogin no

**Connection Test:**
```

```
ssh devops-lab (successful connection using SSH key)
```

```
## Backup Location
```

```
Original configuration backed up to:  
`/etc/ssh/sshd_config.backup.20251204`
```

```
## Rollback Procedure
```

```
If issues occur:
```

```
sudo cp /etc/ssh/sshd_config.backup.20251204 /etc/ssh/sshd_config  
sudo systemctl restart sshd
```

```
Compliance Status
```

```
PASS - All security requirements met  
TESTED - Configuration verified working  
DOCUMENTED - Changes recorded  
REVERSIBLE - Backup created
```

```
Next Steps
```

```
- Schedule quarterly SSH config reviews  
- Monitor auth logs for failed attempts: sudo journalctl -u sshd |  
grep Failed  
- Consider implementing fail2ban for additional protection  
EOF
```

```
cat ~/devops-tasks/configs/ssh-hardening-report.md
```


Acceptance Criteria

Check all boxes:

- ☐ SSH config validated with `sshd -t` (no errors)
- ☐ Can still connect using SSH key
- ☐ Configuration changes documented
- ☐ Backup created
- ☐ All 6 security requirements implemented
- ☐ Service restarted successfully
- ☐ Active config verified with `sshd -T`

If all acceptance criteria are checked, stop here.

You have met security requirements and restored compliance.

What You Learned

L1 was about fixing a broken service.

L2 is about changing security without breaking access — which is harder.

Feeling cautious here is correct.

Technical Skills:

- Editing critical system configuration files
- Testing config syntax before applying
- Using ``sshd -t`` and ``sshd -T``
- Managing SSH daemon with `systemd`
- Creating backups before changes

Security Concepts:

- Key-based vs password authentication
- Principle of least privilege
- Defense in depth (multiple layers)
- Session timeout policies

- Attack surface reduction

Risk Management:

- Always keep backup session open
- Test before committing changes
- Create rollback plans
- Validate after changes

Real-World Context:

- SSH is the primary attack vector for cloud servers
- Security audits are mandatory in enterprises
- One misconfiguration can lock you out
- Documentation proves compliance

TASK L3: Automate System Health Checks

Business Context

Ticket ID: DEVOPS-1355

Priority: Medium

Assignee: You

Situation: The team needs proactive monitoring of critical services. Instead of waiting for alerts from customers, we want automated checks that run every 5 minutes and log results.

Impact: Faster incident detection, reduced MTTR (Mean Time To Recovery), improved SLA compliance.

Your Mission

Create an automated health check system using Bash scripting and systemd timers.(remember what systemd does from task L1 right?)

Here, You are building a **very simple monitoring system**:

- A script that checks system health
- A service that runs the script
- A timer that schedules it every 5 minutes

Nothing alerts users yet, this only **records system health over time**.

Requirements

The health check must:

1. Check if nginx is running
2. Check if payments-api is running
3. Check disk usage (warn if > 80%)
4. Check memory usage (warn if > 90%)
5. Log results to `/var/log/health-check.log`

6. Run every 5 minutes automatically
7. Be idempotent (safe to run multiple times)

Deliverables

1. Bash script: `/usr/local/bin/health-check.sh`
2. systemd timer: `/etc/systemd/system/health-check.timer`
3. systemd service: `/etc/systemd/system/health-check.service`
4. Test results showing it works
5. Documentation

Step-by-Step Guide

Step 1: Create the Health Check Script

```
# Create the script  
sudo vim /usr/local/bin/health-check.sh
```

Copy this script:

```
#!/bin/bash  
#  
# health-check.sh - Automated system health monitoring  
# Runs every 5 minutes via systemd timer  
#  
  
# Configuration  
LOG_FILE="/var/log/health-check.log"  
SERVICES=("nginx" "payments-api")  
DISK_THRESHOLD=80  
MEMORY_THRESHOLD=90  
  
# Note
```

```
# This script uses sudo for simplicity and clarity.
# When run via systemd, it already executes with elevated privileges,
# so sudo is technically redundant.
# We'll clean this up later -- for now, focus on what the script #
does, not optimization.

# Ensure log file exists
sudo touch "$LOG_FILE"
sudo chmod 644 "$LOG_FILE"

# Function: Log with timestamp
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | sudo tee -a
"$LOG_FILE"
}

# Header
log "=== Health Check Started ==="

# Check services
for service in "${SERVICES[@]}; do
    if systemctl is-active --quiet "$service"; then
        log "✅ $service: RUNNING"
    else
        log "❌ $service: DOWN (CRITICAL)"
    fi
done

# Check disk usage
DISK_USAGE=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')
if [ "$DISK_USAGE" -gt "$DISK_THRESHOLD" ]; then
    log "⚠️ Disk usage: ${DISK_USAGE}% (WARNING: Above
${DISK_THRESHOLD}%)"
else
    log "✅ Disk usage: ${DISK_USAGE}%"
fi

# Check memory usage
MEMORY_USAGE=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
```

```
if [ "$MEMORY_USAGE" -gt "$MEMORY_THRESHOLD" ]; then
    log "⚠ Memory usage: ${MEMORY_USAGE}% (WARNING: Above
${MEMORY_THRESHOLD}%)"
else
    log "✅ Memory usage: ${MEMORY_USAGE}%"
fi

log "=== Health Check Complete ==="
log ""

exit 0
```

Save and exit: `Esc` → `:wq`

Step 2: Make Script Executable

```
# Set executable permission
sudo chmod +x /usr/local/bin/health-check.sh
```

```
# Verify permissions
ls -lh /usr/local/bin/health-check.sh
# Should show: -rwxr-xr-x
```

Step 3: Test the Script Manually

Before automating anything, you always test it by hand.
This confirms the script itself works *before* adding systemd timers, which makes troubleshooting much harder.

If the script fails here, the problem is the script, not systemd.

```
# Run the script
sudo /usr/local/bin/health-check.sh
# Check the log file
sudo cat /var/log/health-check.log
```

Expected output:

```
[2025-12-04 10:30:00] === Health Check Started ===  
[2025-12-04 10:30:00] nginx: RUNNING  
[2025-12-04 10:30:00] payments-api: RUNNING  
[2025-12-04 10:30:00] Disk usage: 15%  
[2025-12-04 10:30:00] Memory usage: 35%  
[2025-12-04 10:30:00] === Health Check Complete ===
```

Script works!

Important concept

- The script does the work
- The service defines *how* the script runs
- The timer defines *when* it runs

systemd separates *execution* from *scheduling*, this is intentional.

Step 4: Create systemd Service

Create service file

sudo vim /etc/systemd/system/health-check.service

Add this content:

```
[Unit]  
Description=System Health Check Service  
Documentation=man:systemd.service(5)  
  
[Service]  
Type=oneshot  
ExecStart=/usr/local/bin/health-check.sh  
StandardOutput=journal  
StandardError=journal
```

This systemd service file defines a one-shot service that runs the health check script when triggered and logs its output to the systemd journal.

Save and exit. (:wq)

Step 5: Create systemd Timer

```
# Create timer file
sudo vim /etc/systemd/system/health-check.timer
```

Add this content:

This systemd timer runs the health-check service 2 minutes after boot, then every 5 minutes.

```
[Unit]
Description=System Health Check Timer
Requires=health-check.service

[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
Unit=health-check.service

[Install]
WantedBy=timers.target
```

Explanation:

- `OnBootSec=2min` → Run 2 minutes after boot
- `OnUnitActiveSec=5min` → Run every 5 minutes after last execution
- `Requires=health-check.service` → Links timer to service

Save and exit.

Step 6: Enable and Start the Timer

```
# Reload systemd to recognize new files
sudo systemctl daemon-reload
# Enable timer (start on boot)
sudo systemctl enable health-check.timer
# Start timer now
sudo systemctl start health-check.timer
# Check timer status
sudo systemctl status health-check.timer
# Should show: active (waiting)
```

Step 7: Verify Timer is Scheduled

```
# List all active timers
sudo systemctl list-timers --all | grep health-check
```

Expected output:

```
ubuntu@ip-172-31-71-160:~$ sudo systemctl list-timers --all | grep health-check
Thu 2025-12-04 22:13:16 UTC      4min 23s Thu 2025-12-04 22:08:16 UTC      36s ago health-check.timer      health-check.service
ubuntu@ip-172-31-71-160:~$
```

Shows when the next execution will happen.

Timers can be invisible by design. You verify them by checking:

- `systemctl list-timers`
- log output over time

Step 8: Wait and Verify Automatic Execution

```
# Wait 5 minutes, then check log
# (or manually trigger for testing)

# Manual trigger for immediate test:
sudo systemctl start health-check.service

# Check service ran successfully
sudo systemctl status health-check.service
# Should show: "Finished System Health Check Service"

# Check the log file
sudo tail -20 /var/log/health-check.log
```

You should see multiple entries as the timer runs every 5 minutes.

Step 9: Test Failure Scenarios

Simulate a service failure:

```
# Stop nginx
sudo systemctl stop nginx

# Wait for next health check (or trigger manually)
sudo systemctl start health-check.service
# Check log
sudo tail -10 /var/log/health-check.log
```

Expected output:

```
[2025-12-04 10:40:00] === Health Check Started ===
[2025-12-04 10:40:00] ❌ nginx: DOWN (CRITICAL)
[2025-12-04 10:40:00] ✅ payments-api: RUNNING
...
```

Script correctly detects failure!

Restore nginx:

```
sudo systemctl start nginx
```

Step 10: Document the System

Create documentation

```
cat > ~/devops-tasks/scripts/health-check-documentation.md << 'EOF'
# Automated Health Check System

## Overview

Automated monitoring system that checks critical services and system
resources every 5 minutes.

## Components

### 1. Health Check Script
**Location:** `/usr/local/bin/health-check.sh`

**Checks performed:**
- nginx service status
- payments-api service status
- Disk usage (warns if > 80%)
- Memory usage (warns if > 90%)

**Log location:** `/var/log/health-check.log`

### 2. systemd Service
**Location:** `/etc/systemd/system/health-check.service`

Executes the health check script.
```

3. systemd Timer

****Location:**** `/etc/systemd/system/health-check.timer``

Schedules execution every 5 minutes.

Usage

Check timer status

````bash`

`sudo systemctl status health-check.timer`

View scheduled runs

`sudo systemctl list-timers | grep health-check`

Manually trigger a check

`sudo systemctl start health-check.service`

View logs

`sudo tail -f /var/log/health-check.log`

Stop automated checks

`sudo systemctl stop health-check.timer`

`sudo systemctl disable health-check.timer`

Start automated checks

`sudo systemctl start health-check.timer`

`sudo systemctl enable health-check.timer`

Log Analysis

Count total checks run

`grep "Health Check Started" /var/log/health-check.log | wc -l`

Find all failures

`grep "DOWN" /var/log/health-check.log`

Find warnings

```
grep "WARNING" /var/log/health-check.log
```

## Customization

Change check interval

Edit /etc/systemd/system/health-check.timer:

```
OnUnitActiveSec=5min # Change to desired interval
```

Then reload:

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart health-check.timer
```

Add more services to check

Edit /usr/local/bin/health-check.sh:

```
SERVICES=("nginx" "payments-api" "postgresql" "redis")
```

Adjust thresholds

Edit /usr/local/bin/health-check.sh:

```
DISK_THRESHOLD=80 # Change disk warning level
```

```
MEMORY_THRESHOLD=90 # Change memory warning level
```

## Troubleshooting

Timer not running

```
sudo systemctl status health-check.timer
```

```
sudo journalctl -u health-check.timer -n 50
```

Script failing

```
sudo journalctl -u health-check.service -n 50
```

Test script manually

```
sudo /usr/local/bin/health-check.sh
```

## Future Enhancements

Add email/Slack notifications on failures

Integrate with Prometheus for metrics collection

Add response time checks for HTTP endpoints

Include network connectivity checks

```
Add automatic remediation for common issues
EOF
```

```
cat ~/devops-tasks/scripts/health-check-documentation.md
```

## Acceptance Criteria

If all boxes are checked, stop here. Your monitoring system is complete and running automatically.

### Check all boxes:

- ☐ Script created and executable
- ☐ Script runs successfully when executed manually
- ☐ systemd service created
- ☐ systemd timer created and enabled
- ☐ Timer appears in `systemctl list-timers`
- ☐ Logs being written to `/var/log/health-check.log`
- ☐ Script correctly detects service failures
- ☐ Documentation created
- ☐ Can explain how systemd timers work

## What You Learned

You don't need to memorize these commands. What matters is knowing *where* to check system health, tools can change.

### Technical Skills:

- Writing production ready Bash scripts
- Using systemd timers (modern alternative to cron)
- Creating systemd service units
- Logging and log management
- Idempotent script design
- Service health checking

## **Bash Scripting Concepts:**

- Functions for code reuse
- Arrays for iterating over lists
- Conditionals for decision logic
- Command substitution (``$(command)``)
- Exit codes for success/failure

## **systemd Concepts:**

- Oneshot vs simple service types
- Timer units and scheduling
- Service dependencies (Requires=)
- Timer activation strategies

## **Operations Concepts:**

- Proactive vs reactive monitoring
- Log aggregation
- Automation reduces toil
- Threshold-based alerting

## **Real World Context:**

- Health checks are fundamental to monitoring
- systemd timers are preferred over cron in modern Linux
- Idempotent scripts prevent issues from repeated runs
- Good logging enables troubleshooting

# TASK L4: Investigate and Resolve High CPU Usage

## Business Context

**Ticket ID:** DEVOPS-1401

**Priority:** High

**Assignee:** You

**Situation:** Monitoring alerts indicate the EC2 instance CPU usage spiked to 95% and has remained high for 20 minutes. Users are reporting slow response times from the payments API.

**Impact:** Degraded user experience, potential service timeouts, SLA breach.

## Your Mission

Identify what's causing high CPU usage, determine if it's a legitimate workload or a problem, and resolve the issue.

This task is not about fixing code, it's about understanding how system behavior affects application performance.

## Why this matters (in plain English)

When CPU stays very high, your server has less “brain power” available to answer requests. That's why users feel the app is slow.

In this lab, you'll learn a calm incident workflow:

- Confirm CPU is truly high
- Identify the exact process causing it
- Trace what started that process (root cause)
- Stop the cause safely (no reboot)
- Verify the payments API still works
- Write down what you found

## Constraints (follow these)

- Do not reboot
- Find root cause before killing anything
- Keep the service available



- Document what happened

## Deliverables

1. Identification of CPU-intensive process
2. Root cause analysis
3. Resolution with verification
4. Performance analysis report

## Step-by-Step Guide

### Step 1: Simulate High CPU Load

First, let's create a CPU-intensive scenario:

Create a safe “CPU problem” (lab simulation)

```
cat > /tmp/cpu-hog.sh << 'EOF'
#!/bin/bash
Simulates high CPU usage
while true; do
 echo "scale=5000; a(1)*4" | bc -l > /dev/null
done
EOF
```

```
chmod +x /tmp/cpu-hog.sh
```

Start a few copies in the background (each one increases CPU usage)

```
/tmp/cpu-hog.sh &
/tmp/cpu-hog.sh &
/tmp/cpu-hog.sh &
```

You do NOT need to understand everything here.

This command simply forces the CPU to do continuous calculations.

The goal is not the math, the goal is to create high CPU usage safely.

**Get the background job PIDs (write them down):**

```
jobs -l
```

```
ubuntu@ip-172-31-71-160:~$ /tmp/cpu-hog.sh &
/tmp/cpu-hog.sh &
/tmp/cpu-hog.sh &
[1] 5333
[2] 5334
[3] 5335
ubuntu@ip-172-31-71-160:~$ jobs -l
[1] 5333 Running /tmp/cpu-hog.sh &
[2]- 5334 Running /tmp/cpu-hog.sh &
[3]+ 5335 Running /tmp/cpu-hog.sh &
ubuntu@ip-172-31-71-160:~$ █
```

**Now you have high CPU load. Time to investigate!**

## Step 2: Observe System Load

You are not expected to master these tools yet.

Each one simply shows the same problem from a different angle.

```
Check overall system load
top
```

**What to look for (ignore everything else):**

- If you see something like **0.0 id** (idle), the CPU has no free time → CPU is saturated.
- Press P to sort by CPU (highest first).
- Write down:
  - Process name (**example:** bc)
  - PID (a number like 4552)

- Press **q** to exit.

**Expected:** You'll see **bc** processes at ~100% CPU each.

```

top - 22:48:38 up 3:35, 11 users, load average: 2.98, 1.92, 0.84
Tasks: 148 total, 4 running, 144 sleeping, 0 stopped, 0 zombie
%Cpu(s):100.0 us, 0.0 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 40.7/914.2 [|||||||||||||||||||||]
MiB Swap: 0.0/0.0 []

```

**Better view with htop:**

```
htop
```

**Features:**

- Color-coded CPU bars (red = kernel, green = user)
- Memory usage visualization
- Process tree (F5)
- Search (F3)

**Press F10 to quit.**

### Step 3: Identify the Culprit Process

```

List top CPU consumers (which exact processes are consuming the CPU)
ps aux --sort=-%cpu | head -10

```

**Output will show:**

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
ubuntu	1234	33.5	0.1	8456	3244	pts/0	R	10:30	5:23	bc -l
ubuntu	1235	33.2	0.1	8456	3244	pts/0	R	10:30	5:20	bc -l
ubuntu	1236	33.8	0.1	8456	3244	pts/0	R	10:30	5:18	bc -l

## Key information:

- **PID:** Process ID
- **%CPU:** CPU usage percentage
- **COMMAND:** What's running

## Get more details about the process:

Replace 1234 with actual PID

```
ps -fp 1234
```

## Find what spawned it:

```
Get parent process
ps -o ppid= -p 1234
```

## Step 4: Analyze the Process

```
Check what files it has open
sudo lsof -p 1234
```

## Check system calls it's making:

```
This shows what the process is actually doing
sudo strace -p 1234 -c -e trace=all
Or
echo "scale=5000; a(1)*4" | strace -c bc -l
Press Ctrl+C after 5-15 seconds to see summary
```

This step shows *what the process is actually doing internally*.

If this feels overwhelming, skim the output and move on, the key takeaway is that the process is busy doing calculations.

Expected output for **bc**:

```
% time seconds calls syscall

45.23 0.000523 1234 read
35.12 0.000405 987 write
...
```

This confirms it's doing intensive calculations.

## Step 5: Determine Root Cause

**Hypothesis:** The **bc** processes are not legitimate workload. They're from a test script accidentally left running.

**Test hypothesis:**

```
Check what script launched bc
pgrep -a bc
Output: 1234 bc -l
5701 bc -l
5705 bc -l

Find parent process
ps -o ppid= -p 1234
Output: 5333

Check what 5333 is
ps -fp 5333
Output: /bin/bash /tmp/cpu-hog.sh
```

**Confirmed:** The **cpu-hog.sh** script is causing the issue.

## Step 6: Resolve

**Decision:** Kill the rogue processes.

```
Kill the parent script that's spawning bc processes
pkill -f cpu-hog.sh

Kill all bc processes
pkill bc

Verify they're gone
pgrep bc
(no output = gone)

Check CPU Load
top
Press 'q' to quit
```

Why it's safe to kill these processes

- The process is not a system service
- It was started manually
- You identified its parent script
- You verified it is not required for app functionality

This is the correct time to terminate it.

**CPU usage should drop to normal (~1-15%).**

**Also kill the parent script:**

```
Kill the script
pkill -f cpu-hog.sh

Clean up
rm /tmp/cpu-hog.sh

Verify
```

```
pgrep -a cpu-hog
(no output = cleaned up)
```

## Step 7: Verify Resolution

```
Check system load
uptime
Load average should be below 1.0

Verify services still running
sudo systemctl status nginx payments-api
Both should show: active (running)

Test API endpoint
curl http://localhost:8080/health
Should respond quickly

Check no rogue processes remain
ps aux --sort=-%cpu | head -10
Should show normal system processes
```

**Issue resolved!**

## Step 8: Document Findings

**Create performance analysis report**

```
cat > ~/devops-tasks/logs/cpu-investigation-report.md << 'EOF'
High CPU Usage Investigation Report

Date: 2025-12-04
Duration: ~30 minutes
Severity: High

Incident Summary
```

EC2 instance CPU usage reached 95% sustained load, causing performance degradation for users accessing the payments API.

## ## Investigation Process

### ### 1. Initial Observation

```
```bash
```

```
top
```

```
htop
```

Findings:

CPU usage: 95%+

Three bc processes each consuming ~99% CPU

Load average: 3.2 (normal: <1.0 for single CPU)

2. Process Identification

```
ps aux --sort=-%cpu | head -10
```

Findings:

Process: bc -l (arbitrary precision calculator)

PIDs: 1234, 1235, 1236

Owner: ubuntu

Runtime: >5 minutes each

3. Root Cause Analysis

```
ps -o ppid= -p 1234 # Found parent PID: 1100
```

```
ps -fp 1100          # Parent: /bin/bash /tmp/cpu-hog.sh
```

Root Cause: Test script /tmp/cpu-hog.sh left running, spawning multiple CPU-intensive processes performing infinite mathematical calculations.

4. Impact Assessment

User impact: Slow API response times

Service availability: Services still running but degraded

System stability: No risk of crash but sustained high load

Resolution

Actions Taken

```
# Kill rogue processes
```

```
pkill bc
```

```
pkill -f cpu-hog.sh
```



```
# Remove test script
```

```
rm /tmp/cpu-hog.sh
```

```
# Verify services operational
```

```
sudo systemctl status nginx payments-api
```

```
curl http://localhost:8080/health
```

Verification

CPU usage dropped to 8% (normal)

Load average: 0.3

API response time: <50ms (normal)

All services: operational

Timeline

10:00 - Alert: High CPU usage detected

10:05 - Investigation started (top, htop)

10:10 - Identified rogue bc processes

10:15 - Traced to cpu-hog.sh test script

10:20 - Killed processes and cleaned up script

10:25 - Verified services and performance restored

10:30 - Documented findings

Lessons Learned

What Went Right

Systematic investigation process

Used multiple tools (top, ps, lsof, strace)

Identified root cause before taking action

Verified service health after resolution

What Went Wrong

Test script left running in production environment

No monitoring alert for unusual processes

Prevention Measures

Immediate

- Clean up test scripts after use

- Document all test activities

Short-term

Create policy: Test scripts must be in /tmp with clear naming

Add monitoring alert for processes consuming >80% CPU for >5 minutes

Implement scheduled cleanup of old files in /tmp

Long-term

Separate test and production environments

Implement resource limits (cgroups) for user processes

Add automated process monitoring dashboard

Commands Used

Investigation

top	# Real-time process view
htop	# Enhanced process viewer
ps aux --sort=-%cpu	# Sort processes by CPU
ps -fp PID	# Detailed process info
lsof -p PID	# Files opened by process
strace -p PID -c	# System calls analysis

Resolution

kill PROCESS_NAME	# Kill by name
kill -f PATTERN	# Kill by command pattern

Verification

uptime	# System load
systemctl status SERVICE	# Service health
curl http://localhost:8080/health	# API health check

Reference Metrics

Before Resolution

CPU usage: 295% (3 cores at 99% each)

Load average: 3.2, 3.1, 2.9 (1/5/15 min)

API latency: 500ms+

After Resolution

CPU usage: 8%

Load average: 0.3, 0.5, 1.2 (decreasing)

API latency: 45ms

Sign-off

Investigated by: [Your Name]

Reviewed by: N/A (Lab environment)

Status:  RESOLVED

EOF

```
cat ~/devops-tasks/logs/cpu-investigation-report.md
```

Acceptance Criteria

If all acceptance criteria are checked, the incident is resolved. Stop here and move to the next task.

Check all boxes:

- ☐ Identified high CPU process using top/htop
- ☐ Found process PID and details
- ☐ Traced process to root cause
- ☐ Killed rogue processes
- ☐ Verified CPU returned to normal
- ☐ Confirmed services still operational
- ☐ Created detailed investigation report
- ☐ Can explain the investigation process

What You Learned

Technical Skills:

- Using `top` and `htop` for real-time monitoring
- Sorting processes by resource usage
- Process investigation with `ps`, `lsof`, `strace` Process management (kill, pkill)
- System load interpretation

Troubleshooting Process:

1. Observe before acting
2. Gather data systematically
3. Form hypothesis before killing processes
4. Verify resolution
5. Document thoroughly

Performance Analysis:

- Understanding CPU usage metrics
- Load average interpretation

- Process states and priorities
- Resource contention

Real World Context:

- High CPU can have many causes (legitimate load, runaway processes, attacks)
- Always identify root cause before taking action
- Services can remain operational under high load (degraded performance)
- Documentation helps prevent recurrence

In Summary:

You're troubleshooting high CPU usage. The task simulates the problem by running CPU-heavy scripts (**bc** doing math), then shows how to find them with **top** and **ps**, investigate with **strace** and **lsof**, identify the cause (a test script left running), and fix it by killing those processes. It's a practice run for diagnosing and fixing performance issues in production.

TASK L5: Log Analysis and Root Cause Report

Business Context

Ticket ID: DEVOPS-1489

Priority: Medium

Assignee: You

Situation: The payments-api service experienced intermittent failures overnight. The on-call engineer restarted the service and it's currently stable, but we need to understand what happened to prevent recurrence.

Impact: Understanding failure patterns prevents future outages and improves system reliability.

Your Mission

Analyze system and application logs to determine why the service failed, identify patterns, and write a comprehensive root cause analysis (RCA).

Why this matters (in plain English)

When a service “fails overnight” and then looks fine after a restart, logs are how you prove what happened, when it happened, and why it happened—so you can prevent it next time.

This lab teaches the workflow:

- Collect logs from the right places (payments-api, nginx, system)
- Find error patterns (what errors, how often)
- Build a timeline (when errors started/stopped)
- Correlate upstream and downstream symptoms (nginx 500 ↔ payments-api errors)
- Write a solid RCA (root cause, impact, recommendations)

Requirements

1. Analyze logs from nginx and payments-api
2. Identify error patterns and frequency
3. Determine timeline of events
4. Identify root cause
5. Provide recommendations

Deliverables

- Log analysis summary
- Root cause analysis document
- Recommendations for prevention

Step-by-Step Guide

Before you start (2 important notes)

1) Service names may differ

On your server, the unit might be `payments-api.service` (systemd). Some commands use the unit name **without** `.service`. If a command returns “Unit not found”, run:

```
systemctl list-units --type=service | grep -i payment
```

2) This lab simulates failures (safe, but be intentional)

Step 1 intentionally generates errors so you have something to analyze. Do this in a **lab EC2** instance, not production.

Step 1: Generate test logs (simulate intermittent failures)

Goal: make the payments API sometimes return 500 and log an error, so you can practice analysis.

1A) Stop the service (so we can change the script safely)

```
sudo systemctl stop payments-api.service
```

Expected output:

```
# (Usually no output on success)
```

If it fails, check status:

```
sudo systemctl status payments-api.service --no-pager
```

1B) Write an error-simulating version of the API

This version logs “Database connection timeout” errors randomly after a few requests.

```
sudo tee /opt/payments-api/api-with-errors.py > /dev/null << 'EOF'
#!/usr/bin/env python3
from http.server import HTTPServer, BaseHTTPRequestHandler
import json
import sys
import random

class PaymentsAPI(BaseHTTPRequestHandler):
    request_count = 0

    def do_GET(self):
        PaymentsAPI.request_count += 1

        # Simulate intermittent failures after 10 requests
        if PaymentsAPI.request_count > 10 and random.random() < 0.3:
            sys.stderr.write("ERROR: Database connection failed -
Connection timeout\n")
            self.send_response(500)
            self.end_headers()
            return

        if self.path == '/health':
            self.send_response(200)
            self.send_header('Content-type', 'application/json')
            self.end_headers()
            response = {'status': 'healthy', 'service':
'payments-api'}
            self.wfile.write(json.dumps(response).encode())
        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        sys.stderr.write(f"INFO: {format % args}\n")

if __name__ == '__main__':
```



```
server = HTTPServer(('0.0.0.0', 8080), PaymentsAPI)
print('Payments API starting with error simulation')
server.serve_forever()
EOF
```

```
sudo chmod +x /opt/payments-api/api-with-errors.py
```

1C) Point the systemd service to the new script

```
sudo sed -i
's|/opt/payments-api/api.py|/opt/payments-api/api-with-errors.py|' \
/etc/systemd/system/payments-api.service
sudo systemctl daemon-reload
sudo systemctl start payments-api.service
```

Expected output:

```
# (Usually no output on success)
```

Confirm it's running:

```
sudo systemctl status payments-api.service --no-pager
```

Expected output:

```
Active: active (running)
```

1D) Generate traffic (this triggers errors)

```
for i in {1..50}; do
  curl -s http://localhost:8080/health > /dev/null
  sleep 0.1
done
```

Quick sanity check in logs:

```
sudo journalctl -u payments-api.service -n 20 --no-pager
```

Expected output:

```
... Payments API starting with error simulation
... INFO: "GET /health HTTP/1.1" 200 -
... ERROR: Database connection failed - Connection timeout
... INFO: "GET /health HTTP/1.1" 500 -
```

Step 2: Collect logs (so you can analyze calmly)

Goal: save the logs into files so you can grep/awk without losing your place.

Create a workspace folder:

```
mkdir -p ~/devops-tasks/logs/analysis
```

Collect payments-api logs (last hour):

```
sudo journalctl -u payments-api.service --since "1 hour ago"
--no-pager > \
~/devops-tasks/logs/analysis/payments-api.log
```

If your file contains only -- No entries --, widen the window or collect “last 10 minutes” right after generating traffic:

```
sudo journalctl -u payments-api.service --since "10 minutes ago"
--no-pager > \
~/devops-tasks/logs/analysis/payments-api.log
```

Collect nginx logs (if nginx exists):

```
sudo cp /var/log/nginx/access.log ~/devops-tasks/logs/analysis/
2>/dev/null || true
```

```
sudo cp /var/log/nginx/error.log ~/devops-tasks/logs/analysis/  
2>/dev/null || true
```

Collect system logs (last hour):

```
sudo journalctl --since "1 hour ago" --no-pager > \  
~/devops-tasks/logs/analysis/system.log
```

Make files readable:

```
sudo chmod 644 ~/devops-tasks/logs/analysis/* 2>/dev/null || true  
ls -lah ~/devops-tasks/logs/analysis
```

Expected output:

```
payments-api.log  
access.log  
error.log  
system.log
```

Step 3: Find error patterns (what failed, how often?)

3A) Count errors

```
grep -c "ERROR" ~/devops-tasks/logs/analysis/payments-api.log
```

Expected output:

```
11
```

If you see `-- No entries --:` or `0`

- It usually means no logs exist yet for that service in the time range you requested.
- Generate a little traffic again (the loop above), then re-run `journalctl`.

- If it still shows no entries, confirm the unit name and that the service is managed by systemd.

3B) Extract errors only (easier to read)

```
grep "ERROR" ~/devops-tasks/logs/analysis/payments-api.log > \
~/devops-tasks/logs/analysis/errors-only.txt
cat ~/devops-tasks/logs/analysis/errors-only.txt
```

Expected output:

```
... ERROR: Database connection failed - Connection timeout
... ERROR: Database connection failed - Connection timeout
```

3C) Identify “types” of errors (group similar messages)

```
grep "ERROR" ~/devops-tasks/logs/analysis/payments-api.log | \
awk -F'ERROR: ' '{print $2}' | \
sort | uniq -c | sort -rn
```

Expected output:

```
11 Database connection failed - Connection timeout
```

Step 4: Build a timeline (when did it start/stop?)

Goal: turn noisy logs into a simple “when things broke” list.

```
grep "ERROR" ~/devops-tasks/logs/analysis/payments-api.log | \
awk '{print $1, $2, $3}' > \
~/devops-tasks/logs/analysis/error-timeline.txt
head ~/devops-tasks/logs/analysis/error-timeline.txt
```

Expected output:

```
Jan 12 22:10:01
Jan 12 22:10:03
Jan 12 22:10:07
```

Tip: Your timestamps will differ, that's normal. You're checking that you can see the pattern.

Step 5: Did the service restart during the incident?

If someone restarted it, logs usually show stop/start events.

```
grep -E "(Started|Stopped|Restarted)"
~/devops-tasks/logs/analysis/payments-api.log
```

Expected output:

```
... Started payments-api.service - Payments API Service
... Stopped payments-api.service - Payments API Service
... Started payments-api.service - Payments API Service
```

If you see nothing, check system logs instead:

```
grep -E "payments-api.service.*(Started|Stopped|Restarted)" \
~/devops-tasks/logs/analysis/system.log | tail -20
```

Step 6: Analyze nginx access patterns (what did users experience?)

Goal: confirm what status codes users got (200/404/500).

6A) Count HTTP status codes

```
awk '{print $9}' ~/devops-tasks/logs/analysis/access.log 2>/dev/null
| \
sort | uniq -c | sort -rn
```

Expected output:

```
220 200
15 500
3 404
```

6B) Show nginx 500 errors (symptom)

```
grep " 500 " ~/devops-tasks/logs/analysis/access.log 2>/dev/null |
head
```

Expected output:

```
127.0.0.1 - - [12/Jan/2026:22:10:03 +0000] "GET /api/health HTTP/1.1"
500 0 "-" "curl/7.81.0"
```

Step 7: Correlate events (do payments-api errors line up with nginx 500s?)

This is the key “aha”: an upstream 500 often comes from a downstream failure.

Count each side:

```
echo "Payments API ERROR count:"
grep -c "ERROR" ~/devops-tasks/logs/analysis/payments-api.log

echo "nginx 500 count:"
grep -c " 500 " ~/devops-tasks/logs/analysis/access.log 2>/dev/null
|| echo 0
```

Expected output:

```
Payments API ERROR count:
15
nginx 500 count:
15
```

If the counts are close and the times overlap, it strongly suggests:

- Root cause lives in payments-api
- nginx is showing the symptom to users

Step 8: Write the RCA (Root Cause Analysis)

Beginner rule: the RCA is not “a wall of text.” It’s a clear story with evidence.

What a good RCA must include

- **Summary:** what broke + customer impact
- **Timeline:** when errors started, peaked, stopped, and why
- **Evidence:** key log lines (not every line)
- **Root cause:** the real reason (not just the symptom)
- **Why restart helped:** explain why it was temporary
- **Recommendations:** prioritized actions (now / soon / later)

Create your RCA file:

```
cat > ~/devops-tasks/logs/root-cause-analysis.md << 'EOF'
# Root Cause Analysis: Payments API Intermittent Failures

## Executive Summary
- What happened: payments-api had intermittent failures overnight.
- Customer impact: some requests failed (HTTP 500), causing unreliable payment attempts.
- Current status: service is stable after restart, but restart is a temporary fix.

## Evidence (Logs)
- payments-api error pattern: repeated `ERROR: Database connection failed - Connection timeout`
- nginx symptom: matching increase in HTTP 500 responses

## Timeline (from logs)
- T0: first timeout error observed
```

```
- **T1:** error rate increased
- **T2:** nginx started returning 500s to users
- **T3:** service restarted
- **T4:** errors stopped (temporary recovery)

## Root Cause (What actually caused the incident?)
**Database connectivity timeouts** under load (simulated in this
lab).

## Why restart helped (but is not a permanent fix)
Restart resets the process state and clears stuck connections, but it
does not remove the underlying cause (timeouts / pool sizing / slow
dependencies).

## Recommendations
### Immediate (Today/This week)
- Add alerting for spikes in 500s and payments-api error rate
- Add deeper health checks (dependency checks)

### Short-term (This month)
- Add better retries/timeouts/backoff
- Add dependency monitoring (database latency, connection failures)

### Long-term (This quarter)
- Load test to find limits
- Capacity planning and SLO-based alerting

EOF
```

View it:

```
cat ~/devops-tasks/logs/root-cause-analysis.md
```

Step 9: Create a short “analysis summary” (executive-friendly)

This is what you’d paste into a ticket or Slack update.

```
cat > ~/devops-tasks/logs/analysis-summary.txt << 'EOF'
```


LOG ANALYSIS SUMMARY

=====

Time range analyzed: last 1 hour

Logs: payments-api (journalctl), nginx (access/error), system (journalctl)

Findings:

- payments-api logged repeated timeout errors.
- nginx returned HTTP 500 during the same window.
- A restart stopped the errors, but the root cause must be addressed to prevent recurrence.

Next steps:

- Add error-rate + 500-rate alerts
- Add dependency health checks and monitoring

EOF

```
cat ~/devops-tasks/logs/analysis-summary.txt
```

Acceptance checklist

Stop when all are true:

- ☐ Collected logs from payments-api, nginx, and system
- ☐ Counted errors and grouped error types
- ☐ Built an error timeline
- ☐ Checked for restart events
- ☐ Verified nginx status-code pattern
- ☐ Correlated nginx 500s with payments-api errors
- ☐ Wrote an RCA document
- ☐ Wrote a short summary

Wrap-up (what you can say you did)

Acceptance Criteria (quick checklist)

- ☐ Collected logs from multiple sources
- ☐ Identified error patterns and frequency
- ☐ Built a timeline of events
- ☐ Determined root cause (not just symptoms)
- ☐ Wrote an RCA with recommendations
- ☐ Produced an executive-friendly summary
- ☐ Can explain why a restart was a temporary fix

What you learned (short)

- **Tools:** `journalctl`, nginx logs, `grep`, `awk`, `sort`, `uniq`
- **Techniques:** pattern → timeline → correlation → root cause → recommendations
- **Ops thinking:** restart restores service fast, but doesn't remove the underlying failure mode

Portfolio note

If you saved your outputs under `~/devops-tasks/logs/` (RCA + summary + collected logs), this is solid evidence of real incident work you can show in interviews.

What You've Accomplished

L5 teaches you how to **use logs (payments-api, nginx, and system logs)** to reconstruct what happened during an incident by finding error patterns, building a timeline, and correlating symptoms (like nginx 500s) with the real cause in the service. This matters because services can look “fixed” after a restart, but only log evidence tells you the **root cause** and what changes prevent the same outage from happening again. By the end, you produce a **root-cause analysis (RCA)** and a short summary showing what failed, when it failed, why the restart was temporary, and the recommendations to prevent recurrence.

You've completed 5 real world DevOps tasks:

- **Task L1:** Troubleshoot and restored a broken service
- **Task L2:** Hardened SSH for security compliance
- **Task L3:** Automated health checks with systemd timers
- **Task L4:** Investigated and resolved high CPU usage
- **Task L5:** Performed log analysis and wrote RCA

Skills You've Mastered

Linux Fundamentals:

- systemd service management
- Service status checking
- Log analysis with journalctl
- Process management
- File editing and permissions

Troubleshooting:

- Systematic investigation process
- Hypothesis-driven debugging
- Using logs to find root cause
- Verification after fixes

Automation:

- Bash scripting
- systemd timers
- Idempotent script design
- Logging and monitoring

Security:

- SSH hardening
- Configuration validation
- Risk management

Operations:

- Incident response
- Performance analysis
- Root cause analysis
- Documentation

This is real work you can show in interviews.

SECTION 2.5: Bash Essentials (Beginner-Friendly)

Why this section matters

In Task L3 you wrote your first Bash automation. Now you'll learn the few Bash skills that make scripts safe, reusable, and professional, the kind DevOps teams trust.

By the end, you'll be able to:

- **Reuse code** with functions
- **Avoid common mistakes** (especially with spaces and quoting)
- **Handle errors** instead of silently continuing
- Write scripts that are **idempotent** (safe to run multiple times)

The 8 Bash basics you actually need

1) Variables (store reusable information)

Think of a variable like a **label** on a value.

- You **save** a value once (assignment)
- Then you **reuse** it many times (expansion)

Rules:

- No spaces around `=` (Bash reads spaces as “new words”)
- When you use a variable, prefer quotes: `"$var"` (prevents space bugs)

```
app_name="payments-api.service"
echo "Checking $app_name..."
systemctl status "$app_name" --no-pager
```

What Bash is “thinking” here:

- `app_name="payments-api.service"` means: store the text `payments-api.service` in a box labeled `app_name`
- `$app_name` means: replace this with the value inside that box

- `systemctl status "$app_name"` becomes: `systemctl status payments-api.service`

Expected output:

```
Checking payments-api.service...
• payments-api.service - Payments API Service
  Active: active (running)
```

Common beginner mistakes:

- Wrong: `app_name = "payments-api.service"` (spaces break assignment)
- Wrong: `systemctl status $app_name` (can break later if the value contains spaces)

2) Quotes (the #1 beginner bug)

Quotes tell the shell: “treat what’s inside as one item, even if it has spaces.”

Without quotes (breaks with spaces)

```
file_name="my document.txt"
cat $file_name
```

What can happen:

- The shell splits it into two words: `my` and `document.txt`
- `cat` may print an error like “No such file or directory” because it is looking for the wrong filename

With quotes (correct)

```
file_name="my document.txt"
echo "hello from a file with spaces" > "$file_name"
cat "$file_name"
```

Expected Output:

```
hello from a file with spaces
```

Simple rule: Quote variables unless you deliberately want word-splitting.

Quick way to “see” the difference:

```
file_name="my document.txt"
printf 'WITHOUT quotes: <%s>\n' $file_name
printf 'WITH quotes:      <%s>\n' "$file_name"
```

Expected output:

```
WITHOUT quotes: <my>
WITHOUT quotes: <document.txt>
WITH quotes:    <my document.txt>
```

3) Command substitution (**`$(...)`**)

Use this when you want to run a command and save its output into a variable.

```
current_date=$(date +%Y-%m-%d)
echo "Today is: $current_date"
```

What Bash is “thinking”:

- Run `date +%Y-%m-%d`
- Take whatever it prints (example: `2026-01-15`)
- Store it in `current_date`
- Then `$current_date` expands to that saved value

Expected output:

```
Today is: 2026-01-15
```

Why it's useful:

- Run a command once, reuse the result
- Make filenames/log lines dynamic (timestamps)

Common mistake:

- Writing `current_date=date +%Y-%m-%d` (this does not run the command)

4) Functions (reuse code, reduce copy/paste)

Functions are named blocks of code you can call like commands. They help you avoid copy/paste and make scripts easier to read.

```
cat > ~/check-services.sh << 'EOF'
#!/bin/bash
set -euo pipefail

check_service() {
    local service_name="$1"

    if systemctl is-active --quiet "$service_name"; then
        echo "$service_name: running"
    else
        echo "$service_name: DOWN"
        return 1
    fi
}

check_service "nginx"
check_service "payments-api.service"
EOF
```



```
chmod +x ~/check-services.sh
~/check-services.sh
```

Key ideas:

- `$1` is the first argument you pass into the function (here: "nginx")
- `local` keeps variables from leaking outside the function

What Bash is “thinking” when you call `check_service "nginx"`:

- Set `$1` to `nginx`
- So `service_name="$1"` becomes `service_name="nginx"`
- Then run the `systemctl is-active ...` check

Expected output:

```
nginx: running
payments-api.service: running
```

Common mistakes:

- Calling the function before it is defined (Bash reads top → bottom)
- Forgetting quotes when passing arguments with spaces

5) If statements (make decisions)

If statements let your script choose what to do.

```
if [ -f "/etc/nginx/nginx.conf" ]; then
    echo "nginx config exists"
else
    echo "nginx config missing"
fi
```

What Bash is “thinking”:

- `[ -f "/etc/nginx/nginx.conf" ]` is a test that returns success/failure

- If the test succeeds → run the “then” block
- If it fails → run the “else” block

Expected output:

```
nginx config exists
```

Important: spaces matter:

```
[ -f "/etc/passwd" ]      # correct  
[-f "/etc/passwd"]      # wrong (no spaces)
```

Common beginner mistake:

- Forgetting quotes inside tests: `[ -f $file ]` (breaks if `$file` is empty or has spaces)

6) Loops (repeat without copy/paste)

For loop

```
for service in nginx payments-api.service; do  
    echo "Checking $service"  
    systemctl is-active --quiet "$service" && echo "OK" || echo "DOWN"  
done
```

What Bash is “thinking”:

- Set `service=nginx`, run the body
- Then set `service=payments-api.service`, run the body again

Expected output:

```
Checking nginx  
OK
```

```
Checking payments-api.service
OK
```

While loop (repeat while something is true)

```
attempts=0
while ! systemctl is-active --quiet nginx; do
    attempts=$((attempts + 1))
    echo "Waiting for nginx... attempt $attempts"
    sleep 1

    if [ "$attempts" -ge 5 ]; then
        echo "Timeout waiting for nginx"
        exit 1
    fi
done
```

What Bash is “thinking”:

- “While nginx is NOT active, keep looping”
- Each loop increments `attempts` so you don’t wait forever

Common mistake:

- Infinite loops with no timeout (scripts that never end)

7) Error handling (don’t ignore failures)

Use this simple pattern so your script stops when something breaks:

```
if systemctl restart nginx; then
    echo "nginx restarted"
else
    echo "nginx restart FAILED"
    exit 1
fi
```

What this means:

If the command works, the script continues. If it fails, the script prints an error and stops.

Why this matters:

A common beginner mistake is letting scripts keep running after a failure and assuming everything worked when it didn't.

8) Strict mode (catch mistakes early)

Use this at the top of most scripts:

```
#!/bin/bash
set -euo pipefail
```

What it does (simple):

- `-e` → stop if any command fails
- `-u` → error if you use a variable that doesn't exist
- `pipefail` → fail if *any* command in a pipeline fails

Why this helps:

Without this, scripts can fail silently and still print “done”.

With strict mode, the script stops immediately so you fix the real issue.

Idempotency (safe to run again)

Idempotent = running the script multiple times gives the same result.

Bad example (not idempotent)

Adds duplicates every time:

```
echo "BACKUP_DIR=/tmp/backups" >> ~/.bashrc
```

Run twice → duplicated lines.

Good example (idempotent)

Only adds the line if it's missing:

```
if ! grep -q 'BACKUP_DIR=/tmp/backups' ~/.bashrc 2>/dev/null; then
    echo "BACKUP_DIR=/tmp/backups" >> ~/.bashrc
fi
```

What's happening:

- Check if the line exists
- If not found → add it
- If found → do nothing

Safe repeat example (directories)

```
mkdir -p /tmp/backups
```

Meaning:

Create it if missing. If it already exists, don't fail.

Common beginner mistake

Scripts that:

- Append the same line every run
- Re-create things that already exist
- Break on the second execution

Simple script template

```
cat > ~/template.sh << 'EOF'
#!/bin/bash
set -euo pipefail

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"
}

main() {
```

```
log "Script started"
# your code here
log "Script completed"
}

main "$@"
EOF

chmod +x ~/template.sh
~/template.sh
```

Why this structure works:

- `main()` keeps the script readable top → bottom
- `log()` avoids messy `echo` everywhere
- `"$@"` passes any arguments to the script

Example output:

```
[2026-01-15 01:55:00] Script started
[2026-01-15 01:55:01] Script completed
```

Part 3: Hands On Bash Tasks

Now let's write production ready automation scripts.

TASK B1: Write a Log Backup Script

Business Context

Ticket ID: DEVOPS-1520

Priority: Medium

Assignee: You

Situation: Log files are consuming disk space. Need automated compression of old logs.

Your Mission

Write a script that:

- Finds log files older than 7 days
- Compresses them with gzip
- Logs what it did
- Is idempotent (safe to run multiple times)

Deliverables (What You'll Create)

Main Artifact:

- **Script file:** `~/devops-tasks/scripts/backup-logs.sh`
 - This is the **final deliverable** you'll have at the end
 - A working Bash script that compresses old logs
 - Must be executable and tested

Supporting Artifacts:

- Test verification (you'll run it and show it works)
- Script log file: `/var/log/backup-logs-script.log` (created automatically by the script)

What You'll Have at the End

After completing this task, you'll have:

1. **One script file:** `~/devops-tasks/scripts/backup-logs.sh` (the main artifact)
2. **Proof it works:** Test output showing old logs were compressed
3. **A reusable automation** you can add to cron or systemd timers

Step-by-Step Guide

Step 1: Create the script file

```
mkdir -p ~/devops-tasks/scripts
vim ~/devops-tasks/scripts/backup-logs.sh
```

Step 2: Write the script

```
#!/bin/bash
#
# backup-logs.sh - Compress old log files to save disk space
#
# Usage: sudo bash ./backup-logs.sh
#
set -euo pipefail

# Configuration
readonly LOG_DIR="/var/log"
readonly AGE_DAYS=7
readonly SCRIPT_LOG="/var/log/backup-logs-script.log"
# Function: Log with timestamp
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | sudo tee -a
"$SCRIPT_LOG"
}

# Function: Check if running as root
```



```

check_root() {
    if [ $EUID -ne 0 ]; then
        echo "ERROR: This script must be run as root"
        exit 1
    fi
}

# Function: Compress old logs
compress_old_logs() {
    log "Starting log compression..."

    local file_count=0
    local compressed_count=0

    # Find and compress old .log files
    while IFS= read -r logfile; do
        file_count=$((file_count + 1))

        # Skip if already compressed
        if [[ "$logfile" == *.gz ]]; then
            log "Skipping already compressed: $logfile"
            continue
        fi

        # Compress the file
        log "Compressing: $logfile"
        if gzip "$logfile"; then
            compressed_count=$((compressed_count + 1))
        fi
    done < <(find "$LOG_DIR" -name "*.log" -type f -mtime +$AGE_DAYS)

    log "Compression complete: $compressed_count/$file_count files
compressed"
}

# Main function
main() {
    log "Log Backup Script Started"
    check_root

```

```
    compress_old_logs
    log "Log Backup Script Completed"
}

main "$@"
```

Save: :wq

Step 3: Make executable

```
chmod +x ~/devops-tasks/scripts/backup-logs.sh
```

Step 4: Create test files

Why: Creates a safe, fake log folder and artificial "old" log files (dated 10 days ago) so you can test your cleanup/backup script without touching real [/var/log](#) production logs.

```
# Create test directory
sudo mkdir -p /var/log/test-app

# Create old log files (10 days ago)
sudo touch -t $(date -d '10 days ago' +%Y%m%d0000)
/var/log/test-app/old-app.log
sudo touch -t $(date -d '10 days ago' +%Y%m%d0000)
/var/log/test-app/old-error.log

# Create recent log (should NOT be compressed)
sudo touch /var/log/test-app/recent.log

# Add content
echo "Old log entry" | sudo tee /var/log/test-app/old-app.log
echo "Recent log entry" | sudo tee /var/log/test-app/recent.log
```

Step 5: Run and verify

```
# Run script (use bash explicitly to avoid pipefail error)
```

```
sudo bash ~/devops-tasks/scripts/backup-logs.sh
```

Expected output (example):

```
[2026-01-17 18:28:45] Log Backup Script Started  
[2026-01-17 18:28:45] Starting log compression...  
[2026-01-17 18:28:45] Compressing: /var/log/test-app/old-app.log  
[2026-01-17 18:28:45] Compressing: /var/log/test-app/old-error.log  
[2026-01-17 18:28:45] Compression complete: 2/2 files compressed  
[2026-01-17 18:28:45] Log Backup Script Completed
```

Check results:

```
ls -lh /var/log/test-app/
```

Expected:

- old-app.log.gz (compressed)
- old-error.log.gz (compressed)
- recent.log (NOT compressed - too recent)

Test idempotency (run again):

```
sudo bash ~/devops-tasks/scripts/backup-logs.sh
```

Expected: Script should skip .gz files and show "0/0 files compressed"

Clean up test files:

```
sudo rm -rf /var/log/test-app
```

Acceptance Criteria

Check all boxes:

- Script created at `~/devops-tasks/scripts/backup-logs.sh` and executable
- Finds files older than 7 days
- Compresses `.log` files to `.gz`
- Skips already compressed files (idempotent)
- Logs all actions to `/var/log/backup-logs-script.log`
- Tested and verified working

What You Learned

Technical Skills:

- Writing production-ready Bash scripts
- Using `find` with `-mtime` to locate old files
- Compressing files with `gzip`
- Making scripts idempotent (safe to run multiple times)
- Logging script actions

Real World Context:

- Log rotation is a common DevOps task
- Disk space management prevents server crashes
- Idempotent scripts are safe for cron/systemd timers
- This script can be scheduled to run daily/weekly

TASK B2: Write a Service Health Check Script

Business Context

Ticket ID: DEVOPS-1563

Priority: High

Assignee: You

Situation:

The Problem: Right now, if nginx or payments-api stops working, nobody knows until users complain. That means:

- Service could be down for hours before anyone notices
- Users get frustrated and report issues
- Team finds out too late

What You're Building: A script that automatically checks if critical services are running, and **tells the monitoring system** whether everything is OK or not.

How It Works:

1. You write a script that checks: "Is nginx running? Is payments-api running?"
2. **The script returns a number:**
 - **0** = "Everything is OK" (all services running)
 - **1** (or any non-zero) = "Something is broken" (at least one service is down)
3. A monitoring tool runs your script every 5 minutes automatically
4. If the script returns **1**, the monitoring tool immediately:
 - Sends an alert to the team (email, Slack, PagerDuty)
 - Creates an incident ticket
 - Shows a red alert on the dashboard

Example:

```
10:00 AM - Monitoring runs your script → Returns 0 (all good) → No alert
10:05 AM - Monitoring runs your script → Returns 0 (all good) → No alert
10:10 AM - payments-api crashes
10:15 AM - Monitoring runs your script → Returns 1 (payments-api is down!)
```

→ ALERT SENT

10:15 AM - Team gets notified immediately, not when users complain

Why This Matters

Before (Without Your Script):

- Team finds out about problems when users report them
- Could be hours of downtime before anyone knows
- Manual checking required (someone has to remember to check)

After (With Your Script):

- **Automatic detection:** Problems found within 5 minutes
- **Immediate alerts:** Team knows instantly when something breaks
- **No manual work:** Monitoring tool does the checking automatically
- **Faster response:** Team can fix issues before users notice

Real-World Numbers:

- **Without automation:** Average detection time = 30-60 minutes (when users complain)
- **With your script:** Average detection time = 5 minutes (next monitoring check)
- **Result:** 10-12x faster problem detection = less downtime = happier users

What You'll Actually Do

You're writing a script that:

- Checks if **nginx** service is running
- Checks if **payments-api** service is running
- If both are running → script exits with code **0** (success)
- If either is down → script exits with code **1** (failure)

The monitoring tool reads that exit code:

- **0** = "All good, no action needed"
- **1** = "Something's broken, send alerts!"

This is called "exit codes", it's how scripts communicate success/failure to other tools.

Your Mission

Write a script that:

- Checks if **nginx** service is running
- Checks if **payments-api** service is running
- Returns exit code **0** if both are running (success)
- Returns exit code **1** if any service is down (failure)
- Can be called by monitoring tools every 5 minutes

Deliverables (What You'll Create)

Main Artifact:

- **Script file:** **~/devops-tasks/scripts/check-services.sh**
 - This is the final deliverable you'll have at the end
 - A working Bash script that checks services and returns proper exit codes
 - Must be executable and tested

What the Script Does:

- Checks nginx: running = OK, stopped = FAIL
- Checks payments-api: running = OK, stopped = FAIL
- Returns **0** if all services OK, **1** if any service down

Step-by-Step Guide

Step 1: Create the script file

```
mkdir -p ~/devops-tasks/scripts
vim ~/devops-tasks/scripts/check-services.sh
```

Step 2: Write the script

```
#!/bin/bash
#
# check-services.sh - Check if critical services are running
#
# Usage: ./check-services.sh
# Exit codes: 0 = all services OK, 1 = any service down
#
set -euo pipefail

# Services to check
readonly SERVICES=("nginx" "payments-api.service")

# Track if any service is down
failed_services=()

# Check each service
for service in "${SERVICES[@]"; do
    if systemctl is-active --quiet "$service"; then
        echo "✅ $service is running"
    else
        echo "❌ $service is DOWN"
        failed_services+=("$service")
    fi
done

# Return exit code based on results
if [ ${#failed_services[@]} -eq 0 ]; then
    echo "All services are healthy"
    exit 0
```



```
else
    echo "ERROR: ${#failed_services[@]} service(s) down:
    ${failed_services[*]}"
    exit 1
fi
```

Save: `:wq`

Step 3: Make executable

```
chmod +x ~/devops-tasks/scripts/check-services.sh
```

Step 4: Test the script

Test 1: All services running (should return 0)

```
~/devops-tasks/scripts/check-services.sh
echo "Exit code: $?"
```

Expected output:

```
nginx is running
payments-api.service is running
All services are healthy
Exit code: 0
```

Test 2: Stop a service (should return 1)

```
sudo systemctl stop payments-api.service
~/devops-tasks/scripts/check-services.sh
echo "Exit code: $?"
```

Expected output:

```
nginx is running
```

```
✗ payments-api.service is DOWN
ERROR: 1 service(s) down: payments-api.service
Exit code: 1
```

Test 3: Restart service and verify it works again

```
sudo systemctl start payments-api.service
~/devops-tasks/scripts/check-services.sh
echo "Exit code: $?"
```

Expected output:

```
✓ nginx is running
✓ payments-api.service is running
All services are healthy
Exit code: 0
```

Step 5: Verify exit codes work with monitoring tools

Test with **&&** (runs only if exit code is 0):

```
~/devops-tasks/scripts/check-services.sh && echo "Monitoring: All
services OK"
```

Test with **||** (runs only if exit code is non-zero):

```
sudo systemctl stop payments-api.service
~/devops-tasks/scripts/check-services.sh || echo "Monitoring: ALERT -
Service down!"
sudo systemctl start payments-api.service
```

Acceptance Criteria

Check all boxes:

- ☐ Script created at `~/devops-tasks/scripts/check-services.sh` and executable
- ☐ Checks nginx service status
- ☐ Checks payments-api.service status
- ☐ Returns exit code `0` when all services are running
- ☐ Returns exit code `1` when any service is down
- ☐ Prints clear status messages (`running`, `DOWN`)
- ☐ Tested with all services running (exit code `0`)
- ☐ Tested with one service stopped (exit code `1`)
- ☐ Exit codes work correctly with `&&` and `||` operators

What You Learned

Technical Skills:

- Writing scripts that return proper exit codes
- Using `systemctl is-active` to check service status
- Using arrays in Bash to loop through services
- Making scripts that integrate with monitoring tools

Real-World Context:

- Exit codes are how scripts communicate with other tools
- Monitoring systems rely on exit codes to trigger alerts
- `0` = success, non-zero = failure (standard Unix convention)
- This script can be added to cron or systemd timers for automatic checking

Practical Application:

- This exact pattern is used in production monitoring
- Monitoring tools (Nagios, Zabbix, Prometheus) all use exit codes
- You can extend this to check databases, disk space, API endpoints, etc.

TASK B3: Write a Log Rotation Script with Dry Run Mode

Business Context

Ticket ID: DEVOPS-1598

Priority: Medium

Assignee: You

Situation

The Problem: In Task B1 you wrote a script that compresses old logs into `.gz` files. Over time, those compressed files pile up too. A server might have hundreds of `.gz` files from months ago, and they still use disk space. If nobody deletes them, the disk can fill up and break the application.

What You're Building: A script that finds compressed log files (`.gz`) older than 30 days, deletes them to free space, and always lets you preview what would be deleted first with a `--dry-run` flag.

Why "Dry Run" Matters:

Without dry-run (risky):

- You run the script → it deletes files immediately
- If the script has a bug or wrong path, you lose files before you notice

With dry-run (safe):

- You run: `./rotate-logs.sh --dry-run` → it only prints what it *would* delete
- No files are removed
- You check the list, fix config if needed, then run without `--dry-run` when you're sure

Example:

```
# Monday: You run with --dry-run to see what would happen
$ sudo ./rotate-logs.sh --dry-run
Would delete: /var/log/test-rotation/very-old.log.gz (0.06MB)
Would delete: /var/log/test-rotation/ancient.log.gz (0.04MB)
Summary: Found 2 files totaling 0.10MB. No files deleted (dry-run
mode)

# You verify: "Yes, those are old test files, safe to remove."

# Tuesday: You run for real (no --dry-run)
$ sudo ./rotate-logs.sh
Deleting: /var/log/test-rotation/very-old.log.gz
Deleting: /var/log/test-rotation/ancient.log.gz
Summary: Deleted 2/2 files, freed 0.10MB
```

Why This Matters

Before (No Rotation):

- Compressed logs from B1 keep accumulating
- Disk usage grows until it hits 100%
- Application or system can crash when disk is full

After (With Your Script):

- Old **.gz** files older than 30 days are removed automatically
- Disk space is freed on a schedule
- Dry-run lets you preview and test without risk

Real-World Pattern:

- Tools like **rsync --dry-run**, **git clean -n** (dry-run) use the same idea
- In production, always test destructive operations with a dry-run first

What You'll Actually Do

You're writing a script that:

1. Finds `.gz` files in `/var/log` older than 30 days
2. With `--dry-run`: only lists what would be deleted and the total size
3. Without `--dry-run`: deletes those files and reports space freed
4. Logs all actions with timestamps to `/var/log/rotate-logs-script.log`

Your Mission

Write a script that:

- Finds compressed log files (`*.gz`) in `/var/log` older than 30 days
- Supports `--dry-run`: show what would be deleted without deleting
- Without `--dry-run`: deletes those files and frees disk space
- Logs every action with timestamps
- Reports how many files and how much space (in MB)

Deliverables (What You'll Create)

Main Artifact:

- Script: `~/devops-tasks/scripts/rotate-logs.sh`
 - Must be executable
 - Must support `--dry-run` and normal (delete) mode
 - Must log to `/var/log/rotate-logs-script.log`

Supporting:

- Test run in dry-run mode (proof nothing is deleted)
- Test run in real mode (proof files are deleted and space freed)
- Idempotency: running again finds 0 files when nothing is left to delete

Step-by-Step Guide

Step 1: Create the script

```
mkdir -p ~/devops-tasks/scripts
vim ~/devops-tasks/scripts/rotate-logs.sh
```

Step 2: Write the script

```
#!/bin/bash
#
# rotate-logs.sh - Delete old compressed log files (.gz) older than
# 30 days
#
# Usage:
#   sudo ./rotate-logs.sh          # Actually delete files
#   sudo ./rotate-logs.sh --dry-run # Only show what WOULD be deleted
#
set -euo pipefail

# Configuration
readonly LOG_DIR="/var/log"
readonly AGE_DAYS=30
readonly SCRIPT_LOG="/var/log/rotate-logs-script.log"

DRY_RUN=false

# Log with timestamp
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | sudo tee -a
"$SCRIPT_LOG"
}

# Parse arguments: $1 = first argument (e.g. --dry-run)
parse_arguments() {
    if [ $# -gt 0 ] && [ "$1" = "--dry-run" ]; then
        DRY_RUN=true
        log "DRY RUN MODE: No files will be deleted"
    fi
}
```

```

    fi
}

# Must run as root (we touch /var/log)
check_root() {
    if [ $EUID -ne 0 ]; then
        echo "ERROR: This script must be run as root (sudo)"
        exit 1
    fi
}

# Get file size in bytes (Linux: -c%s; macOS: -f%z)
get_file_size() {
    local file="$1"
    stat -c%s "$file" 2>/dev/null || stat -f%z "$file" 2>/dev/null ||
echo "0"
}

# Find and delete (or dry-run) old .gz files
rotate_logs() {
    log "Searching for .gz files older than $AGE_DAYS days in
$LOG_DIR..."

    local found_count=0
    local deleted_count=0
    local total_size=0

    while IFS= read -r logfile; do
        found_count=$((found_count + 1))
        local file_size=$(get_file_size "$logfile")
        total_size=$((total_size + file_size))
        local size_mb=$(echo "scale=2; $file_size / 1048576" | bc)

        if [ "$DRY_RUN" = true ]; then
            log "Would delete: $logfile (${size_mb}MB)"
        else
            log "Deleting: $logfile"
            if rm "$logfile"; then
                deleted_count=$((deleted_count + 1))
            fi
        fi
    done
}

```



```

        else
            log "ERROR: Failed to delete $logfile"
        fi
    fi
done < <(find "$LOG_DIR" -name "*.gz" -type f -mtime +$AGE_DAYS)

local total_mb=$(echo "scale=2; $total_size / 1048576" | bc)

if [ "$DRY_RUN" = true ]; then
    log "Summary: Found $found_count files totaling
    ${total_mb}MB. No files deleted (dry-run mode)"
else
    log "Summary: Deleted $deleted_count/$found_count files,
    freed ${total_mb}MB"
fi
}

main() {
    parse_arguments "$@"
    log "==== Log Rotation Started ====="
    check_root
    log "Config: LOG_DIR=$LOG_DIR, AGE_DAYS=$AGE_DAYS,
    DRY_RUN=$DRY_RUN"
    rotate_logs
    log "==== Log Rotation Completed ====="
}

main "$@"

```

Save: :wq

Step 3: Make executable

```
chmod +x ~/devops-tasks/scripts/rotate-logs.sh
```

Step 4: Create test files

Why: So you can see dry-run vs real run without touching real logs.

```
# Directory for testing only
sudo mkdir -p /var/log/test-rotation

# Files "40 days old" (will be deleted: older than 30 days)
sudo touch -t $(date -d '40 days ago' +%Y%m%d0000)
/var/log/test-rotation/very-old.log.gz
sudo touch -t $(date -d '40 days ago' +%Y%m%d0000)
/var/log/test-rotation/ancient.log.gz

# File "20 days old" (will NOT be deleted: younger than 30 days)
sudo touch -t $(date -d '20 days ago' +%Y%m%d0000)
/var/log/test-rotation/recent.log.gz

# Add content so they have size
echo "old data" | gzip | sudo tee
/var/log/test-rotation/very-old.log.gz > /dev/null
echo "old data" | gzip | sudo tee
/var/log/test-rotation/ancient.log.gz > /dev/null
echo "recent" | gzip | sudo tee /var/log/test-rotation/recent.log.gz
> /dev/null

# Check
ls -lh /var/log/test-rotation/
```

Expected:

```
-rw-r--r-- 1 root root 45 ... ancient.log.gz
-rw-r--r-- 1 root root 38 ... recent.log.gz
-rw-r--r-- 1 root root 61 ... very-old.log.gz
```

Step 5: Test dry-run (nothing deleted)

```
sudo ~/devops-tasks/scripts/rotate-logs.sh --dry-run
```

Expected:

```
[2025-12-04 16:00:00] ===== Log Rotation Started =====
[2025-12-04 16:00:00] DRY RUN MODE: No files will be deleted
[2025-12-04 16:00:00] Config: LOG_DIR=/var/log, AGE_DAYS=30, DRY_RUN=true
[2025-12-04 16:00:00] Searching for .gz files older than 30 days in
/var/log...
[2025-12-04 16:00:00] Would delete: /var/log/test-rotation/very-old.log.gz
(0.00MB)
[2025-12-04 16:00:00] Would delete: /var/log/test-rotation/ancient.log.gz
(0.00MB)
[2025-12-04 16:00:00] Summary: Found 2 files totaling 0.00MB. No files
deleted (dry-run mode)
[2025-12-04 16:00:00] ===== Log Rotation Completed =====
```

Verify files still exist:

```
ls /var/log/test-rotation/
# Should show: ancient.log.gz recent.log.gz very-old.log.gz
```

Step 6: Run for real (actual deletion)

```
sudo ~/devops-tasks/scripts/rotate-logs.sh
```

Expected (example):

```
[2025-12-04 16:01:00] Deleting:
/var/log/test-rotation/very-old.log.gz
[2025-12-04 16:01:00] Deleting: /var/log/test-rotation/ancient.log.gz
[2025-12-04 16:01:00] Summary: Deleted 2/2 files, freed 0.00MB
```

Check what remains:

```
ls -lh /var/log/test-rotation/
# Only recent.log.gz (20 days old, under 30) should remain
```

Step 7: Test idempotency

Run again; there should be nothing left to delete:

```
sudo ~/devops-tasks/scripts/rotate-logs.sh
```

Expected :

```
Summary: Deleted 0/0 files, freed 0.00MB
```

Step 8: Clean up test directory

```
sudo rm -rf /var/log/test-rotation
```

Understanding the Pieces

Command-line arguments (`$1`, `$#`, `$@`):

- `$1` = first argument (e.g. `--dry-run`)
- `$#` = number of arguments
- `$@` = all arguments (used in `main "$@"`)

`bc` for decimals:

- Bash does only integer math: `echo $((10/3))` → 3
- `echo "scale=2; 10/3" | bc` → 3.33
- 1048576 bytes = 1 MB; we use this to convert bytes to MB.

Dry-run pattern:

- If `DRY_RUN=true`: print "Would delete: ..." and do not run `rm`
- If `DRY_RUN=false`: run `rm` and log "Deleting: ..."

Acceptance Criteria

Check all boxes:

- Script at `~/devops-tasks/scripts/rotate-logs.sh` and executable
- Finds `.gz` files in `/var/log` older than 30 days
- `--dry-run` only lists files and sizes; does not delete
- Without `--dry-run`, deletes those files and logs each deletion
- Logs all actions with timestamps to `/var/log/rotate-logs-script.log`
- Reports count and space (e.g. "Deleted 2/2 files, freed 0.10MB")
- Tested dry-run: files still present after run
- Tested real run: only files older than 30 days removed; recent kept
- Idempotent: second run finds 0 files when none left

What You Learned

Technical:

- Parsing flags: `[ "$1" = "--dry-run" ]`
- `find ... -mtime +30` for "older than 30 days"
- `bc` for decimal math and byte-to-MB conversion
- `stat -c%s` (Linux) / `stat -f%z` (macOS) for file size

Safety:

- Always offer dry-run for delete/cleanup scripts
- Test with dry-run first, then run for real
- Log actions for an audit trail

Real-world:

- Same pattern as `rsync --dry-run`, `git clean -n`, many SaaS "preview" buttons
- Production scripts that remove data should have a dry-run or confirmation step

TASK B4: Improve the Health Check Script

Business Context

Ticket ID: DEVOPS-1621

Priority: Low

Assignee: You

Situation

The Problem: You already have a health check script (from L3 or earlier) that checks nginx and payments-api. It works, but it's one flat script: same output every time, no way to get more detail when debugging or less noise when run by a monitoring tool. As you add more checks (disk, memory, etc.), it gets harder to read and to reuse.

What You're Building: An improved version of the health check script that:

- Is split into **functions** (one per check) so it's easier to read and change
- Supports **modes** via flags:
 - **Normal:** Usual summary (services, disk, memory)
 - **--verbose:** Extra DEBUG lines for troubleshooting
 - **--quiet:** Only print when something is wrong (for cron/monitoring)
 - **--help:** Show usage and exit codes
- Uses **log levels** (INFO, ERROR, DEBUG) so you can filter what gets printed
- Returns **proper exit codes** (0 = all OK, 1 = something failed) for monitoring

How It Works:

Mode	When to use	What you see
Normal	Manual check	INFO lines:  /  for services, disk, memory
--verbose	Debugging a failure	INFO + DEBUG (e.g. "Checking service: nginx")

--quiet	Cron or monitoring	Only ERROR lines (and exit code 1 if any)
--help	Forget the options	Usage and exit code explanation

Example:

```
# You're debugging: "Why did the health check fail?"
$ ./health-check-v2.sh --verbose
[2025-12-04 10:00:00] Health Check Started
[2025-12-04 10:00:00] === Checking Services ===
[2025-12-04 10:00:00] Checking service: nginx ← DEBUG
[2025-12-04 10:00:00] ✓ Service: nginx is RUNNING
[2025-12-04 10:00:00] Checking service: payments-api ← DEBUG
[2025-12-04 10:00:00] ✗ Service: payments-api is DOWN
...

# Monitoring runs it every 5 min; we only want alerts when something breaks
$ ./health-check-v2.sh --quiet
# (no output when all healthy)

$ sudo systemctl stop nginx && ./health-check-v2.sh --quiet
[2025-12-04 10:05:00] ✗ Service: nginx is DOWN
[2025-12-04 10:05:00] Health Check: ISSUES DETECTED ✗
```

Impact

Before (Basic Script):

- One big block of code; hard to add disk or memory checks
- Same output for humans and for cron; noisy in logs
- No --help; you have to remember or read the file

After:

- **Modular:** Each check is a function; easy to add/remove checks

- **Flexible:** `--verbose` for debugging, `--quiet` for automation
- **Clear:** `--help` explains options and exit codes
- **Monitoring-friendly:** Exit 0/1 and `--quiet` work with cron and alerting

Real-World Pattern:

- Many CLI tools use `--verbose`, `--quiet`, `--help` (e.g. `docker`, `kubectl`, `aws`)
- Log levels (INFO/ERROR/DEBUG) match how production logging works

What You'll Actually Do

You will:

1. Copy the existing `health-check.sh` to `health-check-v2.sh`
2. Split logic into functions: `parse_arguments`, `log`, `check_service`, `check_all_services`, `check_disk_usage`, `check_memory_usage`, `main`
3. Add `--verbose`, `--quiet`, `--help`
4. Make `log` respect levels: in `--quiet`, only ERROR is printed to the console; in `--verbose`, DEBUG is also printed
5. Keep exit code 0 when all pass, 1 when any check fails

Your Mission

Improve the health-check script by:

- Breaking it into **modular functions** (one per check and for parsing/logging)
- Adding **CLI options:** `--verbose`, `--quiet`, `--help`
- Implementing **log levels:** INFO, ERROR, DEBUG (and control what is printed)
- Returning **correct exit codes:** 0 = all healthy, 1 = one or more failures

Deliverables (What You'll Create)

Main Artifact:

- **Script:** `~/devops-tasks/scripts/health-check-v2.sh`
 - Must be executable
 - Must support: default, `--verbose`, `--quiet`, `--help`

Behavior:

- **Normal:** INFO messages for each check and final summary
- **--verbose:** INFO + DEBUG (e.g. "Checking service: X", "Current disk usage: Y%")
- **--quiet:** Only ERROR lines to console; still writes all levels to log file
- **--help:** Usage, options, and exit codes; then exit 0

Step-by-Step Guide

Step 1: Find and copy the original

```
cat /usr/local/bin/health-check.sh
# If it doesn't exist, use any simple health-check you have, or the
one from L3.
```

```
mkdir -p ~/devops-tasks/scripts
cp /usr/local/bin/health-check.sh
~/devops-tasks/scripts/health-check-v2.sh
```

If `/usr/local/bin/health-check.sh` is missing, create a minimal starting point:

```
cat > ~/devops-tasks/scripts/health-check-v2.sh << 'EOF'
#!/bin/bash
# Placeholder: will be replaced in Step 3
echo "Replace this with the full script from Step 3"
EOF
```

Step 2: Open for editing

```
vim ~/devops-tasks/scripts/health-check-v2.sh
```

Step 3: Write the improved script

```
#!/bin/bash
#
# health-check-v2.sh - Health check with --verbose, --quiet, --help
#
# Usage:
#   ./health-check-v2.sh           # Normal
#   ./health-check-v2.sh --verbose # Show DEBUG
#   ./health-check-v2.sh --quiet   # Only errors to console
#   ./health-check-v2.sh --help    # Usage and exit codes
#
set -euo pipefail

readonly SERVICES=("nginx" "payments-api.service")
readonly DISK_THRESHOLD=80
readonly MEMORY_THRESHOLD=90
readonly LOG_FILE="/var/log/health-check.log"

VERBOSE=false
QUIET=false
EXIT_CODE=0

usage() {
    cat << EOF
Usage: $0 [OPTIONS]
OPTIONS:
    --verbose    Show detailed DEBUG output
    --quiet      Show only errors (for cron/monitoring)
    --help       Show this help
EXIT CODES:
    0    All checks passed
    1    One or more checks failed
EOF
    exit 0
}

parse_arguments() {
    while [ $# -gt 0 ]; do
        case "$1" in
            --verbose) VERBOSE=true; shift ;;
            --quiet)   QUIET=true;   shift ;;
        esac
    done
}
```

```

        --help)    usage ;;
        *)        echo "ERROR: Unknown option: $1"; exit 1 ;;
    esac
done
}

# level: INFO | ERROR | DEBUG
log() {
    local level="$1"
    local message="$2"
    local timestamp="[$(date '+%Y-%m-%d %H:%M:%S')]"

    # Always append to log file (with level)
    echo "$timestamp [$level] $message" | sudo tee -a "$LOG_FILE" >
/dev/null

    # Console: --quiet only shows ERROR; default shows INFO+ERROR;
--verbose adds DEBUG
    if [ "$QUIET" = true ] && [ "$level" != "ERROR" ]; then
        return
    fi
    if [ "$level" = "DEBUG" ] && [ "$VERBOSE" != true ]; then
        return
    fi
    echo "$timestamp $message"
}

check_service() {
    local service_name="$1"
    log "DEBUG" "Checking service: $service_name"

    if systemctl is-active --quiet "$service_name"; then
        log "INFO" "✅ Service: $service_name is RUNNING"
        return 0
    else
        log "ERROR" "❌ Service: $service_name is DOWN"
        EXIT_CODE=1
        return 1
    fi
}

check_all_services() {
    log "INFO" "=== Checking Services ==="

```

```

    for service in "${SERVICES[@]"; do
        check_service "$service"
    done
}

check_disk_usage() {
    log "INFO" "=== Checking Disk Usage ==="
    local disk_usage=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')
    log "DEBUG" "Current disk usage: ${disk_usage}%"

    if [ "$disk_usage" -gt "$DISK_THRESHOLD" ]; then
        log "ERROR" "⚠️ Disk usage: ${disk_usage}% (threshold: ${DISK_THRESHOLD}%)"
        EXIT_CODE=1
    else
        log "INFO" "✅ Disk usage: ${disk_usage}%"
    fi
}

check_memory_usage() {
    log "INFO" "=== Checking Memory Usage ==="
    local memory_usage=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
    log "DEBUG" "Current memory usage: ${memory_usage}%"

    if [ "$memory_usage" -gt "$MEMORY_THRESHOLD" ]; then
        log "ERROR" "⚠️ Memory usage: ${memory_usage}% (threshold: ${MEMORY_THRESHOLD}%)"
        EXIT_CODE=1
    else
        log "INFO" "✅ Memory usage: ${memory_usage}%"
    fi
}

main() {
    parse_arguments "$@"
    sudo touch "$LOG_FILE" 2>/dev/null || true

    log "INFO" "Health Check Started"
    check_all_services
    check_disk_usage
    check_memory_usage

    if [ $EXIT_CODE -eq 0 ]; then

```

```

        log "INFO" "Health Check: ALL SYSTEMS HEALTHY ✅"
    else
        log "ERROR" "Health Check: ISSUES DETECTED ❌"
    fi

    exit $EXIT_CODE
}

main "$@"

```

Save: `:wq`

Step 4: Make executable

```
chmod +x ~/devops-tasks/scripts/health-check-v2.sh
```

Step 5: Test all modes

Help:

```
~/devops-tasks/scripts/health-check-v2.sh --help
```

Expected Outputs:

```

Usage: ./health-check-v2.sh [OPTIONS]
OPTIONS:
    --verbose    Show detailed DEBUG output
    --quiet      Show only errors (for cron/monitoring)
    --help       Show this help
EXIT CODES:
    0    All checks passed
    1    One or more checks failed

```

Normal (all healthy):

```
~/devops-tasks/scripts/health-check-v2.sh
echo "Exit code: $?"
```

Expected:

```
[2025-12-04 10:00:00] Health Check Started
[2025-12-04 10:00:00] === Checking Services ===
[2025-12-04 10:00:00] ✓ Service: nginx is RUNNING
[2025-12-04 10:00:00] ✓ Service: payments-api.service is RUNNING
[2025-12-04 10:00:00] === Checking Disk Usage ===
[2025-12-04 10:00:00] ✓ Disk usage: 45%
[2025-12-04 10:00:00] === Checking Memory Usage ===
[2025-12-04 10:00:00] ✓ Memory usage: 62%
[2025-12-04 10:00:00] Health Check: ALL SYSTEMS HEALTHY ✓
Exit code: 0
```

Verbose (adds DEBUG):

```
~/devops-tasks/scripts/health-check-v2.sh --verbose
```

Expected: Same as normal, plus lines like:

```
[2025-12-04 10:00:00] Checking service: nginx
[2025-12-04 10:00:00] Checking service: payments-api.service
[2025-12-04 10:00:00] Current disk usage: 45%
[2025-12-04 10:00:00] Current memory usage: 62%
```

Quiet, all healthy (nothing to console):

```
~/devops-tasks/scripts/health-check-v2.sh --quiet
echo "Exit code: $?"
# No output; Exit code: 0
```

Quiet, with a failure (only errors):

```
sudo systemctl stop nginx
~/devops-tasks/scripts/health-check-v2.sh --quiet
echo "Exit code: $?"
sudo systemctl start nginx
```

Expected:

```
[2025-12-04 10:05:00] ✗ Service: nginx is DOWN  
[2025-12-04 10:05:00] Health Check: ISSUES DETECTED ✗  
Exit code:
```

Step 6: Check log file

All levels are always written to the log; --quiet only affects the console.

```
sudo tail -30 /var/log/health-check.log
```

You should see `[INFO]`, `[ERROR]`, and when run with `--verbose`, `[DEBUG]`.

Understanding the Pieces

case for options:

- `case "$1" in ... esac` matches `$1` against `--verbose`, `--quiet`, `--help`
- `shift` removes the first argument so the next `$1` is the next option (if you later support multiple flags)

Log levels and modes:

- **--quiet:** In `log()`, if `QUIET=true` and level is not `ERROR`, `return` without printing to console.
- **DEBUG:** In `log()`, if level is `DEBUG` and `VERBOSE` is not true, `return` without printing to console.
- Log file: we always `tee -a` so the file has `INFO`, `ERROR`, and `DEBUG` regardless of mode.

Exit code:

- `EXIT_CODE=0` at start; any failing check sets `EXIT_CODE=1`. `main` ends with `exit $EXIT_CODE` so monitoring gets 0 or 1.

Acceptance Criteria

Check all boxes:

- ☐ Script at `~/devops-tasks/scripts/health-check-v2.sh` and executable
- ☐ Split into functions: `parse_arguments`, `log`, `check_service`, `check_all_services`, `check_disk_usage`, `check_memory_usage`, `main`
- ☐ Normal mode: INFO lines for services, disk, memory and final summary
- ☐ `--verbose`: also shows DEBUG (e.g. "Checking service: X", "Current disk usage: Y%")
- ☐ `--quiet`: only ERROR lines to console when there is a failure; nothing when all healthy
- ☐ `--help`: prints usage, options, and exit codes; then exits 0
- ☐ Exit 0 when all checks pass, exit 1 when any check fails
- ☐ All levels (INFO, ERROR, DEBUG) written to `/var/log/health-check.log`

What You Learned

Technical:

- `case "$1" in ... esac` and `shift` for parsing options
- Central `log(level, message)` to control console vs log file
- Using a global `EXIT_CODE` so any function can mark failure while `main` does a single `exit`

Design:

- `--verbose` / `--quiet` / `--help` is a common CLI pattern
- Log levels (INFO, ERROR, DEBUG) mirror production logging
- One function per check makes it easy to add or remove checks

Real-world:

- Monitoring and cron often need `--quiet` and correct exit codes
- `--verbose` helps when debugging why a check failed without changing the script